UNDERWATER MARINE LIFE AND PLASTIC WASTE DETECTION AND CONTROL OF ROV

A Project Report Submitted By

RAHUL HEGDE	**4NM16EC101**
SANOBAR PATEL	**4NM16EC125**
ROSHA G. NAIK	**4NM17EC432**
SAGAR K. N.	**4NM17EC433**

Under the Guidance of

Dr. K. S. Shivaprakasha
Professor

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Engineering
in
Electronics and Communication Engineering

from
Visvesvaraya Technological University, Belagavi

NITTE EDUCATION TRUST | NMAM INSTITUTE OF TECHNOLOGY

Nitte-574110, Karnataka, India

July-2020

NITTE EDUCATION TRUST | N.M.A.M. INSTITUTE OF TECHNOLOGY
(An Autonomous Institution affiliated to Visvesvaraya Technological University, Belagavi)
Nitte – 574 110, Karnataka, India
ISO 9001:2015 Certified, Accredited with 'A' Grade by NAAC
☎ :08258 - 281039 – 281263, Fax: 08258 – 281265

Department of Electronics and Communication Engineering

Certificate

Certified that the project work entitled ***Underwater Marine Life and Plastic Waste Detection and Control of ROV*** *is a bonafide work carried out by* ***Rahul Hegde (4NM16EC101), Sanobar Patel (4NM16EC125), Rosha G. Naik (4NM17EC432) & Sagar K. N. (4NM17EC433)*** *in partial fulfillment of the requirements for the award of* ***Bachelor of Engineering Degree in Electronics and Communication Engineering*** *prescribed by Visvesvaraya Technological University, Belagavi during the year 2019-2020. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the Bachelor of Engineering Degree.*

Signature of the Guide **Signature of the HOD**

Signature of the Principal

Semester End Viva Voce Examination

Name of the Examiners	Signature With Date
1. _____	_____
2. _____	_____

Abstract

Computer vision techniques have become increasingly popular in recent times. One of the applications of computer vision is the detection of objects in an image. In an object detection system, a target object is detected, localized and labelled on the basis of the type of object. Detection of objects in harsh environments; accuracy in the detected objects and speed at which they are detected are some of the major challenges in computer vision technology. Underwater imaging has its own set of difficulties namely: reduced visibility; objects possessing highly deformed shapes (plastic bags); means of communication of the captured image for proper processing. In this work, development of a system to detect marine life and plastic wastes in underwater environments is proposed and the designed Remotely Operated Vehicle (ROV) is controlled accordingly by wireless communication. Deep learning technique is used and applied on the images for detection and localization of the objects.

Acknowledgement

Our project would not have been successful without the encouragement, guidance and support by various personalities. First and foremost, we would like to express our sincere gratitude towards our project guide **Dr. K. S. Shivaprakasha**, Professor, Department of Electronics and Communication Engineering, N. M. A. M. Institute of Technology, Nitte, for his guidance, encouragement and inspiration throughout the project work.

We extend our gratitude to **Dr. Rekha Bhandarkar**, Professor and Head, Department of Electronics and Communication Engineering, N. M. A. M. Institute of Technology, Nitte, for her encouragement and providing necessary facilities in the department.

We wish to acknowledge the support of **Dr. Niranjan N. Chiplunkar,** Principal, N. M. A. M. Institute of Technology, Nitte, for providing motivational academic environment.

I would also like to thank **Mr. Anil Kumar Bhat**, **Dr. Ananth A.G.**, **Dr. Subramanya Bhat** for their advice and help in the implementation of the project.

I also thanks **Dr. Adrian Rosebrock** from pyimagesearch, **Nathaniel O Solomon**, **Edje Electronics**, YouTube Channel for the online tutorials on machine learning. We would like to express our sincere thanks to all the teaching and non-teaching staff of the department of Electronics and Communication Engineering, N. M. A. M. Institute of Technology, Nitte, for their patience and help during our project work.

A heart filled thanks to our family who supported and helped us throughout the project. Finally, we would like to thank all our friends and well-wishers who have helped us whenever needed and supported us.

Rahul Hegde
Sanobar Patel
Rosha G. Naik
Sagar K. N.

Table of Contents

Abstract i

Acknowledgement iii

Table of Contents v

List of Figures ix

List of Tables xi

1 Introduction 1

1.1 Aim . . . 1
1.2 Objectives . . . 1
1.3 Problem Formulation . . . 2
1.4 Proposed Method . . . 2
1.5 Methodology . . . 2
1.6 Literature Survey . . . 3
1.6.1 YOLO . . . 3
1.6.2 Faster R-CNN . . . 3
1.6.3 Viola Jones Algorithm . . . 4
1.6.4 Single Shot Detector . . . 4
1.7 Organization of the Report . . . 5

2 System Architecture 7

2.1 General System Description . . . 7
2.2 Block Diagram Description . . . 7
2.2.1 Raspberry Pi camera . . . 7
2.2.2 Neo 6 GPS module . . . 8
2.2.3 Raspberry Pi . . . 8
2.3 Process Flow . . . 8
2.4 Theoretical Background . . . 10
2.4.1 Object Detection . . . 10
2.4.2 Deep Learning . . . 10
2.4.3 Single Short Detector . . . 11
2.4.4 Transfer learning . . . 12
2.4.5 Global Positioning System . . . 12
2.4.6 NMEA Sentences . . . 12

2.4.7 Tensorflow 13
2.4.8 Keras 13
2.4.9 Object detection Evaluation Parameters 13

3 Hardware and Software Description **15**
3.1 Hardware Description 15
3.1.1 Raspberry Pi 15
3.1.2 Neo6M GPS Module 16
3.1.3 Raspberry Pi Camera 16
3.1.4 SD Card 17
3.2 Software Description 17
3.2.1 Python 17
3.2.2 Open CV 17
3.2.3 LabelImg 18
3.2.4 Rasberry Pi OS (or Raspbian OS) 18
3.2.5 Google Colab 18

4 Implementation **19**
4.1 Collection of Images and Data Augmentation 19
4.2 Image annotation 19
4.3 Generation of CSV and TFRecords File 20
4.4 Configuring the Training Pipeline 20

5 Algorithms **23**
5.1 Image Resizing 23
5.2 Data Augmentation 25
5.3 Object Detection Process Flow 29
5.4 GPS Coordinates Acquisition Process Flow 30

6 Testing and Results **33**
6.1 Testing for the Acquisition of GPS Coordinates 33
6.2 Testing for Object Detection in an Image 33
6.3 Results 34
6.3.1 GPS Coordinates Acquisition Results 34
6.3.2 Object Detection Results 35

7 Conclusions and Future Work **37**
7.1 Conclusion 37
7.2 Future Work 37

Bibliography **39**

A Papers Published/Presented **43**

B Certificates **55**

List of Figures

2.1 General Block Diagram 7
2.2 General Process Flow 9
2.3 SSD Architecture 11

3.1 Raspberyy Pi 3B+ Model 15
3.2 Neo 6M GPS Module 16
3.3 Raspberry Pi Camera V1 model 16
3.4 Sandisk 32 GB SD Card 17

5.1 Flowchart of image resizing process 24
5.2 Flowchart of data augmentation process 28
5.3 Flowchart of object detection process 29
5.4 Flowchart of GPS coordinates update process 30

6.1 GPS Coordinates results 34
6.2 Object Dectection Results (Images) 35

List of Tables

1.1 Comparison between Faster R-CNN and YOLOv3. 5
1.2 Comparison between YOLOv3 and SSD. 5
1.3 Parameters of Viola-Jones algorithm. 5
1.4 Parameters of YOLOv3 algorithm 5

6.1 Results of testing of images for object detection 36
6.2 Results of testing of images for object detection calculation 36

Chapter 1
Introduction

Machine learning for object detection has been used since a long time. Object detection is an application of computer vision. In this method, a target object is to be detected by means of a computer algorithm, which is the first phase of object detection. The object is then localized and the class of the object is determined. Several algorithms have been developed to give a better result in object detection. Each algorithm has its own advantages and disadvantages. Here, the object detection algorithm is used for the detection of marine life and underwater plastic waste.

Waste management has been a major issue for mankind since ages. In the current times clearing out accumulated waste is a major challenge. Clearing of underwater waste raises the difficulty of this situation. Detection of plastic waste underwater is hard due to the harsh environment and challenges such as reduced visibility, the water current, the nature of the plastic waste etc. Computer vision has aided in this process as it provides algorithms for effective image processing and machine learning.

Bots have been used for survey purposes. The images acquired by the bots are transmitted for processing. This procedure is done by wireless communication. The ROV is the carrier of the camera and the transmission takes place from the bot to an off-shore unit where the image is processed. The control signals are issued to the bot by wireless means which are processed by a microcontroller unit. The position of the ROV is acquired in the form of Global Positioning System (GPS) coordinates.

1.1 Aim

To detect plastic bags, plastic bottles, bottle caps and marine life underwater using deep learning. To acquire the GPS coordinates of the boat connected to the ROV by means of a GPS module.

1.2 Objectives

1. Detection of plastic bags, plastic bottles, bottle caps and marine life underwater using deep learning technique. The deep learning technique to be used is SSD MobileNet Quantized V2 Model to train the object detector.

2. The GPS coordinates are to be acquired by means of a GPS module. The acquired results are to be stored in the Raspberry Pi in the form of a text file.

1.3 Problem Formulation

Underwater waste is hazardous to both marine and plant life. Efforts have been made in order to remove underwater waste. The primary procedure in this process would be to survey the area under inspection. A survey would determine the type of waste present underwater and may also give information about the quantity. Underwater ROV's act as an excellent means of survey. They can detect and localize objects in an underwater environment using deep learning techniques.

1.4 Proposed Method

SSD Mobilenet V2 Quantized model is used for training the datasets for object detection. It is used as its performance is superior to other deep learning models in low power computing devices such as the Raspberry Pi. Python is used to implement the scripts for image resizing, data augmentations and GPS coordinates acquisition. The GPS coordinates are acquired by Neo 6M GPS module. Pi camera is used to capture images for object detection. The processing of the captured images and the detection of target objects in the images are done by the raspberry pi.

1.5 Methodology

The system consists of a Raspberry-Pi, Raspberry-Pi camera and GPS module. In this project, some sample studies have been carried out to understand the techniques such as object detection, deep learning algorithm and GPS. Datasets of underwater plastic bags, plastic bottles, bottle caps and marine life have been trained using deep learning algorithm. Images of plastic bags, plastic bottles, bottle caps and mackerel fish have been collected. All the images have been resized to a dimension of 300x300. Data Augmentation has been done in order to increase the number of images. Annotation of images has been done in order to prepare datasets. Object detection is being carried out using Single Short Detection (SSD) algorithm. The Pi camera is used for acquiring the images. The Raspberry-Pi is used to implement the object detection algorithm and stores the result in a directory. The location co-ordinates of the ROV is acquired by a GPS and stored in a file in the Raspberry-Pi.

1.6 Literature Survey

Various machine learning algorithms have been proposed in the literature for the detection of objects in the underwater scenario. A qualitative survey of the algorithms has been carried out in the project and following sections present an overview of the algorithms studied.

1.6.1 YOLO

YOLO is an algorithm proposed in [1], [2], [3], [4], and [5]. In YOLOv3 algorithm the image is resized initially. It then divides the image frame into an M x M block grid of three scales being 13 x 13, 26 x 26 and 52 x 52 [2]. It predicts bounding boxes and probability in each region. The YOLOv3 algorithm predicts the bounding boxes by the centroid co-ordinates of the image sub boxes in the grid. The bounding boxes generated have four co-ordinates (kx, ky, kw, kh), where kx and ky are used to obtain the centroid of the bounding boxes, kw and kh are used to get the width and height of the bounding boxes. Each of the boxes use a simple logistic regression (K - means clustering) for classification and binary cross entropy loss. It predicts through three different scales of boxes [1]. It uses 3 x 3 and 1 x 1 convolution network and short connections between them. YOLOv3 is a hybrid version of YOLOv2 and Darknet 19.

1.6.2 Faster R-CNN

Faster R-CNN [6], [7], [8] consists of two modules namely fully convolutional neural network and the Fast R-CNN [9] network. The RPN is used to generate suggestion box classifiers by running region proposals on the image frame to detect the desired object. The Fast R-CNN module uses the region proposals generated as attention detectors to obtain a shared feature map from the image [3]. An Image is first passed through a convolution layer to generate feature maps. The convolution layer uses convolution non-linear transforms and linear regressions to generate the feature maps. In order to generate the region proposals, the feature maps are then passed on through a region proposal network. The region proposals generated are rectangular boxes. These region proposals are characterized with respect to certain fixed parameters relative to the reference boxes called as anchors [9]. The RPN is used for the generation of suggestion boxes and classifiers.

A Softmax layer [9] is used in order to determine the predicted anchor point for foreground and background by means of boundary regression principle. A Region of Interest (ROI) Pooling Layer combines the feature maps and the suggestion boxes and resizes the suggestion boxes to the same size as that of the feature map.

The full link layer is used for the identification of the desired object. The Classification Layer sends the feature map to the classifier for the classification of the objects based on the received set of object classes [1]. The bounding boxes appear on the desired object to be detected in the image with the classifiers name labelled on to it.

1.6.3 Viola Jones Algorithm

Viola-Jones algorithm [10], [11], was developed for the detection of faces, but recently it has been successfully adopted for detection of other objects. It is a machine learning algorithm recognized by three vital parts. They are learning phase, integral image phase and cascade classifier phase. During the learning phase, a number of Haar features are determined by means of Ada boost Algorithm. Three types of Haar features [10] are used, they are two rectangles, three rectangles and four rectangles. The Haar features have a black region and white region [10]. The difference between the values of the black region and the white region is the Eigen value to be calculated [8]. Images where the object of interest is present are called positive image and the images where the object of interest is not present are called negative images [10]. All positive images are scaled to the dimension of that of the sub windows before the learning process. The dimension of the sub window determines the smallest dimension of the object that can be determined. Cascade classifiers [10], are used to reduce computation time and increase the detection speed. In detection phase a window whose dimensions are already known goes through the whole image step by step and pixel by pixel. The process is repeated until the image dimensions become smaller than the sub image. The image is classified as either positive or negative. The process of classification is continued only if the image is positive. The drawback is the pre-determined dimensions of the object [10], [11].

1.6.4 Single Shot Detector

SSD uses a VGG16 as the base network due to its high-quality image classification, and then convolutional feature layers that gradually reduce in size are added to its end so that it is able to predict object at different scales by the aspect ratio [12]. The core of SSD, by applying small convolutional filters to feature maps, is to predict class scores and offsets for a fixed set of the default bounding box. This model has been improved and implemented by many researchers [13]. For example, instead of taking VGG16 as a feature extractor or object classifier for SSD, ResNet, or MobileNet are utilized by researchers. The YOLO model predicts the confidence score for an object and predicts the probability of each class given that there is an object

existing next, whereas the SSD network does not predict the confidence score for each object, but it gives the probability that a class is present in a given bounding box [14], [12]. Moreover, the YOLO model adopts an intermediate fully connected layer while the SSD replaces it with a convolutional filter [12].

Table 1.1: Comparison between Faster R-CNN and YOLOv3.

Parameter	**Faster R-CNN (in %)**	**YOLOv3 (in %)**
Precision	99.66	99.73
Sensitivity	79.40	99.07
Quality	79.17	98.81
Processing time	1.39s	0.057ms

Table 1.2: Comparison between YOLOv3 and SSD.

Parameter	**YOLOv3**	**SSD**
Accuracy	High	Low
Time	0.84 to 0.9 approximately (second / per frame)	0.17 to 0.23 approximately (second / per Frame)
Speed	Low	High

Table 1.3: Parameters of Viola-Jones algorithm.

Detector Name	**Precision (in %)**	**Recall**	**Frames per Second(FPS)**	**Inference Time**
Pedestrian	80.95	81.60	2.6	0.38
Traffic Light	91.60	86.33	3.6	0.28
Stop Sign	87.20	82.02	9.2	0.11

Table 1.4: Parameters of YOLOv3 algorithm

Detector Name	**Precision (in %)**	**Recall**	**Frames per Second(FPS)**	**Inference Time**
Pedestrian	94.29	92.40	1.2	0.83
Traffic Light	80.69	84.17		
Stop Sign	97.20	89.09		

1.7 Organization of the Report

Chapter 2 explains about the general system architecture of the project.
Chapter 3 outlines the system architecture and the concepts used in the project.

Charter 4 details the hardware and software implementations in the project.
Charter 5 presents the algorithms used in the project.
Chapter 6 presents the results and analysis of the project.
Chapter 7 gives the concluding remarks.

Chapter 2
System Architecture

2.1 General System Description

The main function of the system is to perform object detection of plastic bottles, plastic bags, bottle caps and mackerel in an underwater environment. The image is captured by means of Raspberry Pi camera and is then processed by the Raspberry Pi. The model and labels files generated from the training process to detect the target object in the image is stored in the Raspberry Pi. GPS coordinates of the ROV is acquired by means of Neo 6M GPS module. The GPS coordinates are updated in a file in the raspberry Pi. The image and coordinates are acquired at regular intervals of 3 seconds. Figure 2.1 gives the General Block Diagram of the project.

2.2 Block Diagram Description

Figure 2.1 represents the General Block Diagram of the project.

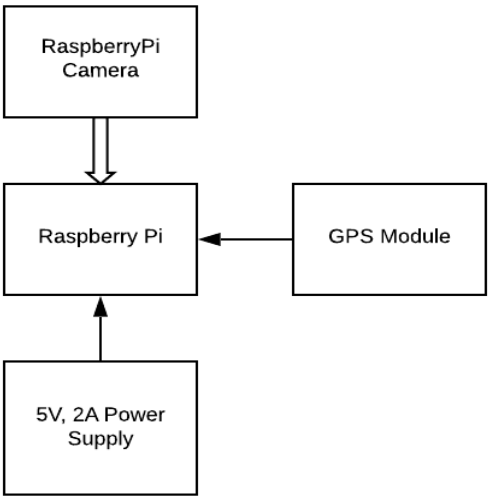


Figure 2.1: General Block Diagram

2.2.1 Raspberry Pi camera

The Raspberry Pi camera is a 5 mega pixel camera. It is specifically suited for use with a Raspberry Pi. It captures images of the surrounding underwater environment and sends the images to the Raspberry Pi for the detection of the target object

in the image.

2.2.2 Neo 6 GPS module

The Neo 6M GPS module is used to acquire the GPS coordinates of the ROV. The GPS module communicates with the Raspberry Pi by means of serial communication. The transmission pin of the GPS module is connected to PIN 14 of the Raspberry Pi. The GPS coordinates are updated at a regular interval of 3 seconds in a file in the Raspberry Pi.

2.2.3 Raspberry Pi

Raspberry Pi looks after the processing and control tasks. It receives the image from the Pi Camera and passes it through the object detection algorithm. The object detection algorithm detects the target object and draws bounding boxes around the detected object. It also receives the coordinates from the GPS module, processes it and stores the coordinates in a file in the raspberry pi. The power requirements of the raspberry pi are 5V, 2A.

2.3 Process Flow

The general process flow is described in Figure 2.2.

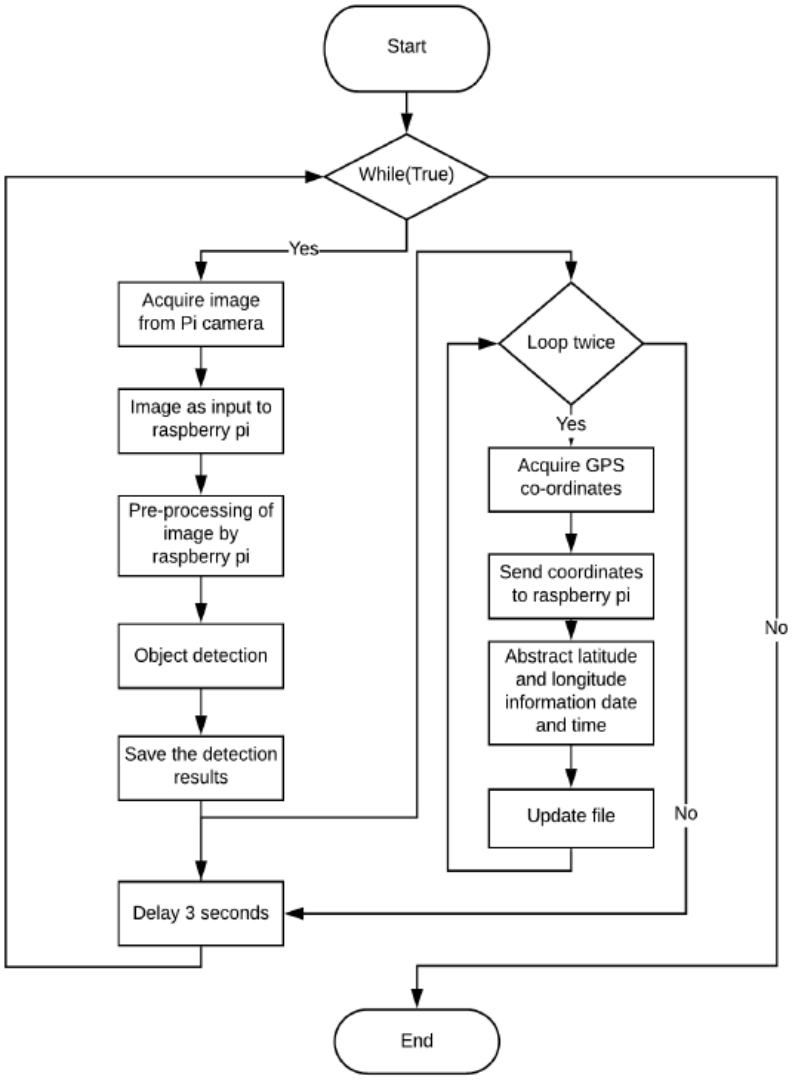


Figure 2.2: General Process Flow

1. The Raspberry Pi camera is used to capture the images from the surrounding environment.

2. The captured image is sent to the Raspberry Pi for the detection of target objects.

3. Neo 6M GPS module is used for the acquisition of GPS coordinates. The GPS coordinates are communicated to the Raspberry Pi via serial communication to be processed and stored in a readable format. The information stored in the file are latitude, longitude, time and date.

4. The image once acquired by the Raspberry Pi is taken through the process of object detection. Here, the image is pre-processed which involves the resizing of the image and conversion of the image into a numpy array.

5. Once the image has completed the process of object detection, the bounding boxes, classes and scores are applied to the image and is stored in a directory.

6. A subprocess takes its effect to process the acquired GPS coordinates. The GPS coordinates are read via the serial communication port of the Raspberry Pi.

7. The GPS coordinates are received in the form of NMEA sentences. It is converted into a readable format by means of pynmea library. It is then updated in a file at a regular interval of 1 seconds. On the completion of processing of GPS coordinated the results are stored and the process returns back to the original flow.

8. The acquisition of image takes place at every 3 seconds.

2.4 Theoretical Background

2.4.1 Object Detection

Object detection is a method in which an object's location in an image is made known by means of bounding boxes. Image classification is used to classify the image as to whether the image consists of a particular class Eg: dog, cat, apple, orange by extracting features from the image. If more than one class is present then the class with the highest score is taken up as the detected class. On the other hand by using object detection, the objects of interest can be identified and localized in the images. It can also localize more than one class in an image. The usual procedure in object detection framework is as follows:

1. Region proposals or Region of interest is generated for an image. These region proposals consist of large number of bounding boxes.

2. Feature extraction is performed on the generated region proposals to obtain the required visual features.

3. In the last step overlapping boxes are combined to a single bounding box. This is known as non-maximum suppression.

2.4.2 Deep Learning

Deep learning is an Artificial Intelligence (AI) function that imitates the workings of the human brain in processing data and creating patterns for use in decision

making. Deep learning is a small part of machine learning in AI that has networks capable of learning unsupervised from data that is unstructured or unlabelled. Deep learning algorithms are trained to not just create patterns from all transactions, but also know when a pattern is signalling the need for a fraudulent investigation. The final layer relays a signal to an analyst who may freeze the user's account until all pending investigations are finalized [15].

2.4.3 Single Short Detector

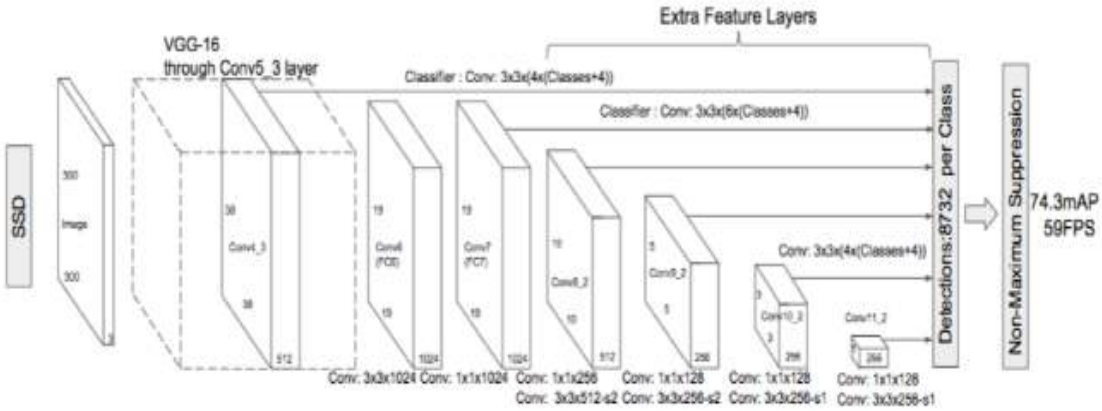


Figure 2.3: SSD Architecture

Single Shot Detector (SSD) uses a VGG16 as its base network for image classification. Then convolutional feature layers that gradually reduce in size are added to its end so that it is able to predict object at different scales by the aspect ratio [1]. In SSD, convolution filters are applied to feature maps to predict the score of a particular class of object and to obtain the offset coordinates of the bounding boxes of the object. Few expressions related to single shot detectors are:

1. Single Shot: The tasks of object localization and classification are done in a single forward pass of the network.

2. MultiBox: This is the name of a technique used for bounding box regression.

3. Detector: The network is an object detector that also classifies those detected object.

VGG-16 has a strong performance in high quality image classification. In an SSD as a substitute to the original VGG fully connected layers, a set of auxiliary convolutional layers were added. This enabled feature extraction at multiple scales and progressively decreased the size of the input to each successive layer.

2.4.4 Transfer learning

Transfer learning is a method in machine learning in which the existing features of a pre-trained model are used during the training procedure of new datasets. Certain features such as contours, light intensity, colours, are common in objects of similar classes. A deep learning network extracts such features in its early layers. For example: when a pre-trained model can detect a flower; features such as the shape of the petals; the stem are nearly common in all flowers and a new dataset of flowers being trained on this model will utilize these features to improve the efficiency of the training. In case the new dataset input to the model is entirely different then the outcome of the training process would be poor. In such cases it is better to build an object detection model from scratch or use a larger training dataset. [16].

2.4.5 Global Positioning System

GPS is a satellite navigation system used to determine the ground position of an object. GPS receivers are included in many commercial products, such as automobiles, smartphones, exercise watches. The GPS system includes 24 satellites deployed in space about at a distance of 19,300 kilometres above the earth's surface. They orbit the earth once every 12 hours at an extremely fast pace of roughly 11,200 kilometres per hour. The satellites are evenly spread out so that a minimum of four satellites are accessible through direct line-of-sight from any location on the globe. A message is broadcasted by each GPS satellite that includes the satellite's current position, orbit, and exact time. A GPS receiver combines the broadcasts from multiple satellites to calculate its exact position using a process called triangulation. Although three satellites are sufficient in order to determine a receiver's location, a connection to four satellites is ideal since it provides greater accuracy.

2.4.6 NMEA Sentences

The National Marine Electronics Association (NMEA) developed a specification that defines the interface between various pieces of marine electronic equipment. GPS receiver communication is defined within this specification. Computers expect real time position data to be in NMEA format. NMEA sends a line of data called a sentence that is totally self-contained and independent from other sentences. All of the standard sentences have a two letter prefix that defines the device that uses that sentence type. For GPS receivers the prefix is GP which is followed by a three letter sequence that defines the sentence contents. Each sentence begins with a '$' and ends with a carriage return/line feed sequence and can be no longer

than 80 characters of visible text including the line terminators. The data is contained within this single line with data items separated by commas. The data itself is just ASCII text and may extend over multiple sentences in certain specialized instances but is normally fully contained in one variable length sentence.
Example: $GPGLL,4916.45,N,12311.12,W,225444,A,*1D

2.4.7 Tensorflow

TensorFlow is an open source library for numerical computation and large-scale machine learning. TensorFlow puts together a slew of machine learning and deep learning models and algorithms and makes them useful by way of a common metaphor. It uses Python to provide a front-end API for building applications with the framework, while executing those applications in high-performance C++.TensorFlow can train and run deep neural networks for handwritten digit classification, image recognition, word embedding's, recurrent neural networks, sequence-to-sequence models for machine translation, natural language processing, and Partial Differential Equation (PDE) based simulations. TensorFlow supports production prediction at scale, with the same models used for training.

2.4.8 Keras

Keras is a high-level neural network API. It is written in Python and supports multiple back-end neural network computation engines. Keras was created to be user friendly, modular, easy to extend, and to work with Python. The API was "designed for human beings, not machines," and "follows best practices for reducing cognitive load" [17].Neural layers, cost functions, optimizers, initialization schemes, activation functions, and regularization schemes are all standalone modules that you can combine to create new models. New modules are simple to add, as new classes and functions. Models are defined in Python code, not separate model configuration files.

2.4.9 Object detection Evaluation Parameters

The performance of an object detector can be estimated from its accuracy, precision, recall, F1 score etc. The parameters that our object detector utilizes to evaluate its performance is given below.

True Positive Rate: The probability of correctly predicting the results when results are actually true.

$$TruePositiveRate = \frac{TruePositives}{TruePositives + FalseNegatives}$$

True Negative Rate: The probability of correctly predicting the results as absent when results are actually not present.

$$TrueNegativeRate = \frac{TrueNegatives}{FalsePositives+TrueNegatives}$$

False Positive Rate: The probability of predicting the results as true when the results in reality are absent.

$$FalsePositiveRate = \frac{FalsePositives}{FalsePositives+TrueNegatives}$$

False Negative Rate: The probability of predicting the results as false when in reality the results are present.

$$FalseNegativeRate = \frac{FalseNegatives}{TruePositives+FalseNegatives}$$

Accuracy: The probability of correctly predicting the results of a test.

$$Accuracy = \frac{TruePositives+TrueNegatives}{Total}$$

Precision: Precision is the measure of identifying the results correctly from the total true cases.

$$Precision = \frac{TruePositives}{TruePositives+TrueNegatives}$$

Recall: Recall is the measure of correctly predicting the true results in a test scenario.

$$Recall = \frac{TruePositives}{TruePositives+FalseNegatives}$$

F1 Score: F1 score gives the accuracy of the model taking into account the precision and recall.

$$F1score = 2x\frac{PrecisionxRecall}{Precision+Recall}$$

Misclassification Rate: The probability of falsely predicting the results of a test.

$$MisclassificationRate = \frac{FalsePositives+FalseNegatives}{Total}$$

Chapter 3
Hardware and Software Description

3.1 Hardware Description

3.1.1 Raspberry Pi

Figure 3.1: Raspberyy Pi 3B+ Model

Figure 3.1 [18] shows the image of Raspberry Pi 3B+ model. The Raspberry Pi 3 Model B+ is the latest product in the Raspberry Pi 3 range. It has a cortex A53 64-bit quad core processor running at 1.4GHz, dual-band 2.4GHz and 5GHz wireless LAN. It has memory upto 1GB LPDDR2 SDRAM [19]. It has 40 GPIO pins. It also has Bluetooth 4.2/BLE, faster Ethernet, and Power over Ethrenet (PoE) capability via a separate PoE HAT. Input power 5V/2.0A DC via micro USB connector. A 5V DC via GPIO header Power over Ethernet (PoE)–enabled (requires separate PoE HAT). It has an environment operating temperature of 0–50C. It also has an SD card slot supporting a maximum SD memory storage of 64GB. It utilizes Micro SD format for loading operating system and data storage.

3.1.2 Neo6M GPS Module

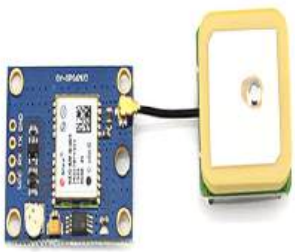


Figure 3.2: Neo 6M GPS Module

Figure 3.2 [20] is of a Neo 6MV2 GPS module. The NEO-6M GPS module is a complete GPS receiver with a built-in 25 x 25 x 4mm ceramic antenna, which provides a strong satellite search capability. With the power and signal indicators, the status of the module can be monitored. It has data backup battery, the module can save the data when the main power is shut down accidentally. Its 3mm mounting holes can ensure easy assembly on the device. It has an active antenna integrated, and a built-in EEPROM to save configuration parameter data. It can be powered by 3-5V. It has a default baud rate of 9600bps [21].It can interface by means of RS232 or TTL.

3.1.3 Raspberry Pi Camera

Figure 3.3: Raspberry Pi Camera V1 model

The Pi camera module as shown in Figure 3.3 [22] is a portable light weight camera that supports Raspberry Pi. It communicates with Pi using the MIPI camera serial interface protocol. It is normally used in image processing, machine learning or in surveillance projects. It has a 5MP colour camera module without microphone for Raspberry Pi. It supports both Raspberry Pi Model A and Model B. It utilizes

Omnivision 5647 Camera Module. It has a resolution 2592 x 1944. It supports 1080p, 720p and 480p image resolutions [23]. It weighs about 3 grams.

3.1.4 SD Card

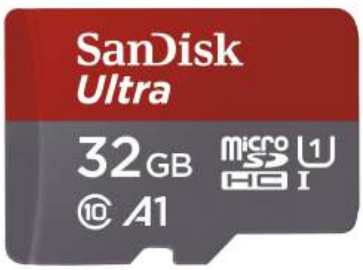


Figure 3.4: Sandisk 32 GB SD Card

A Secure Digital Card (SD) Card as shown in Figure 3.4 [24]is an ultra-small flash memory card designed to provide high-capacity memory in a small size. SD cards are used in many small portable devices such as digital video camcorders, digital cameras, handheld computers, audio players and mobile phones. An SD card typically measures 32 x 24 x 2.1 mm and weighs approximately 2grams [25].

3.2 Software Description

3.2.1 Python

Python is an open source interpreted, high level scripting language. The version of python used for the project was 3.7.4. It was used to create scripts for image resizing, data augmentation, GPS coordinates acquisition, object detection. It was also used in many of the scripts for training the datasets such as xml to csv conversion, generation of TFRecords, the training scripts. In windows the Anaconda IDE was used to code in python. In Raspberry Pi Thorny Python was used as the python IDE.

3.2.2 Open CV

Open CV is an open source computer vision library which is commonly used for image processing applications. The library has more than 2500 optimized algorithms, which includes a comprehensive set of both classic and state-of-the-art computer vision and machine learning algorithms. These algorithms can be used

to detect and recognize faces, identify objects, classify human actions in videos, track camera movements, track moving objects, extract 3D models of objects, find similar images from an image database, remove red eyes from images taken using flash, follow eye movements, recognize scenery and establish markers to overlay it with augmented reality, etc [26].

3.2.3 LabelImg

LabelImg is a graphical annotation tool used in the preparation of the datasets. It utilizes python and Qt for its graphical user interface. LabelImg is used to create annotations. Annotations is a procedure in which data is labelled. Here, the target object classes are labelled in the images. LabelImg saves the annoted data in xml files in Pascal VOC format. YOLO format is also supported [27].

3.2.4 Rasberry Pi OS (or Raspbian OS)

Raspbian OS is a debian based OS for the Raspberry Pi. It is optimized for Raspberry Pi's low-performance ARM CPU's [28]. It comes with 35,000 precompiled software bundled in a nice format for easy installation [29]. It is a community project under development with an aim to improve many debian packages.

3.2.5 Google Colab

Google Colaboratory is a product of Google Research that allows to execute python code through the browser [30]. Google Colab is well suited for machine learning as one can use the Telsa GPU provided by Google. Although the resources for the GPU fluctuates from time to time and has usage limits as each session lasts for 12 hours only.

Chapter 4
Implementation

The datasets for the object detector was prepared from scratch. This includes the process the collection of images, annotation of images, generation of csv data and generating the TFRecords. A brief explanation on configuring the training process is also given in the description below.

4.1 Collection of Images and Data Augmentation

1. The object detection classifier must detect four classes of images in this project. They are namely: plastic bottle, plastic bag, bottle caps and mackerel.

2. The images for the four classes are gathered from Google images.

3. The images are then resized to 300 x 300 dimensions by using a python script.

4. The resized images are then augmented to expand the training dataset. Data augmentation is necessary in order to minimize a problem called over fitting. Data augmentation is done by using a python script.

4.2 Image annotation

1. The augmented images are annoted by using an image annotation tool called LabelImg. Annotation is a procedure where bounding boxes are drawn over the target object in the image. More than one target object can exist in an image.

2. Appropriate class labels are given to the annoted objects.

3. The annoted image are saved in PascalVOC format in a xml file. The xml file contains the bottom left and top right x-y coordinates of the bounding boxes.

4. The images are divided into test images and train images. General convention is to use about 80 % of the total image for training and 20% of the images for testing.

4.3 Generation of CSV and TFRecords File

1. The next procedure is to generate csv files from the xml files. CSV file is an exel file containing the coordinates of the bounding rectangles. This is done by means of a xml to csv conversion python script. The csv files for test and train images are generated seperately.

2. Next the TFRecords are to be generated from the csv files. This procedure is done by means of a python script. The TFRecords files for test and train images are generated separately.

4.4 Configuring the Training Pipeline

1. The training process of the datasets is carried out on Google Colabs.

2. The configuration file of the model to be used for training must be updated with the classes for which the object detector is trained for. Here, the SSD Moblinet V2 300 x 300 quantized model's configuration file is updated with four classes, plastic bottle, plastic bag, bottle caps and mackerel. The configuration files contains information on the loss functions, optimizers, epochs and batch sizes.

3. Next a label map file is to be prepared that contains the class labels along with id numbers. The sequence of class labels and id numbers must be in the same order that was followed in the config file. It is saved as a .pbtxt file.

4. Finally the training pipeline is to be configured by changing few aspects of the training script. The aspects being, the path to the test and train directories containing the images; the path of the TFRecords of test and train images; the number of classes to be detected, the number of test and train images, the batch size of the images.

5. The training file also performs data augmentation by default.

6. A checkpoint file, a graph.pbtext and piplinne.config files are produced post training.

7. The frozen inference graph is obtained by using a python script. Here, the checkpoint file must be provided.

8. The tflight graph is obtained by using a python script. Here, the checkpoint file must be provided.

9. Finally, the tflite file is generated. for this process the dimensions of the image with the channels and other arguments are to be provided. A python script is used to obtain the tflite file.

Chapter 5
Algorithms

SSD model is used to train the datasets for the purpose of object detection. Mobile Net V2 is the deep neural network used for training datasets which are deployed in low power and computation embedded devices. Apart from this algorithm, algorithms for resizing the image, data augmentation and evaluation script to deploy the object detector classifier has been used.

5.1 Image Resizing

The process of manipulating the resolution of images is known as image resizing. An image is represented in the form of a matrix. If an image is downscaled, pixels are removed from an image and when an image is upscaled, pixels are added to the image.

In the process of preparing an image dataset the images collected for it may be of different resolutions. The images that are fed to the classifier should be resized to the same resolution. Hence the process of image resizing is necessary. If the images are of higher dimensions the data would prove to be bulky for the training process.

The object detection model used in the project in SSD Mobilenet V2 quantized model. It accepts images of dimensions 300 x 300 for training purpose. A python script has been implemented to resize the images to a resolution of 300 x 300. The flow of the process of image resizing has been described in Figure 5.1

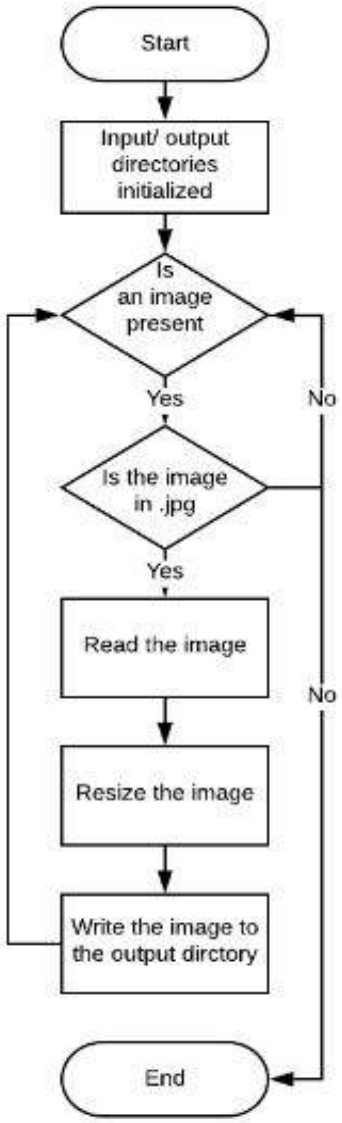


Figure 5.1: Flowchart of image resizing process

1. The path to the directory containing the images to be resized is provided to the algorithm.

2. An iterator stores the image shape in it. In this case it is (300, 300, 3), where 300 and 300 correspond to the height and width respectively and 3 corresponds to the channels in the image. The three channels in the image are red, green and blue.

3. The image is read into a variable which then passes it on to the resize function to resize the image. The arguments for the height and width of the image have been specified in the resize function. Here the arguments for height and width are 300 and 300 respectively.

4. The resized image is then written into a seperate directory.

5. The iterator iterates through all the images in the directory until it has no image to resize.

5.2 Data Augmentation

Data Augmentation is the process of replacing the original dataset by modified datasets. It is a procedure conducted to expand the training datasets. Data augmentation is necessarry in order to overcome the problem of overfitting. Overfitting occurs when a model learns excessive detail and noise in the data [31]. It occurs when the model learns the data too well.

There are different types of data augmentations that are applicable to images. Horizontal Flip, vertical flip, image rotation, shearing, adjustment in the brightness of the image, channel shift are a few of them.

In this project keras is used for data augmentation. The attributes for data augmentation are set in the Image Data Generator class. The data augmentations used in the project are given below.

Image Rotation

It is used to rotate the image by a desired angle. The command used for image rotation is 'rotation_range' It accepts the angles in degrees. It does not accept floating point values. The rotation angle can be prefixed with a '+' or '-'. It rotates the image clockwise or anticlockwise.
Example1: rotation_range = 40 (Default prefix is positive).
It rotates the image by $30°$ in the clockwise direction.
Example2: rotation_range = -40
It rotates the image by $30°$ in the anti-clockwise direction.)

Zoom Operation

The zoom range specifies whether the image is magnified or zoomed out. The command used to magnify or obtain a zoomed out image is 'zoom_range'. If the zoom range is less than 1 then the image is magnified. If it is greater than one, then the image is zoomed out. Zoom range can be specified in the form of a list.
Example1: zoom_range = [0.25, 1.75]
Example2: zomm_range = 0.75
In Example2 the zoom range is given as a floating point number. It takes it in the form of [1-zoom_range, 1+zoom_range].

Width Shift

Width shift is used to shift the image randomly to the right or left. The command used for width shift is 'width_shift_range'. The amount of shift is specified by a tuple or a floating point number. Floating point number is any number in the range of 0 to 1.
Example1: width_shift_range = 0.4
Example2: width_shift_range = [-1, 0, +1]

Height Shift

Height shift is used to shift the image vertically. The command used to shift the height is 'height_shift_range'. It can be specified in a floating point number, a one dimentional array or an integer number. If the number specified to the height_shift_range is less than 1, then the whole height of the image is shifted. If the value is more than one then the pixels are shifted.
Example1: height_shift_range = 0.5
Example2: height_shift_range = [-1, 0, +1]

Shearing an image

Shearing an image means to stretch an image about a particular axis. The command for shearing an image is 'shear_range'. The image is initially rotated by a random amount and then stretched about a particular axis. The value given to shear_range is floating point and in degrees.
Example1: shear_range = 30.0
The image is rotated by 30° and stretched.

Horizontal Flip

Horizontal flip is used to flip the image about the horizontal axis. The command used for this purpose is 'horizontal_flip'. The horizontal flip command is given by: horizontal_flip = True.

Vertical Flip

Vertical flip is used to mirror the image about the vertical axis. The command used for this purpose is 'The command used for this purpose is'. The vertical flip is issued by: vertical_flip = True.

Brightness Range

The brightness range defines the colour intensity of the image. It's value ranges from 0 to 1, where '0' represents a dark shade and '1' represents the brightest shade. The command used to change the brightness 'brightness_range'.
Example: bright_range = (0.3, 0.9)

Fill Mode

Fill mode is used to fill in the pixels when an image has undergone the process of shearing, width shift etc. The command used for this is 'fill_mode'. The fill mode used in the augmentation of the images here is "nearest". Nearest duplicates the end pixels of the image to fill in the gap when the image has been rotated, sheared or shifted. Nearest follows a sequence as follows: aaaaaa | abcd | dddddd, where a and d denoted the ending pixels.
Example: width_shift_range = 0.3, fill_mode = "nearest"

The process for the data augmentation of the image used in this project is as shown in Figure 5.2.

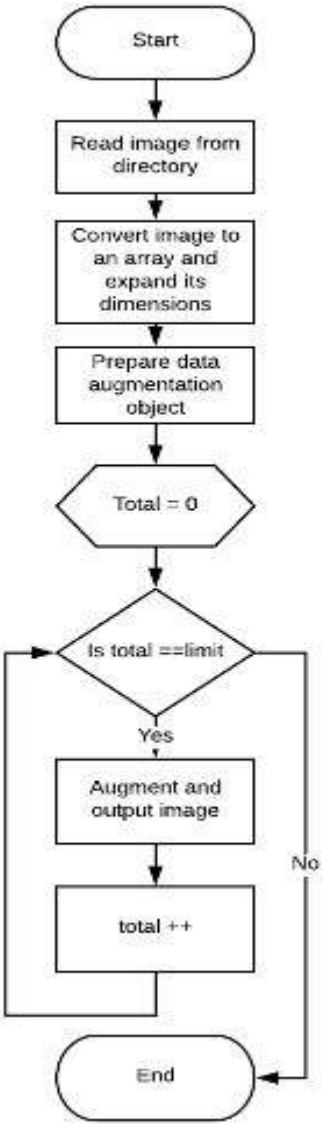


Figure 5.2: Flowchart of data augmentation process

1. The path of the image that must be augmented is set.

2. The image is read and is stored in a numpy array.

3. The dimensions of the images are expanded.

4. An augmentation object is declared and the augmentations to be performed are declared in it.

5. The augmentation are performed on the image and the image is saved to a specified directory using the prefix for the images.

5.3 Object Detection Process Flow

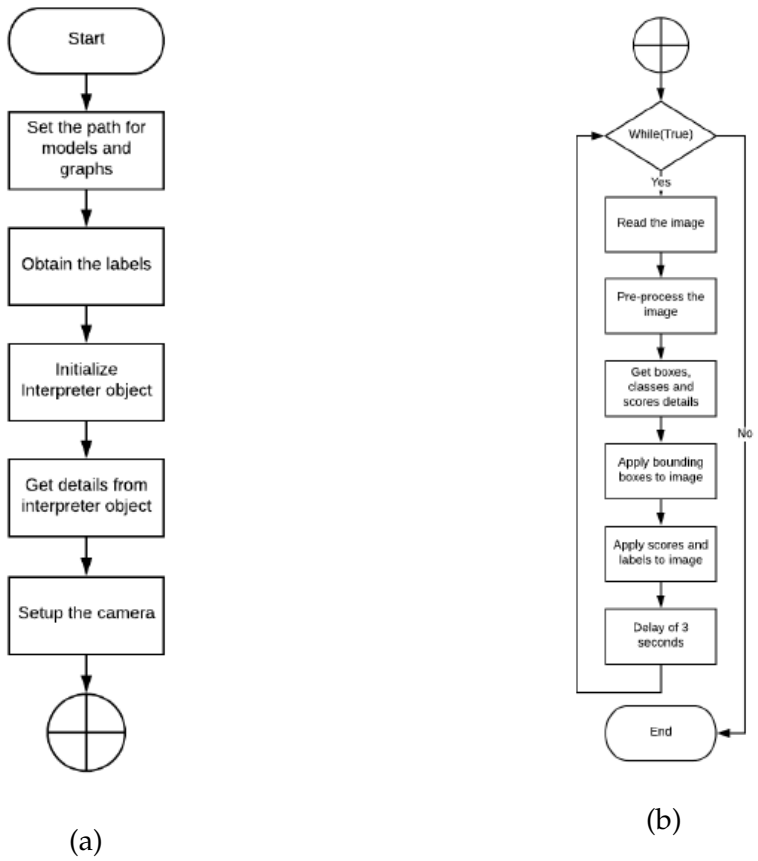


Figure 5.3: Flowchart of object detection process

1. Figure 5.3 describes the process of detecting a target object in an image.

2. The path model, graph and labelmap are set. This is essential as the object detection code utilizes the results from the model, graph and labelmap to draw bounding boxes and label the detected images.

3. The labels are read from the labelmap file and stored in a list.

4. A pi camera is used to capture images at intervals of every 3 seconds. The captured images are read into an object. The dimensions of the image is acquired from the model.

5. The image is resized to the dimensions used by the images during the training process.

6. The bounding box, classes, scores details are acquired by the interpreter object and stored in it.

7. For the input image, the bounding boxes along with their respective scores and class label are applied.

8. The processed image is saved in a directory in the raspberry pi.

5.4 GPS Coordinates Acquisition Process Flow

The GPS coordinates are acquired by the Neo 6 GPS module and transmitted to the raspberry pi via UART communication. The GPS coordinates are updated in a text file at every 1 second interval.

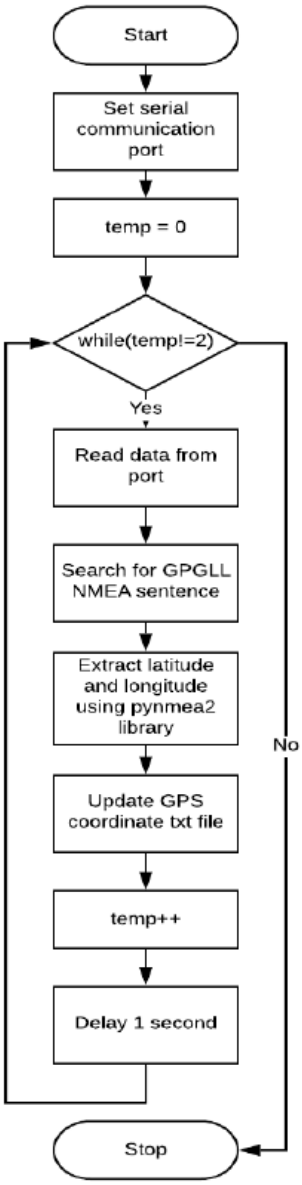


Figure 5.4: Flowchart of GPS coordinates update process

1. The Neo 6 module is used to capture the GPS coordinates of the ROV.
2. It transfers it to the raspberry pi via its serial terminal by means of UART communication.
3. The GPS coordinates are read and GPGLL sentence is extracted from the National Marine Electronics Association (NMEA) data format.
4. On acquring the GPGLL sentence the latitude and longitude are extracted from it.
5. The raspberry pi updates the GPS coordinates of latitude and longitude in a file at an interval of every 1 seconds.

Chapter 6
Testing and Results

6.1 Testing for the Acquisition of GPS Coordinates

1. The GPS module attached to the Raspberry Pi was taken to an outdoor environment.

2. As stated in the algorithm the GPS coordinates acquisition takes place as a subprocess. The normal flow of the programme is broken and the python script for GPS acquistion is run.

3. The GPS coordinates were acquired as shown in the diagram and were written into a Text file.

4. The contents of the file contain information of the latitude, longitude, directions of the latitude and longitude, the date and time of the acquisition of the GPS coordinates.

5. The latitude and longitude information are in terms of degrees.

6.2 Testing for Object Detection in an Image

1. For testing out object detection of plastic waste and marine life a simulation based testing was undertaken.

2. A set of 200 images were given as input to the object detector. This consisted of 45 images of plastic bottle, plastic bags, plastic caps and marine life each. It also consisted of 5 images each of apples, mangoes, cities and plants for testing false positives and true negatives

3. A table was formulated that contained information on number of images given, images in each categories, true positive, true negatives, false positives, false negatives, all their rates, the precision, accuracy, misclassification rate, recall, precision, F1 score.

4. The values for the parameters have been recorded in the table accordingly.

5. The threshold for the object detection has been taken as 0.5. The values for the parameters in Table 6.1 and Table 6.2 can vary based on the change in

threshold or change in datasets. A larger dataset will lead to more accurate results.

6.3 Results

Raspberry Pi served as the target device when obtaining the results for image acquisition and processing. It also processed GPS coordinates.

6.3.1 GPS Coordinates Acquisition Results

The result for the acquisiton of GPS coordinates are shown in the Figure 6.1. The Latitude and Longitude are depicting the degrees. Date and time are in Universal Coordinated Time (UTC).

```
Latitude = 12.8698N Longitude = 74.8527E Date and time = 19-07-2020 01:27:48 PM
Latitude = 12.8697N Longitude = 74.8526E Date and time = 19-07-2020 01:27:49 PM
Latitude = 12.8699N Longitude = 74.8526E Date and time = 19-07-2020 01:27:58 PM
Latitude = 12.8699N Longitude = 74.8526E Date and time = 19-07-2020 01:27:59 PM
Latitude = 12.8697N Longitude = 74.8525E Date and time = 19-07-2020 01:28:08 PM
Latitude = 12.8696N Longitude = 74.8525E Date and time = 19-07-2020 01:28:09 PM
```

Figure 6.1: GPS Coordinates results

6.3.2 Object Detection Results

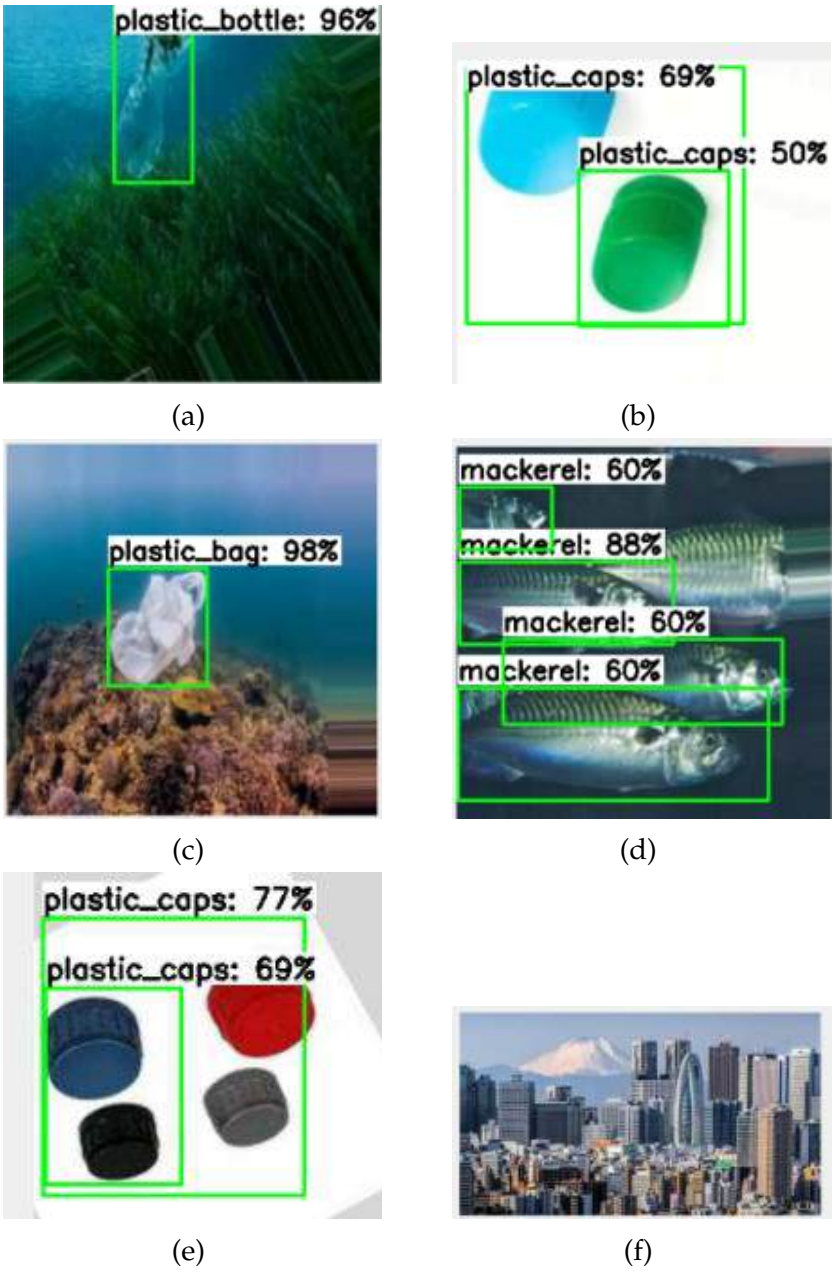


Figure 6.2: Object Dectection Results (Images)

Figure 6.2 shows the object detection results for the test images given during the testing process. Here, (a) ,(b) and (c) show the detection results of plastic bottle, cap, bags and (d) shows the results of mackerel. Figure (e) shows ambiguity is multiple detection of bottle caps and Figure (f) shows True negative detection where none of the classes are falsely detected. The tabulation of the parameters obtained from the testing process are given in Table6.1 and Table 6.2.

Table 6.1: Results of testing of images for object detection

Parameter	True Positive	True Negative	False Positive	False Negative
Plastic Bags	42	19	2	14
Plastic Bottle	44	19	1	9
Plastic Caps	46	11	11	50
Mackerel	53	19	2	8

Table 6.2: Results of testing of images for object detection calculation

Parameter	Plastic Bags (in %)	Plastic Bottles (in %)	Plastic Caps (in %)	Mackerel (in %)
True Positive Rate	75	83.01	47.91	86.88
False Positive Rate	9.5	5	50	9.52
True Negative Rate	90.47	95	50	90.47
False Negative Rate	25	16.98	52.08	13.11
Accuracy	72.22	86.30	48.30	87.80
Missclassification Rate	20.77	13.69	51.69	12.19
Precision	95.45	97.77	80.70	96.36
Recall	75	83.01	47.91	86.88
F1 Score	83.99	89.75	60.12	91.37

Chapter 7
Conclusions and Future Work

7.1 Conclusion

1. The object detector achieved a resonable accuracy in the range of 70-90% excluding the accuracy of plastic caps.

2. The detector could detect singular plastic caps correctly but does not identify individual plastic caps present in a large group of caps accurately. It is for this very reason the scores of plastic caps for accuracy, true positive rate, true negative rate are low and the scores for false positive rate, false negative rate and misclassification are high.

3. The object detector is successful in detecting multiple plastic bags, plastic bottles and mackerel in the same image.

4. The GPS coordinates are updated in a file in the Raspberry Pi. The GPS coordinates print the latitude, longitude, date and time.

7.2 Future Work

1. A camera with Higher resolution than a raspberry pi can be used.

2. The object detector can be trained to detector can be further trained to detect a large class of objects metallic cans, other variety of marine life and marine plants.

3. The object detector can be trained on other deep learning networks such as Faster-RCNN, YOLOv3 models. The object detector trained on these models must be deployed on a target device other than the raspberry pi as these models consume more processing power than the SSD Mobilenet Quantized V2 model.

Bibliography

[1] P. Liu, Y. Hongbo, Y. Hu, and J. Fu, "Research on target recognition of underwater robot," in *2018 IEEE International Conference on Advanced Manufacturing (ICAM)*. IEEE, 2018, pp. 463–466.

[2] Q.-C. Mao, H.-M. Sun, Y.-B. Liu, and R.-S. Jia, "Mini-yolov3: Real-time object detector for embedded applications," *IEEE Access*, vol. 7, pp. 133 529–133 538, 2019.

[3] B. Benjdira, T. Khursheed, A. Koubaa, A. Ammar, and K. Ouni, "Car detection using unmanned aerial vehicles: Comparison between faster r-cnn and yolov3," in *2019 1st International Conference on Unmanned Vehicle Systems-Oman (UVS)*. IEEE, 2019, pp. 1–6.

[4] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[5] S. Liu, X. Li, M. Gao, Y. Cai, R. Nian, P. Li, T. Yan, and A. Lendasse, "Embedded online fish detection and tracking system via yolov3 and parallel correlation filter," in *OCEANS 2018 MTS/IEEE Charleston*. IEEE, 2018, pp. 1–6.

[6] R. Girshick, "Fast r-cnn," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1440–1448.

[7] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2014, pp. 580–587.

[8] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in neural information processing systems*, 2015, pp. 91–99.

[9] M. Manana, C. Tu, and P. A. Owolawi, "Preprocessed faster rcnn for vehicle detection," in *2018 International Conference on Intelligent and Innovative Computing Applications (ICONIC)*. IEEE, 2018, pp. 1–4.

[10] J. Ciberlin, R. Grbic, N. Teslić, and M. Pilipović, "Object detection and object tracking in front of the vehicle using front view camera," in *2019 Zooming Innovation in Consumer Technologies Conference (ZINC)*. IEEE, 2019, pp. 27–32.

[11] W.-y. LU and Y. Ming, "Face detection based on viola-jones algorithm applying composite features," in *2019 International Conference on Robots & Intelligent System (ICRIS)*. IEEE, 2019, pp. 82–85.

[12] An overview of object detection: one-stage methods. [Online]. Available: https://www.jeremyjordan.me/object-detectionone-stage/#ssd

[13] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," in *European conference on computer vision*. Springer, 2016, pp. 21–37.

[14] Object detection and image classification with yolo. [Online]. Available: https://www.kdnuggets.com/2018/09/objectdetection-image-classification-yolo.html

[15] Deep learning. [Online]. Available: https://www.investopedia.com/terms/d/deep-learning.asp

[16] Ml | introduction to transfer learning. [Online]. Available: https://www.geeksforgeeks.org/ml-introduction-to-transfer-learning

[17] What is keras? the deep neural network api explained. [Online]. Available: https://www.infoworld.com/article/3336192/what-is-keras-the-deep-neural-network-api-explained.html

[18] Raspberry pi 3 model b+ bcm2837b0 soc, iot, poe enabled. [Online]. Available: https://robu.in/product/raspberry-pi-3-model-b-bcm2837b0-soc-iot-poe-enabled

[19] Raspberry pi 3 model b+. [Online]. Available: https://cdn.sparkfun.com/assets/8/3/9/b/2/Raspberry-Pi-Model-B_-Product-Brief.pdf

[20] Neo-6mv2 gps module. [Online]. Available: https://components101.com/modules/neo-6mv2-gps-module

[21] Ublox neo-6m gps module. [Online]. Available: http://wiki.sunfounder.cc/index.php?title=Ublox_NEO-6M_GPS_Module

[22] Raspberry pi camera module 5mp. [Online]. Available: https://www.indiamart.com/proddetail/raspberry-pi-camera-module-5mp-12397016933.html

[23] Pi camera module – 5mp. [Online]. Available: https://components101.com/misc/pi-camera-module

[24] Sandisk 32gb sd card. [Online]. Available: https://www.indiamart.com/proddetail/sandisk-memory-card-32-gb-micro-sd-14064569755.html

[25] Sd card. [Online]. Available: https://www.webopedia.com/TERM/S/SD_Card.html

[26] Opencv. [Online]. Available: https://opencv.org/about

[27] Labelimg. [Online]. Available: https://github.com/tzutalin/labelImg

[28] Raspberry pi os. [Online]. Available: https://en.wikipedia.org/wiki/Raspberry_Pi_OS

[29] Raspberry pi os. [Online]. Available: https://www.raspberrypi.org/documentation/raspbian/

[30] Colaboratory. [Online]. Available: https://research.google.com/colaboratory/faq.html

[31] Overfitting and underfitting. [Online]. Available: https://chemicalstatistician.wordpress.com/2014/03/19/machine-learning-lesson-of-the-day-overfitting-and-underfitting/

Appendix A
Papers Published/Presented

International Journal of
Embedded Systems and Emerging Technologies
ISSN: 2456-723X
Vol. 5: Issue 2
www.journalspub.com

Object Detection using Machine Learning: A Survey

Rahul Hegde[1,] *, Sanobar Patel[1], Rosha G Naik[1], Sagar K N[1], K. S. Shivaprakasha[2]
[1]Research Scholar, Associate Professor[2]
Department of Electronics and Communication Engineering, N. M. A. M. Institute of Technology, Nitte, Karnataka, India

ABSTRACT
Computer vision techniques have become increasingly popular in the modern era. One of the applications of computer vision is in the detection of object in an image or video. In object detection a target object is detected, localized and labelled based on the type of object. Real time detection of objects in harsh environments, accuracy and speed are some of the major challenges in computer vision technology. A variety of algorithms have been proposed and implemented for the detection of objects and tracking of objects. In this paper an attempt is made to discuss and compare a few of the algorithms for object detection namely, You Only Look Once (YOLO), Region Convolution Neural Networks R-CNN and Viola-Jones. Detection of plastic waste and maritime life underwater by means of a Remote Operated Vehicle is the focus of object detection in this paper.

Keywords: *Object Detection, You Only Look Once (YOLO), Fast-Region Convolution Neural Network (Fast-RCNN), Faster-Region Convolution Neural Network (Faster RCNN), Viola Jones, Haar Features.*

****Corresponding Author***
E-mail: rahulhegde98@gmail.com

INTRODUCTION

Machine learning for object detection has been used since a long time. In this method, a target object is to be detected by means of a computer algorithm. Object detection is an application of computer vision. Several algorithms have been developed to give a better result in object detection. Each algorithm has its own advantages and disadvantages. A qualitative analysis and comparison between three different algorithms are done in this paper.

Waste management has been a major issue for mankind since ages. In the current times clearing out accumulated waste is a major challenge as it could be in a challenging scenario such as underwater. Detection of plastic waste underwater is hard owing to the harsh environment and challenges such as reduced visibility, the water current, the nature of the plastic waste etc. Computer vision has aided in this process as it provides algorithms for effective image processing and machine learning. Accuracy and speed are the two main factors in any computer vision algorithms. The type of processor used as to whether it is Graphical Processing Unit (GPU) or a Central Processing Unit (CPU) plays an important role in the computations based on the algorithms.

The paper is organised as follows: Section II, highlights different algorithms and their

performances in different scenarios. Section III presents a comparison amongst the algorithms discussed in Section II. Finally, Section III gives the conclusion.

MACHINE LEARNING ALGORITHMS FOR UNDERWATER OBJECT DETECTION

An ample number of algorithms using Machine learning have been proposed in the literature. In this paper, we have considered three popular and promising algorithms developed for underwater applications. A brief insight into each of these methods is given in subsequent subsections.

A. Yolov3

YOLO is an algorithm proposed in [1]. In YOLOv3 algorithm the image is resized initially. It then divides the image frame into an M×M block grid of three scales being 13×13, 26×26 and 52×52 [2].It predicts bounding boxes and probability in each region.The YOLOv3 algorithm predicts the bounding boxes by the centroid co-ordinates of the image sub boxes in the grid. The bounding boxes generated have four co-ordinates (kx,ky,kw,kh), where kx and ky are used to obtain the centroid of the bounding boxes, kw and kh are used to get the width and height of the bounding boxes. Each of the boxes use a simple logistic regression (K - means clustering) for classification and binary cross entropy loss. It predicts through three different scales of boxes [1]. It uses 3×3 and 1×1 convolution network and short connections between them.After the image is processed in the first convolution layer in the 13×13 grid, it proceeds to 26×26 grid and then further on to the next 52×52 grid [2]. YOLOv3 is a hybrid version of YOLOv2 and Darknet 19. The training net uses 53 convolution layers and is called Darknet-53 [3], [4], [5],[6].

B. Faster r-CNN

Region proposal networks are used on image frames to detect objects, but they were limited by the fact that the generation of proposals were time consuming. An advancement in region proposal generation was the development of Fast R-CNN [7], but this too had a limitation in the computation of region proposals. This limitation was overcome by Faster RCNN [8] by the incorporation of a new region proposal generating mechanism.

Faster RCNN consists of two modules namely fully convolutional neural network and the Fast RCNN network. The RPN is used to generate suggestion box classifiers by running region proposals on the image frame to detect the desired object. The Fast R-CNN module uses the region proposals generated as attention detectors to obtain a shared feature map from the image [4]. An Image is first passed through a convolution layer to generate feature maps. The convolution layer uses convolution non-linear transforms and linear regressions to generate the feature maps. In order to generate the region proposals, the feature maps are then passed on through a region proposal network. The region proposals generated are rectangular boxes. These region proposals are characterized with respect to certain fixed parameters relative to the reference boxes called as anchors [9]. Anchors are associated with the sliding window, the scale and aspect ratio of the reference boxes. The RPN is used for the generation of suggestion boxes and classifiers. A Softmax layer [10] is used in order to determine the predicted anchor point for foreground and background by means of boundary regression principle. A Region of Interest (ROI) Pooling Layer combines the feature maps and the suggestion boxes and resizes the suggestion boxes to the same size as that of the feature map and sends it to the next layer which is a full link layer. The full

link layer is used for the identification of the desired object. The next layer is a Classification Layer. It sends the feature map to the classifier for the classification of the objects based on the received set of object classes [1]. The bounding boxes appear on the desired object to be detected in the image with the classifiers name labelled on to it.

C. Viola-Jones Algorithm

Viola-Jones algorithm [11] was created for the detection of faces, but recently it has been successfully adopted for detection of other objects. It is a machine learning algorithm recognized by three vital parts. They are learning phase, integral image phase and cascade classifier phase. During the learning phase, a number of Haar features are determined by means of Ada boost Algorithm. Three types of Haar features [3] are used, they are two rectangles, three rectangles and four rectangles. The Haar features have a black region and white region [3]. The difference between the values of the black region and the white region is the Eigen value to be calculated [11].

In the detection of objects two variants of the images are used they are positive images and negative images. Images where the object of interest is present are called positive image and the images where the object of interest is not present are called negative images [3]. All positive images are scaled to the dimension of that of the sub windows before the learning process. The dimension of the sub window determines the smallest dimension of the object that can be determined. After each training sequence a large set of Haar features [3] are extracted.

Cascade classifiers [3], are used to reduce computation time and increase the detection speed. This process consists of multiple stages. Each stage denotes a weak classifier. The detectors created represent a cascade classifier. In detection phase a window whose dimension are already know goes through the whole image step by step and pixel by pixel. The process is repeated until the image dimensions become smaller than the sub image. The image is classified as either positive or negative. The process of classification is continued only if the image is positive. The drawback is the pre-determined dimensions of the object [3], [11].

COMPARATIVE ANALYSIS

In this section we provide a qualitative analysis of the algorithms that were presented in Section II. The comparison is made based on the various quality parameters of the algorithm. Table I presents the comparison between Faster R – CNN with YOLOv3 [4].

Table 1. *Comparison between Faster R-CNN and YOLOv3.*

Parameter	Faster R-CNN (in %)	YOLOv3 (in %)
Precision	99.66%	99.73%
Sensitivity	79.40%	99.07%
Quality	79.17%	98.81%
Processing time	1.39s	0.057ms

From Table 1, we can infer that when it comes to the processing time of the frames YOLO v3 is roughly about 2000 times superior than that of Faster R-CNN.This would be a critical factor when scenarios include detection of objects in motion. The quality of detection in YOLOv3 is also better in comparison with Faster R-CNN. This is an important factor in scenarios like detection of objects underwater, a snowy weather or other similar harsh conditions. The sensitivity which is the measure of correctly detected image (True Positives) is higher in case of YOLOv3 than in Faster R-CNN. This can also be observed by observing the true positive given in Table I. With these observations, we can infer that YOLOv3 would be a better Machine learning algorithm than

that of Faster R-CNN algorithm. Nevertheless, YOLOv3 fails to detect large objects and Faster R-CNN performs superior in this case [4].

In the paper [3], Viola Jones algorithm and YOLOv3 were used to detect vehicles, traffic signs and pedestrians. The comparison was done by taking an image of 640×480 resolution image on a Core i5-6200 CPU. Table 2 gives the set of parameters of Viola Jones algorithm.

Table 2. *Parameters of Viola-Jones algorithm*

Detector Name	Precision (in %)	Recall	Frames per Second (FPS)	Inference Time
Pedestrian	80.95	81.60	2.6	0.38
Traffic Light	91.60	86.33	3.6	0.28
Stop Sign	87.20	82.02	9.2	0.11

Table 3. *Table depicting the parameters of YOLOv3 algorithm*

Detector Name	Precision (in %)	Recall	Frames per Second (FPS)	Inference Time
Pedestrian	94.29	92.40	1.2	0.83
Traffic Light	80.69	84.17		
Stop Sign	97.20	89.09		

In comparison with the above table 3, YOLO v3 based detector has a better precision in detecting pedestrians than compared to Viola-Jones. But Viola Jones has a better precision in case of detection of traffic signals. The collective frames per second and Inference time of YOLOv3 algorithm is higher than that of Viola-Jones algorithm. Thus YOLOv3 would be a better algorithm in case of real-time application. As the above algorithm was tested on a CPU rather than a GPU [13], [14] is FPS would generally be lower.

CONCLUSION

Object detection is underwater scenarios and harsh environments are one of the primary research challenges of the modern era. A variety of machine learning algorithms have been proposed and validated for its performance. In this paper, we have discussed and compared three such algorithms. From the comparative analysis, the following inferences were made: YOLOv3 algorithm has a better precision compared to the Faster R-CNN and Viola Jones algorithm. The implementation of YOLOv3 algorithm would be more beneficial in real time situation than Faster R-CNN and Viola Jones algorithms. When the GPU's are used to train algorithms the inference times are in milliseconds. For real time applications, the implementation of the algorithms in GPU is preferred. Also, the ability of YOLOv3 algorithm to detect True Positive image is greater than that of Faster R-CNN and Viola Jones.

REFERENCES

[1]. P. Liu, Y. Hongbo, Y. Hu and J. Fu, "Research on target recognition of underwater robot," *2018 IEEE International Conference on Advanced Manufacturing (ICAM)*, Yunlin, 2018, pp. 463-466.

[2]. Q. Mao, H. Sun, Y. Liu and R. Jia, "Mini-YOLOv3: Real-Time Object Detector for Embedded Applications," in *IEEE Access*, vol. 7, 2019, pp. 133529-133538.

[3]. J. Ciberlin, R. Grbic, N. Teslić and M. Pilipović, "Object detection and object tracking in front of the vehicle using front view camera," *2019 Zooming Innovation in Consumer Technologies Conference (ZINC)*, Novi Sad, Serbia, 2019, pp. 27-32.

[4]. B. Benjdira, T. Khursheed, A. Koubaa, A. Ammar and K. Ouni, "Car Detection using Unmanned Aerial Vehicles: Comparison between Faster R-CNN and YOLOv3," *2019 1st International Conference on Unmanned Vehicle Systems-Oman (UVS)*, Muscat, Oman, 2019, pp. 1-6.

[5]. Z. Lu, J. Lu, Q. Ge and T. Zhan, "Multi-object Detection Method based on YOLO and ResNet Hybrid Networks," *2019 IEEE 4th International Conference on Advanced Robotics and Mechatronics (ICARM)*, Toyonaka, Japan, 2019, pp. 827-832.

[6]. R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Richfeature hierarchies for accurate object detection andsemantic segmentation," in Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on, V 5,2014, arXiv:1311.2524v5**.**

[7]. R. Girshick, "Fast R-CNN," arXiv preprint arXiv:1504.08083, 2015.

[8]. M. Manana, C. Tu and P. A. Owolawi, "Preprocessed Faster RCNN for Vehicle Detection," *2018 International Conference on Intelligent and Innovative Computing Applications (ICONIC)*, Plaine Magnien, 2018, pp. 1-4.

[9]. Ren, Shaoqing, Kaiming He, Ross Girshick, and Jian Sun. "Faster r-cnn: Towards real-time object detection with region proposal networks." In *Advances in neural information processing systems*, pp. 91-99. 2015.

[10]. Multiclass Neural Networks Softmax, "https://developers.google.com/machine-learning/crash-course/multi-class-neural-networks/softmax" Available[Online], Accessed on 21.10.2019

[11]. W. LU and M. YANG, "Face Detection Based on Viola-Jones Algorithm Applying Composite Features," *2019 International Conference on Robots & Intelligent System (ICRIS)*, Haikou, China, 2019, pp. 82-85.

[12]. Redmon, Joseph, and Ali Farhadi. "Yolov3: An incremental improvement." *arXiv preprint arXiv:1804.02767* (2018).

[13]. S. Liu *et al*., "Embedded Online Fish Detection and Tracking System via YOLOv3 and Parallel Correlation Filter," *OCEANS 2018 MTS/IEEE Charleston*, Charleston, SC, 2018, pp. 1-6.

[14]. T. Manderson and G. Dudek, "GPU-Assisted Learning on an Autonomous Marine Robot for Vision-Based Navigation and Image Understanding," *OCEANS 2018 MTS/IEEE Charleston*, Charleston, SC, 2018, pp. 1-6.

Cite this Article: Rahul Hegde, Sanobar Patel, Rosha G Naik, Sagar K N, K. S. Shivaprakasha. Object Detection using Machine Learning: A Survey. *International Journal of Embedded Systems and Emerging Technologies*. 2019; 5(2): 7–11p.

Detection of Plastic Bags in an Underwater Environment Using Raspberry Pi

Rahul Hegde, Sanobar Patel, Rosha G Naik, Sagar K N, Rekha Bhandarkar, K. S. Shivaprakasha
Department of Electronics and Communication Engineering
N. M. A. M. Institute of Technology, Nitte, India.

rekhabhandarkar@nitte.edu.in, shivaprakasha.ks@nitte.edu.in, rahulhegde98@gmail.com, sanobarpatel98@gmail.com, naikrosha6@gmail.com, sagarkodur@gmail.com

ABSTRACT: Trash deposits in aquatic environments have a destructive effect on marine ecosystem and pose a long-term economic and environmental threat. A Remotely Operated Vehicles (ROVs) could very well contribute to the solution of this problem by finding and eventually removing trash. Detection of objects in underwater environment due to reduced visibility and the accuracy of the detected object are some of the major challenges in underwater scenario. In this paper, Single-Short-Multi-Box Detector (SSD) with MobileNet algorithm is used for performing the task of visually detecting plastics bags in underwater environments. A large and publicly-available datasets of plastics bags are annotated for training a number of SSD architectures for object detection. The object detection algorithm is deployed on a Raspberry Pi.

Keywords: Object detection, Deep Learning, Single Shot Detector (SSD), MobileNet, Raspberry Pi.

IINTRODUCTION

Accumulation of plastic waste in rivers, ponds, lakes, seas, oceans and other water bodies has become an increasing concern in recent days. Plastic waste not only pose a threat to animals but also plant life. Several initiatives to clean out plastic waste in rivers, ponds and lakes are undertaken. Cleaning of plastic waste in seas and oceans has proven to be great challenge. In any scenario, a survey is conducted on water bodies before clearing out plastic waste. Although survey is harder in seas and oceans than in smaller water bodies. Survey of plastic waste using deep learning algorithms is emerging.

Survey of underwater waste raises a lot of problems. Detection of plastic waste underwater is hard due to the harsh environment and challenges such as reduced visibility, the water current, the nature of the plastic waste etc. The target device comprising of the object detector is deployed in an Autonomous Vehicle (AUV) or Remotely Operated Vehicle (ROV). Several algorithms have been used to train object detector for detecting objects underwater. Each algorithm has its own set of advantages and disadvantages, this includes size and shape of the objects and also the detection speed.

The rest of the paper is organized as follows: In section II a brief description of the literature survey is presented. Section III presents a brief note on SSD is provided. A description to the system architecture of the project is given in Section IV. The Results and tabulation of the performance of the object detector are presented in Section V. Section VI gives the concluding remarks of the paper.

II LITERATURE SURVEY

Deep learning algorithms such as YOLOv3 and Single Shot Detector were surveyed for their efficiency and robustness for this project. YOLOv3 algorithm was used to train the object detector for underwater imaging in [1]. The implementation of YOLOv3 algorithm in real time object detection was outlined in [2, 3, 4]. YOLOv3 had a shortcoming of detecting smaller objects. SSD algorithm [5, 6] is also used in real time object detection as it had a high accuracy but lacked in the ability of detecting similar objects. The use of SSD was decided as Raspberry Pi was the target device as its resources such as processing power is constrained.

Tutorials on deep learning by Dr. Adrian Rosebrock [7, 8] and Edje Electronics [9, 10] provided a basis to understand machine learning and the approach towards the object detection. Code basics [11] Youtube channel provided the basics for machine learning and deep learning.

SSD models use VGG net or MobileNet as the image classification network. In this project SSD with MobileNet [12] was taken as the model to train the datasets. The reason being MobileNet performs better in resource constrained devices

such as android and Raspberry Pi. SSD MobileNet Quantized V2 model [13, 14] was used for the training process for the object detector.

III SINGLE SHOT DETECTOR

Single Shot Detector (SSD) uses a VGG16 as its image classification network. Convolutional feature layers that progressively reduce in size are added to its end so that it is able to predict object at different scales [1]. In SSD, convolution filters are applied to feature maps to predict the score of a particular class of object and to obtain the offset coordinates of the bounding boxes of the object. Multi box technique is used for bounding box regression.

VGG-16 has a robust performance in high quality image classification. In an SSD as a substitute to the original VGG fully connected layers, a set of auxiliary convolutional layers are added. This enabled feature extraction at multiple scales and progressively decreased the size of the input to each successive layer. [2]

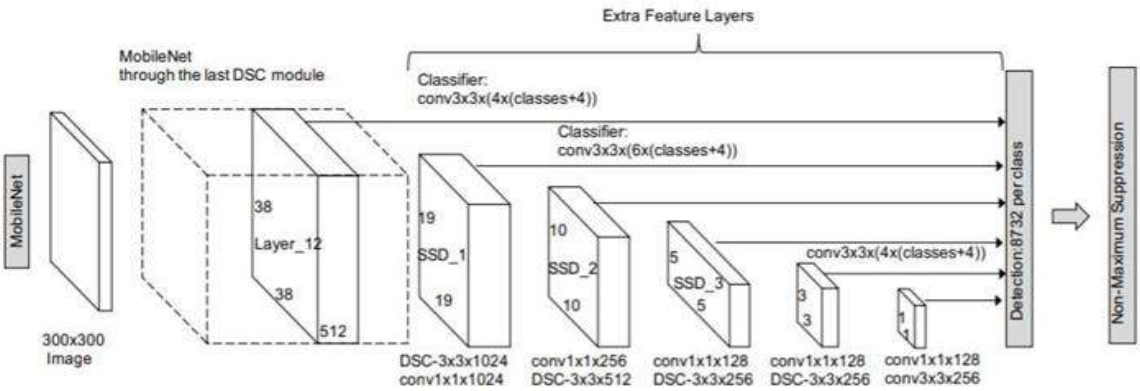


Figure2: Layers of SSD Architecture with Mobilenet for Image Classification [15]

Figure 2 shows SSD Mobilenet [13] layers. It replaces the VGG16 neural network used for image classification. Mobilenet is faster in detection speed and smaller in size when compared to VGG net [16].

IV GENERAL SYSTEM DESCRIPTION

The function of the system is to detect plastic bags in an underwater environment. Images are acquired by a Raspberry Pi camera. The captured image is then processed by a raspberry pi. The object detection script is used to detect plastic bags in the image and draw bounding boxes around it. The image is captured at regular intervals of 2seconds. Figure 3 gives the basic block diagram of the project. Figure 4 shows the general flow of the project.

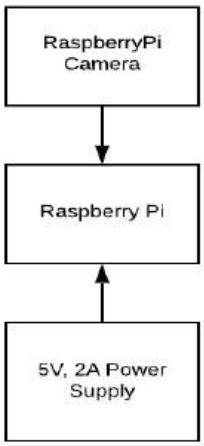


Figure 3: General Block Diagram Discription

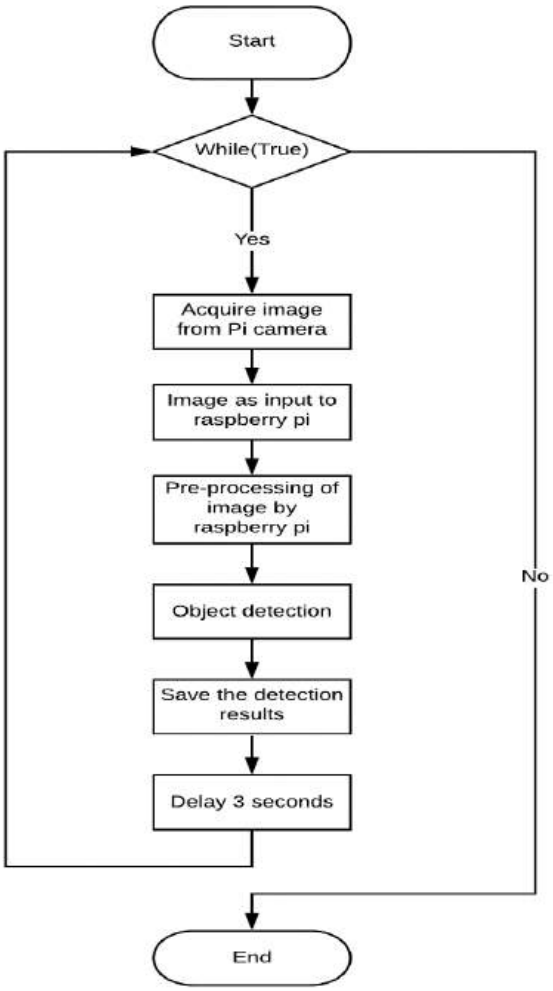


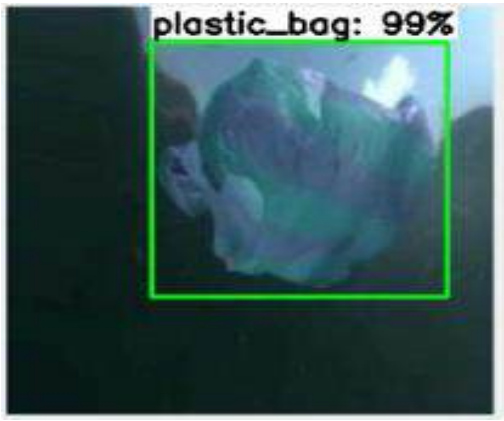
Figure 4 Process flow diagram of the project.

Datasets to train the model was obtained from Google Images. The images were resized to a dimension of 300 x 300 and augmented [17] in order to produce the images for the train and test dataset. SDD MobileNet Quantized V2 model was used for training the object detector. 2500 images of plastic bags formed the dataset. Out of these 80% were taken as training dataset and 20% were taken for test dataset.

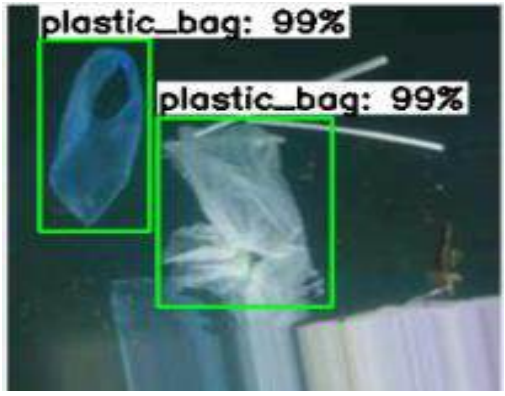
V RESULTS AND ANLYSIS

Testing of Object detection

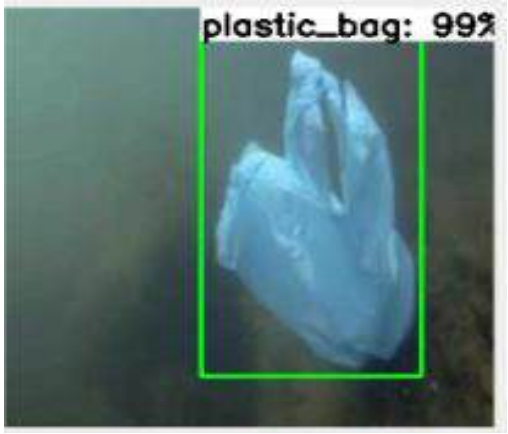
- A set of 100 images was given as an input to the object detector. This had of 85 images of plastic bags, 5 images each of Plastic bottle, plants and cities. All the images were taken from Google.
- Table 1 details the results of the object detector regarding parameters such as True positive, true negatives, false positives, and false negatives. The threshold for object detection was kept at 0.4. Any object having a score less than 0.5 will not get detected. The detection results can vary based on the threshold score.
- Table 2 contains results such as True positive, true negatives, false positives, false negatives rates, the precision and accuracy.
- Images of detected objects from Google images are shown below.



5.(a)

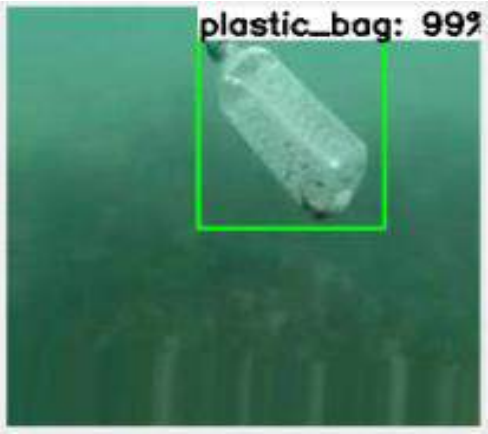


5.(b)



5.(c)

5.(d)



5.(e)

5.(f)

Figure 5. (a), 5. (b), 5. (c) and 5.(d) shows the detection of plastic bags. Figure 5.(e) shows a plastic bottle being falsely detected as a plastic bag and Figure 5.(f) shows an image of a city thus plastic bag is not detected in the image

Table 1. Table describing the detection results of the object detector

Class	True Positive	True Negative	False Positive	False Negative
Plastic Bags	96	7	12	6

Table 2. Parameters of object detection

Parameter (in %)	Plastic Bags (in %)
Accuracy	85.12
Precision	88.88
True Positive Rate	94.11
False Positive Rate	63.15
True Negative Rate	36.84
False Negative Rate	5.88
Misclassification Rate	14.87

VI CONCLUSION

A simulation based testing was conducted by providing 100 images. The object detector detected objects with an average confidence score ranging from 70-90%. The object detector detects single as well as multiple objects with a high score. Although as seen in the images plastic bottle was falsely detected as plastic bag. From Table 2 it can be observed that the Accuracy is good. The object detector did perform not up to the mark in True Negative cases as the score is a little low. The probability that the object detector would misclassify plastic bags is low.

REFERENCES

[1]. Liu, Ping, Y. A. N. G. Hongbo, YuZhen Hu, and JingNan Fu. "Research on target recognition of underwater robot." In *2018 IEEE International Conference on Advanced Manufacturing (ICAM)*, pp. 463-466. IEEE, 2018.

[2]. Mao, Qi-Chao, Hong-Mei Sun, Yan-Bo Liu, and Rui-Sheng Jia. "Mini-YOLOv3: real-time object detector for embedded applications." *IEEE Access* 7 (2019): 133529-133538.

[3]. B. Benjdira, T. Khursheed, A. Koubaa, A. Ammar and K. Ouni, "Car Detection using Unmanned Aerial Vehicles: Comparison between Faster R-CNN and YOLOv3," *2019 1st International Conference on Unmanned Vehicle Systems-Oman (UVS)*, Muscat, Oman, 2019, pp. 1-6.

[4]. Z. Lu, J. Lu, Q. Ge and T. Zhan, "Multi-object Detection Method based on YOLO and ResNet Hybrid Networks," *2019 IEEE 4th International Conference on Advanced Robotics and Mechatronics (ICARM)*, Toyonaka, Japan, 2019, pp. 827-832.

[5]. W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," in European Conference on Computer Vision (ECCV), 2016, pp. 21–37.

[6]. Understanding Real-Time SSD Multi Box Object Detection In Deep Learning [Online], Available:https://towardsdatascience.com/understanding-ssd-multibox-real-time-object-detection-in-deep-learning-495ef744fab, Accessed on: 12.10.2019.

[7]. Image Classification with Keras and Deep Learning [Online] Available:https://www.pyimagesearch.com/2017/12/11/image-classification-with-keras-and-deep-learning/, Accessed on: 16.01.2020.

[8]. Image Classification with Keras and Deep Learning [Online] Available:https://www.pyimagesearch.com/2017/12/11/image-classification-with-keras-and-deep-learning/, Accessed on: 16.01.2020.

[9]. How to run tensorflow lite for object detection on raspberry pi [Online], Available: https://www.youtube.com/watch?v=aimSGOAUI8Y, Accessed on: 10.01.2020.

[10]. How To Train an Object Detection Classifier Using TensorFlow (GPU) on Windows 10 [Online], Available: https://www.youtube.com/watch?v=Rgpfk6eYxJA&t=1167s., Accessed on 10.01.2020.

[11]. Deep Learning with Python, Tensorflow and Keras Neural Network for Image Classification [Online], Available: https://www.youtube.com/watch?v=qEyEijUDOCA, Accessed on: 02.05.2020.

[12]. Howard, Andrew G and Zhu, Menglong and Chen, Bo and Kalenichenko, Dmitry and Wang, Weijun and Weyand, Tobias and Andreetto, Marco and Adam, Hartwig, "Mobilenets: Efficient convolutional neural networks for mobile vision applications" in arXiv preprint arXiv:1704.04861 , 2017.

[13]. Accelerated Training and Inference with the Tensorflow Object Detection API [Online], Available:https://ai.googleblog.com/2018/07/accelerated-training-and-inference-with.html, Accessed on: 22.06.2020.

[14]. Jacob, Benoit and Kligys, Skirmantas and Chen, Bo and Zhu, Menglong and Tang, Matthew and Howard, Andrew and Adam, Hartwig and Kalenichenko,

Dmitry, "Quantization and training of neural networks for efficient integer-arithmetic-only inference" *in proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704—2713.

[15]. Difference between SSD and Mobilenet [Online], Available: https://stackoverflow.com/questions/50585597/difference-between-ssd-and-mobilenet Accessed on: 18.06.2020.

[16]. Compressing Deep Neural Nets, [Online], Available: https://machinethink.net/blog/compressing-deep-neural-nets/, Accessed on: 16.06.2020.

[17]. Keras ImageDataGenerator and Data Augmentation, [Online], Available:https://www.pyimagesearch.com/2019/07/08/keras-imagedatagenerator-and-data-augmentation/, Accessed on: 10.04.2020.

Appendix B
Certificates

NITTE EDUCATION TRUST | NMAM INSTITUTE OF TECHNOLOGY

(An Autonomous Institution affiliated to VTU, Belagavi)
NITTE – 574110, Karkala Taluk, Udupi District, Karnataka, India
ISO 9001:2015 Certified, Accredited b NAAC with 'A' Grade

IEEE Mangalore Sub Section

Department of Electronics and Communication Engineering

In Association with

NMAMIT IEEE Student Branch (STB 34551)

Best Paper NCECCT-20

Best Paper Award

is presented to the paper titled

DETECTION OF PLASTIC BAGS IN AN UNDERWATER ENVIRONMENT

Best Paper NCECCT-20

Presented by

RAHUL HEGDE, SANOBAR PATEL, ROSHA G NAIK, SAGAR K N, Dr. REKHA BHANDARKAR, Dr. K S SHIVAPRAKASHA

of

Department of Electronics & Communication Engg., NMAM Institute of Technology, Nitte, Karkala

at the ***National Conference on Electronics, Communication and Computing Technology (NCECCT-20)***, organized by the Department of Electronics & Communication Engineering at NMAM Institute of Technology, Nitte-574110, Karnataka, India on **10th July 2020**.

Ravindra K S **Organizing Secretary**	**Dr. Rekha Bhandarkar** **Convener**	**Dr. Niranjan U C** **Chairman, IEEE MSS**	**Dr. Niranjan N Chiplunkar** **General Chair**

Master's Thesis

3D Mapping of Structured Environments Utilizing LiDAR Scanners and IMU Mounted on a Mobile Robot

Rahul Hegde

Examiner: Prof. Dr. Alexander Reiterer
Examiner: Prof. Christoph Müller
Supervisor: Dominik Merkle

Albert-Ludwigs-Universität Freiburg
Faculty of Engineering
Department of Embedded Systems Engineering
October 16th, 2023

Writing period

17. 04. 2023 – 17. 10. 2023

Examiner

Prof. Dr. Alexander Reiterer

Examiner

Prof. Christoph Müller

Supervisor

Dominik Merkle

Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.
I hereby also declare, that my Thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Freiburg im Breisgau, 16.10.2023

Place, Date

Signature

Abstract

Mapping is a process in which one collects information of the surrounding environment. One obtains important features of the environment, which enables us to use the information present in the features to navigate or explore the environment. Robots have been used in the process of obtaining environment maps. Robots are incorporated with sensors such as cameras, Light Detection And Ranging (LiDAR) sensors, Radio Detection And Ranging (RADAR) sensors, Sound Navigation And Ranging (SONAR) sensors, Global Navigation Satellite System (GNSS) sensors, Inertial Measurement Unit (IMU) etc., to collect information regarding the environment. Robots can be utilized to obtain maps in environments where it would be difficult for humans to navigate such as underground mines, caves, dense forests, underwater environment, regions with dense snow etc. With the information of an environmental map, one can navigate the environment with path planning algorithms to perform necessary tasks. Mapping can either be performed real time or offline, where the data is obtained prior and processed later on a computing machine. We look into real time mapping of an environment, where the process of mapping takes place online with inputs from various sensors. In our project, the environment under test contains a bridge and the mapping process is carried out by using a four wheeled robot called Husky from Clearpath. The robot has two LiDAR sensors and an IMU attached to it to collect information regarding the environment. Seven different datasets are gathered with different robot trajectories from the test environment. The datasets are then evaluated on three state-of-the-art mapping algorithms to produce environmental maps. The three state-of-the-art mapping algorithms are: (1). Asynchronous Multiple LiDAR Inertial Odometry (MA-LIO), (2). Scan Context based LiDAR Inertial Odometry Smoothing And Mapping (SC-LIO-SAM), (3). Keep It Short and Simple Iterated Closest Point (KISS-ICP). The trajectories and point cloud of the environment obtained from the algorithms are compared with ground truth trajectories obtained by a tachy meter and ground truth point cloud obtained by a Leica RTC360 degree scanner. A comparative analysis on the correctness of results obtained using the algorithms is

performed, and the results are reported. We draw a conclusion regarding the results of the comparative analysis and state the advantages and disadvantages of using the three algorithms of mapping in a bridge scenario.

Contents

1 Introduction **1**

1.1 General Mapping Overview 1

1.2 General SLAM overview 2

1.3 General project overview 3

2 Related Work **5**

3 Background **7**

3.1 Simultaneous Localization and Mapping (SLAM) 7

3.2 Georgia Tech Smoothing and Mapping (GTSAM) 8

3.2.1 IMU Preintegration Factor 9

3.2.2 LiDAR Odometry Factor 9

3.3 Lidar Inertial Odometry 9

3.4 Iterative Closest Points (ICP) 10

3.5 Kalman Filters 12

3.6 Iterated Error State Kalman Filter 12

3.7 Basis Spline Interpolation 13

3.8 Scan Context 13

3.9 Loop Closure 14

3.10 Velodyne LiDAR frame 15

3.11 Procedure for IMU factor integration 16

4 Approach **17**

4.1 Motivation and general approach overview 17

4.1.1 Motivation 17

4.1.2 General Approach Overview 17

4.2 Hardware System Overview 18

4.2.1 Clearpath Husky Robot 18

4.2.2 Velodyne sensor 18

4.2.3 Nvidia Jetson Xavier NX 19
4.2.4 Inertial Measurement Unit (IMU) 20
4.2.5 Leica RTC360 degree scanner 20
4.2.6 Leica MS60 Tachymeter 21
4.3 Software System Overview 22
4.3.1 Robotic Operating Systems 22
4.3.2 Cloud Compare 22
4.4 Dataset 23
4.5 KISS-ICP 24
4.5.1 Parameters of KISS-ICP 24
4.5.2 Overview of KISS-ICP 25
4.5.3 Flow of KISS-ICP 26
4.6 LIO-SAM and SC-LIO SAM 29
4.6.1 SC-LIO-SAM parameters 29
4.7 Asynchronous Multiple LiDAR-Inertial Odometry 32
4.7.1 Parameters of Asynchronous Multiple LiDAR-Inertial Odometry 32
4.7.2 MA-LIO Description 32
4.7.3 LiDAR Data Pre-Processing 34
4.7.4 Uncertainty Propagation 34
4.7.5 Measurement Model with Uncertainty 35
4.7.6 Iterated Error State Kalman Filter 35
4.7.7 Mapping with Uncertainty 35
4.8 Data Acquisition procedure 35
4.8.1 Creation of environment map using KISS-ICP 36
4.8.2 Creation of environment map using SC-LIO 37
4.8.3 Creation of environment map using MA-LIO 38
4.8.4 Data processing: KISS ICP 38

5 Experiments **41**
5.1 Data Acquisition 41
5.2 Location and setup 41
5.2.1 Location 41
5.2.2 Experimental setup 42
5.3 RTC 360 Point Cloud 44
5.4 Dataset Trajectories and Point Clouds using KISS-ICP 44

5.5 Dataset Trajectories and Point Cloud using MA-LIO 45
5.6 Dataset Trajectories and Point Cloud using SC-LIO 46
5.7 Dataset trajectory comparison 46
5.8 Cross-section view of long distance scan 47
5.9 Visual temporal analysis of point cloud formation 49
5.9.1 Temporal point cloud formation analysis of MA-LIO algorithm 49
5.9.2 Temporal point cloud formation analysis of SC-LIO-SAM algorithm 49
5.9.3 Temporal point cloud formation analysis of KISS-ICP algorithm 50
5.10 Point cloud distance error analysis for temporal scans 51
5.11 Correctness evaluation of the point clouds to ground truth scan . . . 53
5.11.1 Distance error analysis for point clouds by MA-LIO algorithm 54
5.11.2 Distance error analysis for point clouds by SC-LIO-SAM algorithm 55
5.11.3 Distance error analysis for point clouds by KISS-ICP algorithm 55

6 Conclusion **59**

7 Acknowledgments **61**

8 Future Work **63**

Bibliography **64**

List of Figures

1 Example sensor maps 2

2 LiDAR slam examples 8
3 ICP point cloud registration 11
4 Scan context bin encoding and matrix 13
5 Loop closure pose uncertainty change 15

6 Clearpath Husky UGV, source: [1] 18
7 Velodyne VLP-16 LiDAR, source: [2] 19
8 Nvidia Jetson Xavier NX, source: [3] 19
9 Inertial Measurement Unit, Model: 3DM-GX5-25, source: [4] 20
10 Leica RTC360 degree 3D scanner, source: [5] 20
11 Leica MS60 tachymeter, source:[6] 21
12 Cloud Compare Graphical User Interface 22
13 Dataset trajectories 24
14 KISS-ICP process flow diagram 26
15 SC-LIO-SAM process flow diagram 31
16 Asynchronous Multiple LiDAR-Inertial Odometry process flow diagram 33
17 Data processing flow for KISS-ICP 39

18 Bridge at Lörrach (Wisentalbrücke) 42
19 Experimental setup of the robot 43
20 RTC360 ground truth point cloud 44
21 Trajectories obtained for the datasets using KISS-ICP algorithm . . 45
22 Trajectories obtained for the dataset using MA-LIO algorithm . . . 46
23 Trajectories obtained for the dataset using SC-LIO-SAM algorithm 47
24 Trajectory comparison for all datasets 48
25 Side view of the bridge point cloud by MA-LIO, SC-LIO-SAM and KISS-ICP algorithm 48

26 Temporal analysis of point cloud formation for MA-LIO algorithm . 50
27 Temporal analysis of point cloud formation for SC-LIO-SAM algorithm 51
28 Temporal point cloud analysis for KISS-ICP algorithm 52
29 Distance error analysis of point clouds at different time intervals . . 56
30 Ground truth and predicted point cloud comparison 57
31 Distance error histogram 58

1 Introduction

1.1 General Mapping Overview

Mapping is a method in which one collects information about the surrounding environment, processes it, and then represents it in a structured manner. Sensors are used to gather information from the environments. The data collected from the sensors are then processed to produce an environmental map. A 2D map can be produced by means of a 2D Light Detection And Ranging (LiDAR) sensors, 2D Ultrasonic sensors, 2D Radio Detection And Ranging (RADAR) sensors etc. Similarly, a 3D map of the environment can be produced by using a 3D LiDAR sensor, a Sound Navigation And Ranging (SONAR) sensor, a 3D RADAR, cameras etc. We can observe a few of the sample maps created by LiDAR scanners, RADAR, scanners, SONAR scanners and cameras in Figure 1. The different sensors mentioned use different technologies to create an environmental map. LiDAR calculates distances and orientation of objects by calculating the time of flight of the light beams sent from the sensor. Similarly, RADAR measures the time-of-flight of radio waves and SONAR works on the principle of calculating the time-of-flight of sound waves. The information regarding maps is usually stored in the form of frames of 2D or 3D point clouds. Point clouds are a discrete set of points in space which posses point co-ordinates, that are usually represented by Cartesian co-ordinate system [7]. During the process of mapping, the frames of point clouds are aligned by means of algorithms to reconstruct environmental maps.

Robot platforms have been used in-order to navigate an area and collect data for map construction. Robots would require knowledge of its position to construct a map. This is required because we would require the global pose of the robot to combine with the acquired point clouds to align them in order to produce the map of the environment. This process is called scan registration or point cloud registration. Similarly, we would require the environment information where the robot is located, in-order to find the global pose of the robot. The process of requiring the pose of the

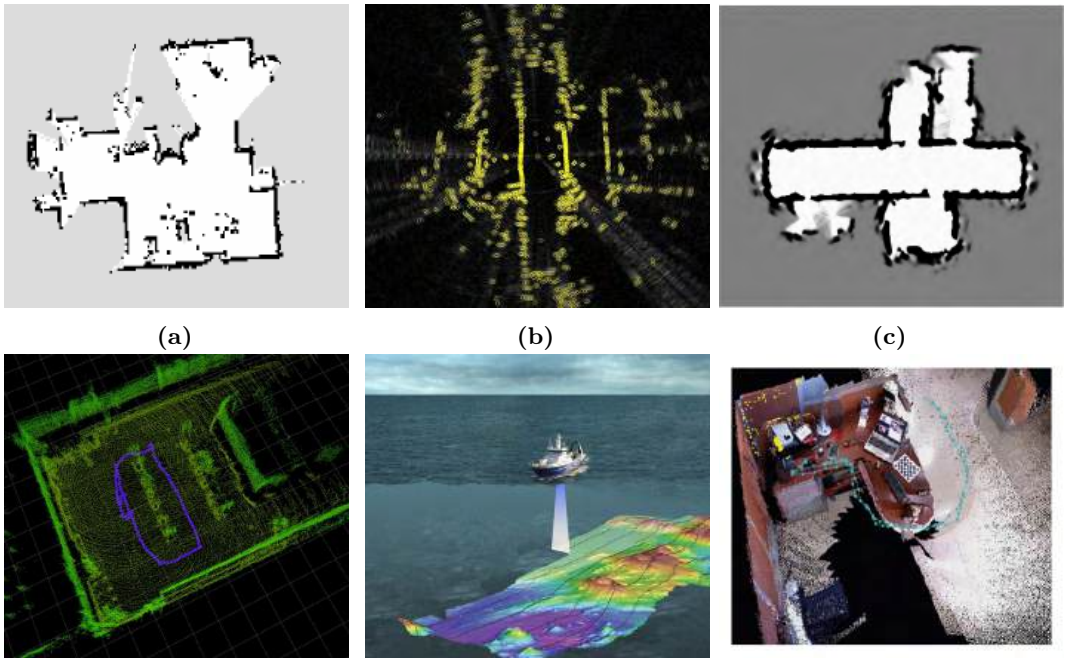


Figure 1: (a). 2D LiDAR scanner grid map, source: [8], (b). 2D RADAR top-view, source: [9], (c) ultrasonic sensor grid map, source: [10] (d). 3D LiDAR map, (e). SONAR map, source: [11], (f). camera point cloud, source: [12]

robot and the map of the environment at the same time is a chicken-and-egg problem. This problem is commonly solved using a concept called Simultaneous Localization and Mapping (SLAM).

1.2 General SLAM overview

SLAM is a fundamental part of robot navigation and mapping and has works on it dating back to the 1990s [13] [14]. In SLAM, one must localize and map the environment parallelly. Several SLAM algorithms have been devised to address the issue of SLAM, initial ones being Extended Kalman Filter based SLAM [15] [16], Unscented Kalman Filter based SLAM [17], grid based SLAM and Graph based SLAM [18] [19]. For the proper functioning of SLAM, sensor information is very vital. Nevertheless, SLAM can be performed by using fixed landmarks, but sensor information plays a vital role in unknown environments where the landmarks are

not fixed. Few of the sensors used during the process of SLAM are electromagnetic technology based sensors such as LiDAR sensors, RADAR sensors etc. These sensors use laser, radio waves in order to gather information from the environment to aid in navigating the environment. Visual sensors such as stereo cameras or monocular cameras can also be used in the SLAM pipeline. In recent times, a combination of both are used to produce better results in SLAM.By fusing the data from multiple sensors, one can reduce the disadvantages of the individual sensors. LiDAR sensors have a short-coming as they can have specular reflection which could cause the point clouds to become distorted, although they have an advantage as they are not affected by intensity changes. The output of RADAR senors are usually noisy, but on the contrary they are less expensive when compared to LiDAR sensors. Cameras on the other hand are affected by intensity changes but do not have issues with specular reflection.

1.3 General project overview

We use a robotic platform to map an environment under consideration, here we use the robotic platform to map an environment containing a bridge. The mapping process is tested using multiple algorithms. Robotic Operating System (ROS) framework is used in the process of data collection and point cloud processing. The data is collected in rosbag files.

We test three different algorithms on the 7 different datasets which were collected by the robot. The three algorithms that we test are, (1). Keep It Short and Simple - Iterated Closest Point (KISS-ICP), (2). Multiple Asynchronous Lidar Inertial Odometry (MA-LIO), (3). Scan Context Lidar Inertial Odometry Smoothing And Mapping (SC-LIO-SAM). The codebases for MA-LIO, SC-LIO-SAM and KISS-ICP are [20], [21], [22], [23] and [24]

The point clouds are then processed in order to transform them to a common frame of reference, which is the tachy frame of reference. The tachy frame of reference corresponds to the ground truth points. The analysis for the point clouds are performed in a software called Cloud Compare and a comparative assessment on the obtained point clouds are reported. The chapters for the report are organized as

follows:

1. **Chapter 1:Introduction.** The introduction to the report is given.
2. **Chapter 2:Related Work.** A literature for the previously accomplished work with regard to the contents of the report is looked into.
3. **Chapter 3:Background.** Description regarding concepts and formulas are detailed here.
4. **Chapter 4:Approach.** The method to solve different processing stages in the project is given here.
5. **Chapter 5:Experiments.** Results and analysis is provided here.
6. **Chapter 6:Conclusion.** A conclusion to the obtained results is detailed here

2 Related Work

The process of SLAM dates back to the earls 1990s [13], [14]. Since then, a lot of work was done in the field of SLAM by the incorporation of different concepts. A few notables of them are EKF based SLAM [15], [16], UKF based SLAM [17], grid based SLAM and Graph based SLAM [18], [19]. The mentioned SLAM methods served as a basis for other advanced SLAM based methods. Graph SLAM based methods are commonly used in large scale SLAM algorithms to produce 2D or 3D maps. Multiple frameworks make good use of graph based SLAM algorithms to enhance their abilities and increase the ease of integration with the overall pipeline of SLAM. A few such mentionable frameworks are Georgia Tech Smoothing and Mapping (GTSAM) [25] and g2o: A General Graph Optimization Framework [26]. SLAM algorithms such as LIO-SAM [27], SC-LIO-SAM [28] work on this principle.

One of the foremost and popular mapping algorithms was LiDAR Odometry And Mapping (LOAM) [29] which focuses on mapping and localization aspect of a robot platform. It does not have loop closure incorporated into it. This causes an issue with respect to SLAM and long distance mapping. It uses LiDAR scanners and IMU to map the environment. The IMU input serves as a motion prior during the mapping process, although the IMU data is not used in the optimization process while build the map [27]. There are other variants of LOAM such as Semantic-aided LiDAR SLAM with Loop Closure (SA-LOAM) proposed by Li *et. al* [30] which introduced loop closure into its pipeline. A more advanced algorithm for SLAM using LiDAR sensor, IMU and GNSS was developed by Shan *et. al* [27].It borrows the concept of scan matching from LOAM. LIO-SAM uses loop closure to recognize previously visited places. LIO-SAM algorithm functions with LiDAR sensors, IMU and an optional GNSS. LIO-SAM generally uses a single LiDAR with IMU for SLAM. Scan matching gives the LiDAR odometry and IMU data is used as a correction factor to correct the LiDAR odometry. Another extension to LIO-SAM was made with the addition of Scan Context for coarse place recognition task in

the mapping process by Kim *et. al* [28] know as SC-LIO-SAM. A tightly-coupled dual LiDAR based Inertial Odometry system [31] was proposed by Xiao *et. al* which utilizes two LiDAR scanners and an IMU sensor to produce a map of the environment.

In situation where multiple LiDAR scanners are to be utilized, Jung *et. al* proposed [32] which is also called as Multiple Asynchronous LiDAR Inertial Odometry. It is a real time mapping framework which uses multiple LiDAR sensors to create a map of the environment. It works off the principle of error propagation through LiDAR scans. It uses Kalman Filters for state prediction and B-Spline interpolation for state linearization. According to the paper, using B-Spline linearization is beneficial in as it provides a good approximation for curves. MA-LIO has the provision to include multiple LiDAR sensors in its base code format. The multiple LiDAR sensors are time synchronized by the use of an adaptive time synchronization algorithm. MA-LIO builds on several concepts from FAST-LIO by Xu *et. al* [33]. It is a tightly coupled LiDAR Inertial Odometry framework. It was developed to be deployed on a mobile platform. It uses an iterative kalman filter for state update and uses a modified method to calculate the kalman gain, which is based on the state of the system rather than the measurement data. This is done so to reduce the computation load due to the high dimensions of the measurements.

Another approach in the field of local mapping was made by Vizzio *et. al* know as KISS-ICP [34]. It is LiDAR odometry pipeline for navigation. It creates a local map of the environment currently being navigated by the robot. It is an odometry only pipeline and does not have loop closures. It uses point-to-point ICP to create a local map of the environment. It is robust in operation, but does not perform well in long range trajectories.

3 Background

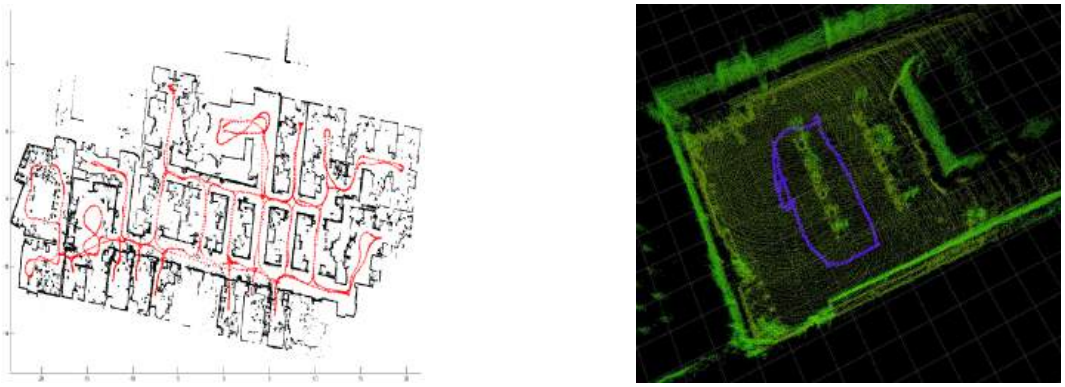
3.1 Simultaneous Localization and Mapping (SLAM)

Simultaneous Localization and mapping is an approach used in the field of robotics to navigate and map an environment. In this approach, a robot must be able to localize itself in an environment and map the environment at the same time. This has proven to be a chicken-and-egg problem, as a robot must know the map of the environment to localize itself in the environment. On the other hand, it must know its position to create an accurate map of the environment. A number of works are attributed to SLAM since the 1990s [13] [14].

In a typical SLAM scenario, a robot gets its positional data from IMU, wheel odometry, GNSS sensors etc. They provide the robot with the x, y, z co-ordinates of the robot and also the heading of the robot which is usually represented in Euler angles or quaternions. The robot also gets information of the environment through visual sensors such as LiDAR sensors, RADAR sensors, SONAR sensors, cameras etc. The information from these sensors are utilized to generate a map of the environment. In order to produce an accurate map of the environment, the data obtained from visual sensors must be corrected using information from sensors such as IMU, GNSS sensors. This process is necessary as the robot during navigation would accumulate drift and would result in producing false maps of the environment.

An important concept in SLAM is loop closure, which is discussed in Section 3.9. In this concept the previously visited place by a robot is compared with the current place, if the place is found to be similar an optimization process is carried out to correct the map of the robot. Loop closure is at the backbone of the SLAM process.

Multiple works have been done in the field of SLAM, from few early works leading to the development of EKF based SLAM [15] [16], UKF based SLAM [17], grid



(a) (b)

Figure 2: LiDAR SLAM examples (a) represents SLAM performed using input from a 2D LiDAR scanner, source: [35], (b) represents SLAM performed using input from a 3D LiDAR scanner, source: [36]

based SLAM and Graph based SLAM [18] [19]. More advanced SLAM algorithms being LIO-SAM [27], SC-LIO-SAM [28]. Real-Time Appearance-Based Mapping (RTAB Mapping) [37] is a camera, LiDAR based Graph SLAM approach based on incremental loop closure detection. It can be used to produce high resolution maps of the environment by fusing information from LiDAR scanners and cameras.

3.2 Georgia Tech Smoothing and Mapping (GTSAM)

GTSAM [25] is a graph optimization framework which works on the basis of factor graphs. A general structure of this framework can mainly be attributed to the existence of nodes and its associated factors. A node can contain multiple factors, of which few examples are IMU factor, LiDAR factor and GPS factor. Nodes are represented by symbols or variables. The variables can be numeric in nature (e.g. 1, 2, 3, ...) or symbolic (e.g. a, b, c, ...) [25]. Nodes are connected to each other by constraints. Factors convey information generally regarding the information present in the nodes. A few examples of factors are IMU factors (contains IMU information such as pose of a body frame, linear velocity, angular acceleration, biases), LiDAR factor (they contain information regarding the point clouds of the LiDAR scanner), GPS factor (they contain information regarding GPS co-ordinates of the respective body frame). A brief description of a few of the factors are given below.

3.2.1 IMU Preintegration Factor

IMU Preintegration Factor is used to incorporate IMU data to a node of GTSAM [25] [27]. The IMU factor consists of pose information, linear and angular acceleration information, information regarding IMU bias etc. Each of the components have a noise component associated with it, indicating the presence of uncertainty in the measurements. Each IMU factor also has a prior factor which corresponds to the initial state of the body frame. Usually in the case of IMU factors, a process called IMU Preintegration is performed before adding the IMU factor. This process is necessary as IMU publishes data at a very high frequency (100 - 500 Hz). A detailed explanation regarding IMU Preintegration is given in 3.11. The IMU factor can be added to the node of the graph, which can later provide information during the process of graph optimization.

3.2.2 LiDAR Odometry Factor

The LIDAR Odometry factor [25] incorporates LiDAR sensor data into the factor graph. LiDAR sensor data consists of the 3D point cloud data (i.e. the x, y z co-ordinates of the scan of the environment). Similar to the IMU factor, LiDAR odometry factor is also added to the factor graph.

3.3 Lidar Inertial Odometry

Lidar Inertial Odometry is the procedure in which scans obtained from the LiDAR sensor are fused with IMU data [27], [38], [31]. A static LiDAR scanner provides data to produce a map of the environment. When the LiDAR scanner is in motion, the map produced by the LiDAR scanner maybe distorted during the scan matching procedure, if the scan matching procedure only takes into account feature points and odometry information from a robot platform. Factors such as slippage of the wheels, external vibrations experienced by the LiDAR sensor can cause distortions in the map construction procedure. Coupling the LiDAR sensor data with IMU data produces stability in the localization of the robot platform. This process is commonly known as sensor fusion.

There are two notable variants in LiDAR Inertial Odometry, they are Loosely coupled LiDAR Inertial Odometry and Tightly Coupled LiDAR inertial Odometry.

In loosely coupled LiDAR inertial Odometry the LiDAR and IMU measurements are processed separately and then fused together. A common method is to obtain the pose of the LiDAR and then fuse it with the processed IMU measurements [33]. In tightly coupled LiDAR inertial odometry the raw feature points with the IMU data [33] [31].

There are two methods in which tightly coupled LiDAR odometry is carried out. Optimization based and filter based. Optimization based techniques optimize poses and sensor data with constrains imposed on them. Usually a pose graph is constructed and sensors are incorporated into the pose graph and optimization process is carried out [27]. Filter based methods use particle filters and it's variants or Kalman filter and it's variants to fuse LiDAR and IMU data [33].

3.4 Iterative Closest Points (ICP)

The iterative closest points are used in the context of scan matching. ICP is done usually to align scans. The point clouds maybe 2D or 3D in nature. The number of points in each point clouds can be different. The ICP algorithm would try to initially find corresponding points among the two point clouds. In 2D points, the correspondences can be obtained as normals between the point clouds or through other means. In 3D point clouds, the point correspondences are usually obtained in the form of features. The features include corners, edges, surfaces. The features can be obtained through geometric methods as done in [27], [33], [29]. A general algorithm for ICP is given in Algorithm 1 [39].

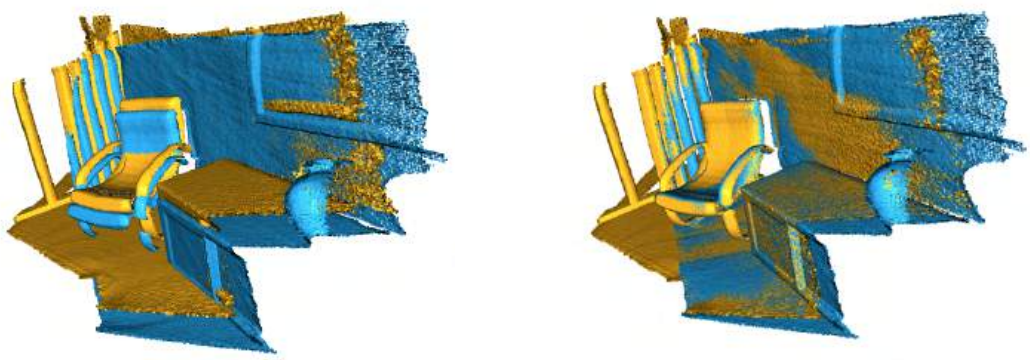
Here R represents the rotation component, t represents the translation component, c represents the scaling, μ_s represents the centroid of source point cloud, μ_g represents the centroid of target point cloud, σ_g^2 represents variance of source point cloud, σ_s^2 represents variance of target point cloud, Σ represents the covariance of the source and target point cloud. The algorithm describes a simple ICP, where we need to obtain the rotation and translation of the source point cloud in order to align with the target point cloud. Figure 3 depicts ICP point-to-point registration. As we can observe in Figure 3a the source point cloud is not well aligned with the target point cloud. After ICP is run on the two point clouds, we can notice that they are aligned in Figure 3b

Algorithm 1 Basic ICP algorithm

Data: Source point cloud ${s_i}_{i=0}^{i=N-1}$, Target point cloud ${g_i}_{i=0}^{i=N-1}$
Result: To find c, R, t that minimize $\sum_{i=0}^{i=N-1} \|g_i - cRs_i - t\|^2$
Calculate centroid of the point clouds
$\mu_s = \frac{1}{N}\sum_{i=0}^{i=N-1} s_i$
$\mu_g = \frac{1}{N}\sum_{i=0}^{i=N-1} g_i$

Calculate variance of the point clouds
$\sigma_s^2 = \frac{1}{N}\sum_{i=0}^{i=N-1} \|s_i - \mu_s\|^2$
$\sigma_g^2 = \frac{1}{N}\sum_{i=0}^{i=N-1} \|g_i - \mu_g\|^2$

Calculate covariance between the point clouds
$\Sigma = \frac{1}{n}\sum_{i=1}^{n}(s_i - \mu_s)(g_i - \mu_g)^T$
Singular value decomposition: $\Sigma = UDV^T$
if $det(U)det(V) < 0$ **then**
    $W = diag(1, 1, -1)$
else
    $W = \mathbf{I}_{3x3}$
end if
$R = UWV^T$
$c = \frac{1}{\sigma_s^2} trace(DW)$ or c = 1 if the scale is known
$t = \mu_g - cR\mu_s$



(a) Before ICP registration

(b) After ICP registration

Figure 3: The blue point cloud represents the target point cloud and the yellow point cloud represents the source point cloud. The figure depicts point-to-point registration, source: [40]

3.5 Kalman Filters

Kalman Filters are used to predict states of dynamic systems on the basis of given control inputs [10]. The states and control inputs of a Kalman filter are denoted as $\mathbf{x}$ and $\mathbf{u}$ respectively. Kalman Filters usually work on linear systems. In most cases, the modeled systems are non-linear in nature. In such cases other variants of Kalman filters such as Extended Kalman Filters (EKF), Unscented Kalman Filters (UKF) are used. Kalman filtering generally works as a twofold process: (1). prediction step, (2). update step.

Prediction step:

In the prediction step, the state for the next time step is obtained by passing the current state of the system and the control inputs for the current state through the dynamics function of the system. In this step, the uncertainty for next state is also updated.

Correction step:

In the correction step, the sensor measurements obtained from external sensors are incorporated to a measurement model. A Kalman gain is calculated on the basis of a measurement model [10]. The Kalman gain is a deciding factor to trust either the robot dynamics model or the measurement model. A small Kalman gain means the trust is placed on the robot dynamics model (this occurs when there is lack of sensor information, i.e. cases such as tunnels, under the bridge) and a large Kalman gain means that the trust is placed on the measurement model. The state and the co-variances are updated according to the Kalman gain.

3.6 Iterated Error State Kalman Filter

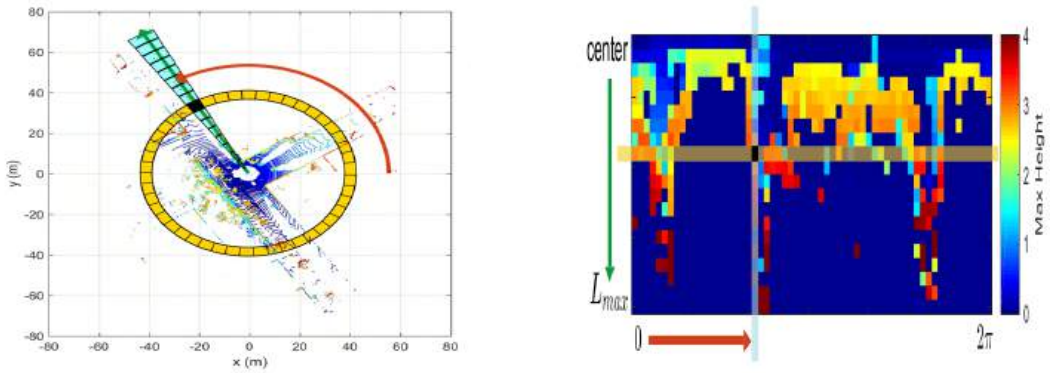
An Extended Kalman Filter is used in situations where states and control inputs do not have a linear relationship between them [41]. In situations of long horizon tasks, here in the case of SLAM which is done for a longer duration, EKF accumulates errors in the pose of the robot during localization. An error state filter corrects the initial guess of the pose, thus enabling a better localization of the robot.

3.7 Basis Spline Interpolation

A spline is a curve which passes through a number of defined points. B-Splines form a special case of splines in the case of Bézier curve generalization [42]. B-Splines are made up of curves that are polynomial in nature, which have the same degree as that of the B-Splines.

B-splines are commonly used in computer graphics in the creation of trajectories, in interpolation of data, in polynomial curve fitting [42]. In our project, MA-LIO uses B-Splines to interpolate between IMU data and to create trajectories.

3.8 Scan Context



(a) **(b)**

Figure 4: (a) describes the bin division of LiDAR scan in scan context along azimuthal and radial direction, (b) describes a matrix heat map of the value of the bins as a function of the point having highest z-value

Scan context is a global 3D feature descriptor used commonly for LiDAR scans. It does not utilize histograms to encode scene information, but encodes the information in a matrix. The method of scan context is rotation invariant and robust to noise. It is commonly used as a visual feature descriptor for scan matching [28].

In scan context, LiDAR scan points are encoded into bins and each bin is assigned a value based on the maximum height (z value) of the points present in the bin [43]. We can observe the bin division for LiDAR scans along the azimuthal and radial directions in Figure 4a and matrix heat map for the encoded bins in Figure 4b. For

the purpose of matching scans, a similarity scone is calculated between different scans context by utilizing cosine distances [43]. In order to search for a loop, a concept of ring keys is used, where each scan context gathers key values for each row of the scans [43]. The key values are compared with key values of other scans to obtain the closest value according to a threshold criterion, which then contributes to a loop closure [43].

The method of scan context can be added to different SLAM algorithms to enable place recognition in a more effective manner, with regard to its advantages in situations involving rotation invariance and robustness. In our studies we use SC-LIO which has a background of LIO-SAM and a place recognition system provided by scan context header.

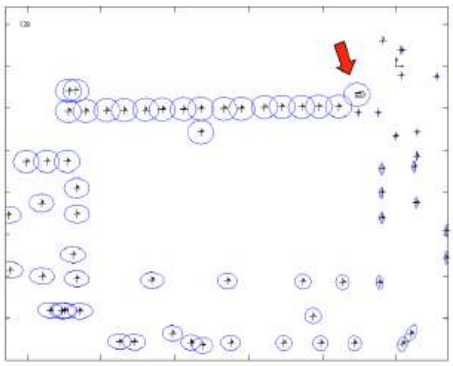
3.9 Loop Closure

Loop Closure is a concept in which an agent would recognize a particular scene in a given environment after traversing the environment [10], [27]. An odometry only pipeline helps an agent well in the process of navigation in the current state, as it would provide information of the environment surrounding the agent. It gathers information regarding the surrounding environment in the form of a map, which is built by utilizing data from various sensors. In the scenario of producing a global map it becomes necessary to correct the acquired data as there is a possibility of drift affecting the robot's pose which in-turn can affect the process of mapping.

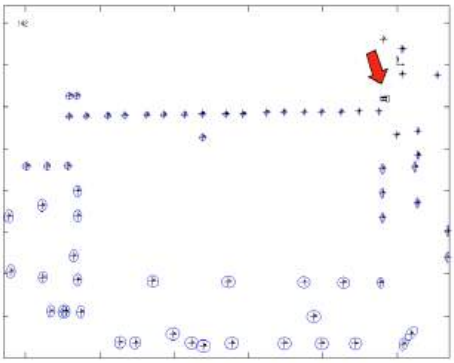
The principle of loop closure lies in recognizing a previously visited landmark or a previously visited location, which is known as place recognition. In case of test scenarios, the landmarks can be marked or a database of visual landmarks can be stored. In the case where LiDAR sensors are used, features such as edges, surfaces and corners are used in place recognition. A robot in the current state does a search if the current place where the robot is present has been visited by the robot at an earlier stage. If the robot had visited the place at a previous point of time, an optimization process is carried out to correct the scan. Performing loop closure reduces the uncertainty in the pose of the robot, which in-turn leads to a better map. Figure 5 describes the effect of loop closure on the pose uncertainty of a robot. The pose uncertainty in the figure is represented by an ellipse. Initially the pose uncertainty is high but once the robot recognizes a previously visited place which in this case is a

landmark, the pose uncertainty reduces as shown in Figure 5b. This in-turn corrects the pose of the robot, thus improving the scan matching. Correct loop closures are necessary. False loop closure can lead to scan miss-alignment and lead to map distortions.

There are multiple ways in which the process of loop closure can be carried out. In case of using graph based methods, the loop closure is recognized using loop closure constraints which are added at a node where the loop closure in recognized [27], [33], [31]. SC-LIO performs loop closure using Graph based methods by using GTSAM framework [27], [25]. KISS-ICP is an odometry only framework and does not have loop closure. MA-LIO is an odometry based framework but involves correction through means of IMU integration.



(a) Robot pose uncertainty before loop closure



(b) Robot pose uncertainty after loop closure

Figure 5: (a) describes the poses of the robots with their associated uncertainty before loop closure, (b) describes the poses of the robots with their associated uncertainty after loop closure

3.10 Velodyne LiDAR frame

The velodyne LiDAR sensor takes a 3D scans of the environment and parses the data for processing. Several software packages are available to decode the data and visualize the point clouds generated by the LiDAR scanner. ROS has dedicated packages for the task of visualizing data from the LiDAR scanner. The point clouds from the velodyne LiDAR can be extracted by using ROS velodyne drivers. The point cloud information of the velodynes are published as topics while using ROS. One of the main topics for obtaining velodyne information is *velodyne_points*. Velodyne

contains the 3D points obtained by the LiDAR sensor in a frame basis. One frame consists of a set of velodyne points with other information like the x, y, z coordinates of the point, the intensity of the point in RGB value, the timestamp of the point, the ring number of the point. The ring number corresponds to the scan line the corresponding point belongs to. For a VLP-16 velodyne the row number ranges from 0 to 15.

3.11 Procedure for IMU factor integration

IMU's usually operate at a frequency of 100 - 250 Hz, providing linear accelerations and angular velocities. State estimation at high frequency can become a computationally intractable task. Hence, the process of IMU pre-integration is carried out to pre-integrate IMU measurements

1. The IMU gives orientation data of the body in the form of quaternions. It also gives information regarding linear acceleration and angular acceleration. It also gives the co-variance matrices related to angular and linear accelerations.

2. To integrate IMU factors, a method called IMU pre-integration is carried out where all IMU measurements are collectively recorded for a specified duration of time. During this time, we must also set an initial value for IMU biases.

3. An optimization step for the graph is carried out after a specified time interval. At this time, the IMU pre-integration is reset.

4. The update here is carried out in an incremental manner using Incremental Smoothing and Mapping (ISAM) method.

4 Approach

4.1 Motivation and general approach overview

4.1.1 Motivation

Robot mapping is a process of obtaining information regarding the surrounding environment by means of a robot and incorporated sensors. A map of an environment is necessary for multiple reasons, a few to name are survey for construction activities, navigation through the environment, to check for structural stability of the environment etc. The end goal of the project is to check for structural deformities in a bridge structure. In this project, we focus on a small part of the major goal, which is to generate a 3D map of the environment. We also focus on assessing the correctness of the map generated. In the later stages of the project, the generated 3D map can be processed and then be utilized by the robot to traverse the environment to check for structural defects.

4.1.2 General Approach Overview

We perform robot mapping in our project to produce a map of a bridge environment. The map is generated by collecting data from sensors such as LiDAR scanners and IMU sensor. The map of the environment is generated by three different algorithms, namely KISS-ICP, SC-LIO-SAM, MA-LIO. The generated map and generated trajectories by the algorithm is then compared with ground truth point cloud generated by a Leica RTC360 degree scanner and ground truth trajectories generated by Leica MS60 Tachymeter. A comparative analysis on the correctness of the generated map is also performed.

4.2 Hardware System Overview

4.2.1 Clearpath Husky Robot

The robot used is a Husky from clearpath robotics. It is an Unmanned Ground Vehicle (UGV). It weighs about 50 kgs and has a maximum speed of $1.0ms^{-1}$. It has a maximum payload capacity of 75 kg. It is powered by a 24V 20Ah rechargeable Lithium-ion battery [44]. It has a maximum operating time of about 8 hrs. It communicates with the onboard computer using RS232 protocol at the rate of 115200 baud. It provides battery status and wheel odometry as feedback to be observed [44]. It has been modified from its base form to include other sensors such as velodynes, stereo camera, and other inspection cameras.

Figure 6: Clearpath Husky UGV, source: [1]

4.2.2 Velodyne sensor

The husky uses a LiDAR scanner to obtain point clouds of the environment. The LiDAR scanner used here is the velodyne VLP-16. It has 16 scan lines. The horizontal resolution of the LiDAR scanner is 0.2° and the vertical resolution is 20°. It operates at a variable frequency of 5 - 20 Hz. It has a maximum range reading of 100 m [45]. It has a horizontal field of view of 360° and a vertical field of view of 30° [45].

Figure 7: Velodyne VLP-16 LiDAR, source: [2]

4.2.3 Nvidia Jetson Xavier NX

The main computer of the robot is a Zed box with Nvidia jetson xavier nx. It has an internal memory of 16GB eMMc and an extended memory of 256GB [3]. It has a USB 3.1 port, a USB 2.0 port, a 5 - 12V power supply port, an OTG port for MicroUSB connection, a GbE Ethernet (RJ-45, PoE) connection and an HDMI port for display [3].

Figure 8: Nvidia Jetson Xavier NX, source: [3]

4.2.4 Inertial Measurement Unit (IMU)

The IMU gives the pose information of a platform in terms of quaternions for angles and x, y, z co-ordinates. The IMU unit used here is Microstrain 3DM-GX5-25. It outputs the co-ordinates at a maximum frequency of 250 Hz according to the data sheet. It contains a fully calibrated and temperature compensated triaxial accelerometer, gyroscope and a magnetometer.

Figure 9: Inertial Measurement Unit, Model: 3DM-GX5-25, source: [4]

4.2.5 Leica RTC360 degree scanner

Figure 10: Leica RTC360 degree 3D scanner, source: [5]

The Leica RTC360 degree scanner captures an environment in 3D. It produces a

high definition colored 3D point cloud of the environment. The point cloud produced by the scanner can contain over a million points [5]. It can produce a complete scan of the environment in about 2 minutes [5]. It can be mounted in the environment under test by a tripod stand. In our experiments, we use the RTC360 degree scanner to produce ground truth 3D point clouds of the environment. The ground truth scans serve as a reference point cloud when performing point cloud analysis with generated point clouds from algorithms.

4.2.6 Leica MS60 Tachymeter

Figure 11: Leica MS60 tachymeter, source:[6]

The Leica MS60 is a multipurpose survey and measurement equipment. It has a scan rate of 30 Hz [46]. The maximum range that it can scan is based on the frequency that it operates on. In our project, the tachymeter was used to obtain the ground truth trajectories by the aid of tracking the motion of a mounted prism on the robot. It was also used to obtain the positions of measurement targets, which was used to mark the tachy coordinate frame.

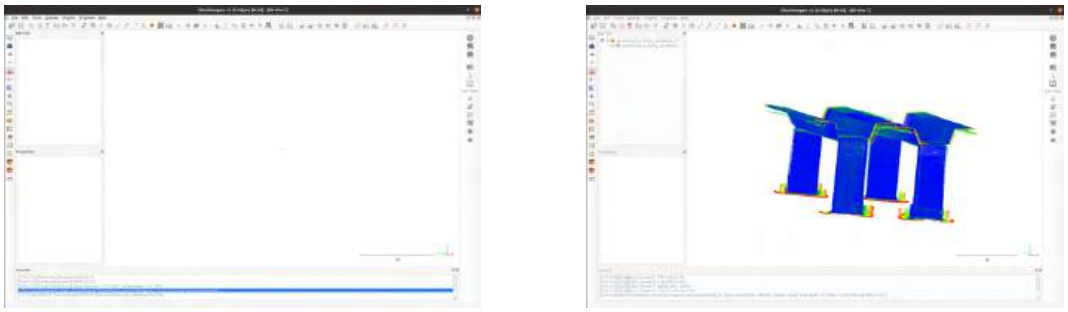
4.3 Software System Overview

4.3.1 Robotic Operating Systems

Robotic Operating system (ROS) is an open source framework used to ease the tasks of sensor data processing, robot navigation, robot manipulation, robot communication etc. It contains a large set of packages to simplify complicated task tasks in robotics. Few of such tasks are namely sensor data retrieval and processing, basic time synchronization between sensor data, creation of a transformation tree, robot path planning, robot manipulation, data collection and storage etc. The ROS version we are using in the project is ROS Noetic Ninjemys.

ROS is compatible to be used with both C++ and Python. There are multiple other commands in ROS. A few of them commonly used in the project are explained below. When using ROS, the data can be accessed through ROS topics. They are the means by which data is communicated to the user and the system in use. We can subscribe to data using a subscriber topic and publish data using a publisher topic. If we have to execute multiple commands, or execute multiple files, we would use **roslaunch**. For recording the data, we can use a command called **rosbag**. Rosbag stores data on the basis of *rostopics*. The rosbags can be replayed to obtain information regarding the topics.

4.3.2 Cloud Compare



(a) **(b)**

Figure 12: Cloud compare GUI. (a) Cloud compare GUI with no data points loaded, (b) Cloud compare GUI with point clouds produced by clockwise dataset through MA-LIO algorithm

Cloud Compare is an open source 3D point cloud and mesh processing software licensed under GNU General Public License (GPL) [47]. One can process 2D and 3D

point clouds. It enables one to perform a variety of point cloud processing operations such as downsampling, filtering, cropping, point cloud comparisons etc. In our project, we use Cloud Compare to analyze the 3D point clouds present in the collected datasets. We perform operation such as downsampling, filtering, cropping to preprocess the raw 3D point clouds obtained from the datasets to make them suitable for point cloud analysis operations. The GUI of Cloud Compare can be seen in Figure. 12a and Figure. 12b shows a point cloud obtained by running the clockwise dataset through MA-LIO algorithm.

4.4 Dataset

The datasets are collected in the form of rosbag files. In the rosbag files, the following contents are stored.

- **2D LiDAR scan:** A 2D LiDAR scan which can be used to generate a 2D map of the environment.
- **3D Lidar scan as velodyne points:** Contains x, y, z points in space, intensity values, timestamps and ring number as obtained by the LiDAR scanner.
- **3D LiDAR scans as velodyne packets:** Contains information of the scans taken of the environment in the form of packets. The packets have to be extracted to obtain information regarding the x, y, z points, their intensity values, time stamps and ring numbers.

There are seven different datasets collected from Lörrach bridge in the form of rosbags. They are described below:

1. **Clockwise dataset:** The robot in this dataset moves in a clockwise manner around the pump track. An example of this dataset can be seen in Figure. 13a
2. **Counter-Clockwise dataset:** The robot in this dataset moves in the counterclockwise manner around the pump track. An example can be observed in Figure. 13b
3. **Linear traversal dataset:** The robot moves in an almost straight line in this dataset. An example of this can be seen in Figure. 13d
4. **Through the pump track dataset:** The robot moves through the pump track in this dataset. An example can be seen in Figure. 13e and Figure. 13c

5. **Different start and endpoint dataset:** The robot starts and ends at different points in this dataset. The example can be observed in Figure. 13f

6. **Long Distance dataset:** The robot here moves to a grate span of the bridge. This is the largest dataset collected.

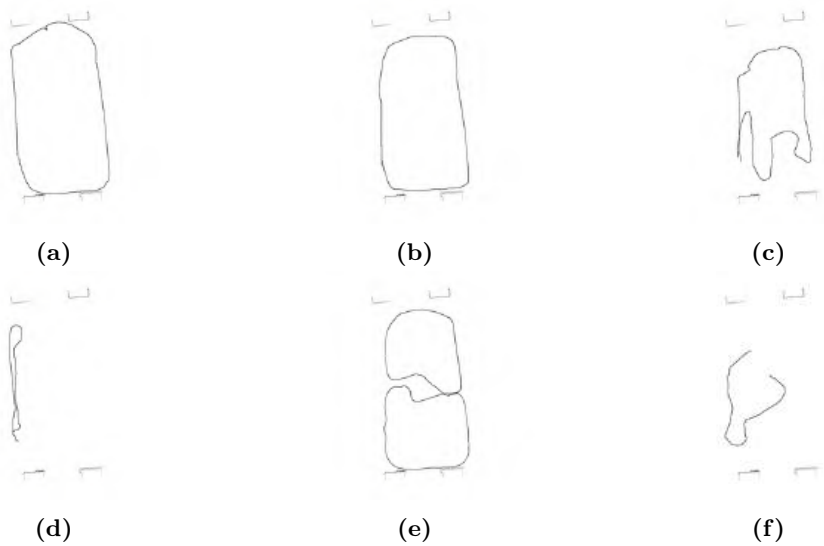


Figure 13: Dataset trajectories from ground truth tachy scan on RTC 360 point cloud data transformed to the tachy co-ordinates. (a). clockwise dataset, (b). counter-clockwise dataset, (c) and (e). through the pump track dataset, (d). linear traversal dataset, (f). different start and end point dataset

4.5 KISS-ICP

4.5.1 Parameters of KISS-ICP

KISS-ICP has hyperparameters which modify the performance of map generation of the algorithm. They are as follows:

Voxel size:

The voxel size describes the size of the individual voxel in length x breadth x height. A small voxel size would be essential to produce a more accurate map as we would have more number of points when processing the data. On the contrary, it would

have a higher overhead on the memory and computation. For our experiment, the voxel size was chosen to be 0.01 cm. The reason for this selection is that a larger voxel size caused a sparse scan and improper scan matching.

Maximum number of points per voxel:

Maximum number of points per voxel correspond to the maximum number of points that can be stored in a voxel before a particular voxel cannot accommodate any more points. The lesser the number of points in a voxel, the sparser the scan will be. This might also lead to loss in few important features which would be essential for scan matching. The maximum number of points per voxel was chosen as 20.

Max range and Min range:

Max range and Min range correspond to the maximum and minimum distance of the local map radius. Data frames accumulated after the threshold values will not be counted during the scan matching procedure, although they will be stored in a voxel grid. The limit on the maximum range can be removed, and all data frames can be used for the scan matching process. Although this does not prove to be a good idea as there is no correction mechanism from external sensors to correct the odometry and thus would cause a large drift in long range scans. The max and min range are set as 100 m and 5 m respectively.

Min motion threshold and Initial threshold

The threshold corresponds to the minimum threshold required to calculate a change in adaptive threshold criteria to obtain nearest neighbor points in KISS-ICP algorithm. The min motion threshold and initial threshold is set to 0.

Deskew:

The deskew option is set to activate scan deskew operation, which would compensate for motion errors during the motion of the LiDAR scanner. We have set the deskew flag to true.

4.5.2 Overview of KISS-ICP

KISS-ICP is an odometry pipeline which uses LiDAR as input to create a local map of the environment. As it is an odometry only pipeline, it does perform loop closures.

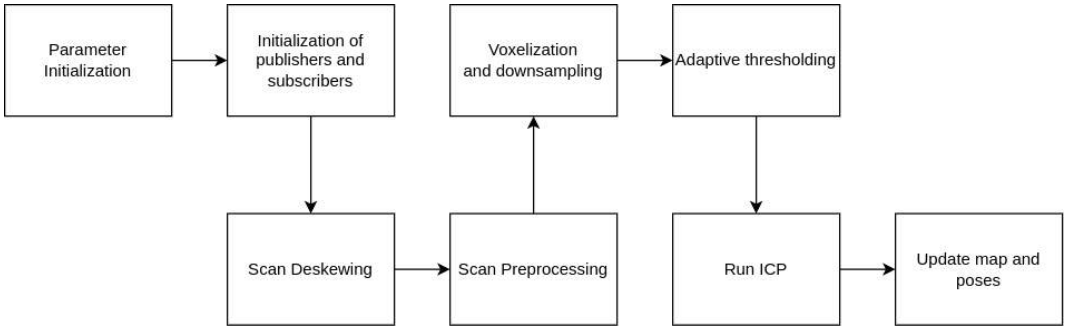


Figure 14: KISS-ICP process flow diagram

As a result of this, a prolonged usage of KISS-ICP in a mapping process can induce localization and mapping errors, causing localization and mapping errors. Mapping errors manifest themselves in the form of dual scans, which is a result of erroneous localization. Figure 14 depicts the general flow of KISS-ICP.

4.5.3 Flow of KISS-ICP

KISS-ICP uses a LiDAR scanner as input for environmental mapping process. We have used Velodyne VLP-16 LiDAR in the project. The LiDAR scanner operates at a frequency of 10 Hz.

Data Acquisition

The LiDAR scanner data is acquired on a frame-by-frame basis as ROS messages through the means of ROS subscribers as a PointCloud2 message format. The LiDAR frame contains information regarding the name of the reference frame, timestamp, LiDAR point data co-ordinates, LiDAR point intensity values, ring numbers.

Scan Deskewing

The process of scan deskewing is done to compensate for the errors induced by the LiDAR during motion. Here, the timestamps are taken into consideration to perform the scan deskew. The process of scan deskew can be skipped by changing the deskew flag in the code base, but it is highly recommended to perform scan-deskewing.

Scan Preprocessing

During scan preprocessing, the frames outside the range set by max_range are not considered.

Point Cloud Subsampling and Voxelization

During the process of Point Cloud Subsampling, the input point cloud is downsampled. The process of downsampling is done using a voxel grid. The procedure of data-association can be performed faster when compared to a dense point cloud as the search space is spatial reduced. The voxel size here chosen to be 0.01 - 1.0 cm.

Local map formation

The local map is formed in an incremental manner. A voxel grid is used to store the contents of the local map. The voxel grid used here is implemented as a hash table for in nearest neighbor search [34]. When a new processed frame is acquired, it undergoes the process of registration and is added to the existing local map. The limit for the maximum number of points in a voxel is maintained. The local map bounds are maintained by removing points outside a maximum range.

Adaptive Thresholding

During the process of point cloud registration, corresponding feature points are identified in the scans and the scans are then aligned according to the pose error [34]. When obtaining the data associations, a nearest neighbor search is performed. A maximum distance criteria is enforced to limit the nearest neighbor search. The maximum distance criteria can either be a fixed value or it can be adaptive in nature [34]. When an adaptive criterion is used for maximum distance thresholding it depends on a number of factors such as the pose error, sensor noise, presence of dynamic objects [34]. Here the error in the pose can be denoted as ΔP. It corresponds to the pose correction that has to be applied during ICP to align the scans. According to [34], the error correction can be deduced from the deviation from the constant velocity model according to Equation (1)

$$\delta(\Delta P) = \delta_{rot}(\Delta R) + \delta_{trans}(\Delta t) \tag{1}$$

where $\Delta R \in SO(3)$ and $\Delta t \in \mathbb{R}^3$ are the rotational and translation components of

the deviations. We can obtain $\delta_{rot}(\Delta R)$ and $\delta_{trans}(\Delta t)$ can be obtained as per the given Equation (2) and Equation (3) [34].

$$\delta_{rot}(\Delta R) = 2r_{max} \sin\left(\frac{1}{2}\arccos\left(\frac{tr(\Delta R - 1)}{2}\right)\right) \tag{2}$$

$$\delta_{trans}(\Delta t) = \|\Delta t\|_2 \tag{3}$$

To compute the adaptive threshold, τ_t a Gaussian distribution over the values computed for $\delta(\Delta P)$ over the trajectory computed till current time is taken. The condition to be taken care is that the deviation must be greater than δ_{min} [34]. This implies that there are being significant deviation from the constant velocity model. The standard deviation for the deviation of the trajectory is given by the following equation:

$$\sigma_t = \sqrt{\frac{1}{M_t}\sum_{i \in M_t} \delta(\Delta T_i)^2} \tag{4}$$

$$M_t = \{i | i < t \wedge \delta(\Delta T_i) > \delta_{min}\} \tag{5}$$

Robust alignment

The registration is done on the basis of point-to-point ICP. The global pose estimate of the robot is obtained by applying the pose transform of the prediction model to the point clouds P present in the local co-ordinates. We transform the global co-ordinate frame using previous pose estimate, T_{t-1} as depicted by the equation 6 [34].

$$S = \{s_i = T_{t-1} T_{pred,t} p | p \in P\} \tag{6}$$

We obtain the correspondences between the local map Q and the global point cloud frame S in the ICP algorithm. This is updated in every iteration [34]. The correspondences are found through nearest neighbor search. The adaptive threshold τ_t serves as a distance minimum criteria when searching for correspondences [34]. To perform alignment of the local map with the source point cloud, an optimization criterion is carried out as given by the equation below:

$$\Delta T = \underset{T}{\operatorname{argmin}} \sum_{(s,q) \in C(\tau_t)} \rho(\|Ts - q\|_2) \tag{7}$$

where $C(\tau_t)$ is the set of correspondences obtained by nearest neighbors whose minimum distance is limited by adaptive threshold factor τ_t and ρ is a Geman-McClure which is an M-estimator [34]. The source points are updated as per Equation (8)

$$\{s_i - \Delta T_{est,j} s_j | s_i \in S\} \tag{8}$$

This process is repeated until the ICP algorithm converges. The final global pose of the robot is obtained as given by the following equation:

$$T_t = \Delta T_{icp,t} T_{t-1} T_{pred,t} \tag{9}$$

The stopping criteria for ICP algorithm used here is a convergence threshold.

4.6 LIO-SAM and SC-LIO SAM

4.6.1 SC-LIO-SAM parameters

The process of map generation in SC-LIO-SAM can be controlled by means of certain parameters. A few of the parameters of SC-LIO-SAM are as follows:

surroundingkeyframeAddingDistThreshold:

This specifies the translation in meters on the basis of which a LiDAR frame must be added to a node as a keyframe. It is set to 1m, as we want to obtain a good amount of keyframes.

surroundingkeyframeAddingAngleThreshold:

This specifies the angular change in radians on the basis of which a LiDAR frame must be added to a node as a keyframe. It is set to 0.2 radians.

surroundingKeyframeSearchRadius:

The parameter specifies the search radius of a matching keyframe in order to perform the operation of loop closure. It is set to 50.0 m. A larger number can sometimes lead to false loop closure detection.

loopClosureEnableFlag:

The loop closure enable flag is used to enable the process of loop closure. It is set to *true*.

historyKeyframeSearchRadius:

It specifies the search radius within which the keyframes will be considered for loop closure. Here, it is set to 15 m.

historyKeyframeSearchTimeDiff:

It specifies the time duration within which the keyframes will be considered for loop closure. Here, it is set to 30 s for the short scans and 2000 s for long range scan. It is set to 2000sfor the full range scan, as a smaller number caused false loop closure and distorted the map.

LIO-SAM is a Lidar Inertial Odometry based SLAM algorithm. It contains loop closures. It uses information from LiDARs, IMU and an optional GPS sensor to construct a map of the environment. The flow of LIO-SAM is as described below:

The initial process is of IMU preintergration. In this stage, the IMU data is integrated to for a given duration of time to be used for a given key-frame for which it is assigned in the GTSAM pose graph. LIO-SAM deals with the 4 factors, they are as follows: (1). IMU preintegration factor, (2). Lidar odometry factor, (3). GPS factors (4). Loop closure factor. We do not consider GPS factor in our project. It is also an optional factor whose use can be deactivated in the script. The process flow of SC-LIO-SAM is as shown in Figure 17

1. The received IMU data undergoes the process of IMU-Preintegration to be added to the GTSAM pose graph as IMU preintegration factor. This in-turn is used to predict the IMU odometry through the graph optimization process.

2. The received point cloud data is formatted in a structured manner to correctly store the coordinates, intensity values, ring number and timestamps to be used for processing.

3. The initial guess for odometry to start the map optimization process is given by the IMU.

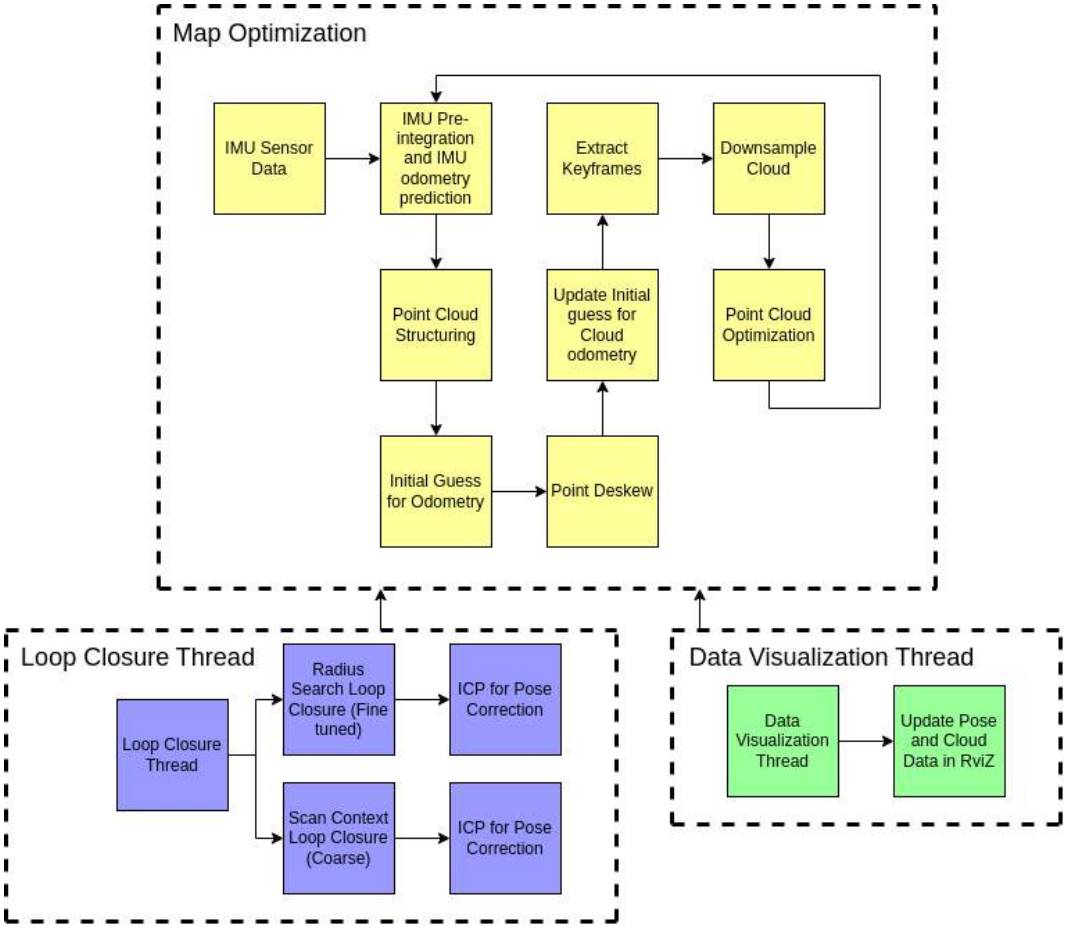


Figure 15: SC-LIO-SAM process flow diagram

4. A point cloud deskew is carried out to correct the positions of the point, which may have been distorted to rapid changes of the velocity of the LiDAR scanner.

5. The point cloud is projected as a range image to be later used for feature extraction.

6. During the feature extraction procedure, the features are extracted on the basis of smoothness of a surface. This is evaluated in a local region. Points with low roughness are classified as planar surfaces, and points with high roughness are classified as edges.

7. The surface and edge point clouds are downsampled and the cloud information data is published for map optimization process.

8. During the map optimization process, the IMU preintegration measurements are used to update the initial guess of the point cloud odometry.

9. The surrounding keyframes required for the map optimization process are extracted, and the point cloud is downsampled.

10. The LiDAR odometry factors, loop closure factors are updated in the factor graph of GTSAM and Levenberg Marquardt optimization process is carried out to correct the pose errors induced during the process of mapping.

11. A loop closure thread is run parallelly to provide for the loop closure factor. There are two types of loop closure calculated (1). Radius search loop closure, (2). Scan context based loop closure.

12. The loop closure here is a Euclidean distance based loop closure. On the addition of a new state to the pose graph, a search is conducted for poses with a threshold of Euclidean distance.

13. Radius based loop closure is performed to obtain loop closure factors for the immediate vicinity of the scans, and scan context based loop closure is performed on a global sense to obtain a coarse loop closure.

14. Both loop closure methods perform scan matching on the basis of ICP for pose correction, thus providing loop closure factors to be added to the factor graph.

15. A data visualization thread is run in parallel to visualize the mapping process.

4.7 Asynchronous Multiple LiDAR-Inertial Odometry

4.7.1 Parameters of Asynchronous Multiple LiDAR-Inertial Odometry

The quality of map generated in controlled by few hyperparameters. The IMU bias, acceleration and velocity covariance are set by default to aid in the Kalman filtering process. The covariance threshold is set to 0.5 to check for convergence criteria for state update according to Figure 16

4.7.2 MA-LIO Description

In Asynchronous Multiple LiDAR-Inertial Odometry also in short MA-LIO algorithm we can incorporate multiple LiDAR's whether they are of the same type or differ-

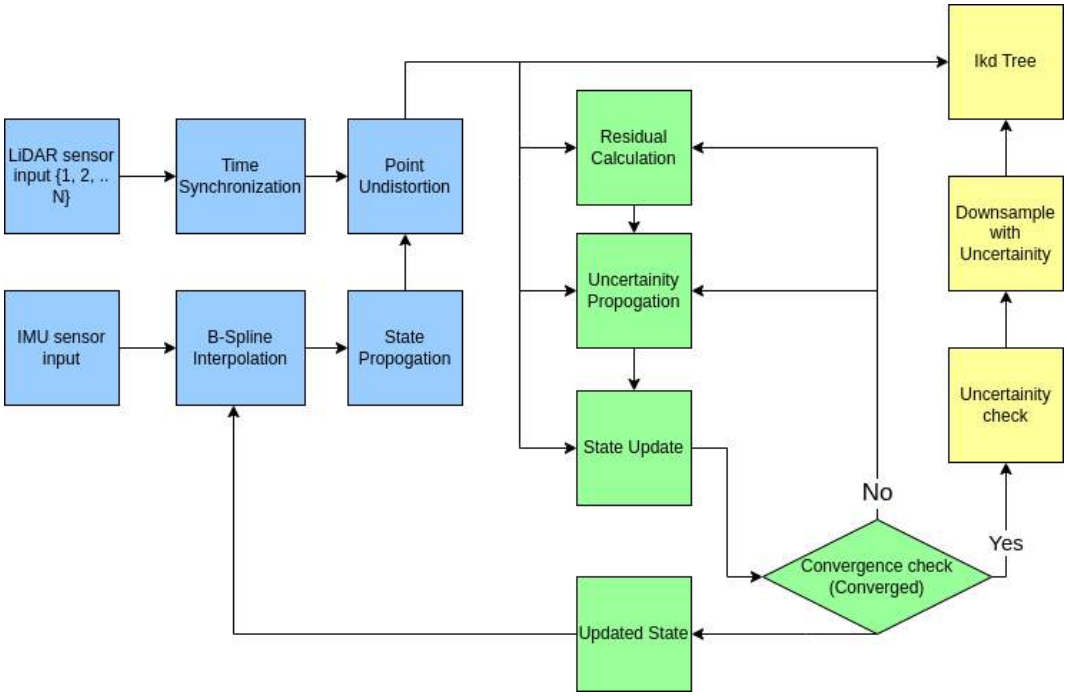


Figure 16: Asynchronous Multiple LiDAR-Inertial Odometry process flow diagram, source: [32]

ent (Velodyne, Ouster, LiVOX). MA-LIO uses software based time synchronization method called as adaptive time synchronization to synchronize the LiDAR scans.

MA-LIO consists of a state vector consisting of IMU biases, IMU data in global rotational and translational components, IMU to LiDAR rotational components $\mathbf{x} \triangleq [\mathbf{R}_{GI}^T \quad \mathbf{t}_{GI}^T \quad \mathbf{v}_{GI}^T \quad \mathbf{b}_\omega^T \quad \mathbf{b}_a^T \quad \mathbf{g}_G^T \quad \mathbf{R}_{IL_i}^T \quad \mathbf{t}_{IL_i}^T]^T$. It consists of a control vector, $\mathbf{u} \triangleq \begin{bmatrix}\omega_m^T & a_m^T\end{bmatrix}^T$ consisting of angular velocity and linear acceleration. It also consists of a noise vector $\mathbf{w} \triangleq \begin{bmatrix}n_\omega^T & n_a^T & n_{b\omega}^T & n_{ba}^T\end{bmatrix}^T$ consisting of white noises.

The continuous time model of the system is as presented in equation 10

$$\mathbf{f}(\mathbf{x}, \mathbf{u}, \mathbf{w}) = \begin{bmatrix} \omega_m - b_\omega - n_\omega \\ \mathbf{v}_{GI} + \frac{1}{2}\left(\mathbf{R}_{GI}\left(a_m - b_a - n_a\right) + \mathbf{g}_G\right)\Delta \mathrm{t} \\ \mathbf{R}_{GI}\left(a_m - b_a - n_a\right) + \mathbf{g}_G \\ n_{b\omega} \\ n_{ba} \\ 0_{3\times 1} \\ 0_{3\times 2N} \end{bmatrix}$$

The model can be discretized and B-Spline interpolation is performed using control points from $\mathbf{T}_{GI}^{k-1}$ to $\mathbf{T}_{GI}^{k+2}$ [32]. By means of this interpolation, we can estimate the trajectory at any given point of time. These transformations can be obtained by using the following Equation (10) [32].

$$^{B}\mathbf{T}_{GI}\left(s\left(t\right)\right) = \mathbf{T}_{GI}^{k-1}\Pi_{n=1}^{3}\exp\left(\tilde{\mathbf{B}}_{j}\left(\left(t\right)\right)\Omega_{k+n-1}\right), \tag{10}$$

where $t_k = k\Delta t$ with $\Delta t = t_{k+1} - t_k$, and $s(t) = (t - t_k)/\Delta t$. Ω_k denotes the incremental control pose which is calculated as $\Omega_k = \log\left(\left(\mathbf{T}_{GI}^{k-1}\right)^{-1}\mathbf{T}_{GI}^{k}\right)$. $\tilde{\mathbf{B}}_j$ is the matrix that includes the square term of s, and notation $^{B}\mathbf{T}$ means that it is calculated by B-spline [32].

4.7.3 LiDAR Data Pre-Processing

The LiDAR scans from multiple LiDAR devices are synchronized using adaptive time synchronization in the beginning. This acts as the first step of LiDAR cloud pre-processing [32]. The LiDAR points are then transformed to a single co-ordinate frame using B-spline interpolation [32]. The distorted LiDAR points are transformed to undistorted points by Equation (11) in the current frame t_l.

$$p_{S^j}^{u} = \mathbf{T}_{IS}^{-1}{}^{B}\mathbf{T}_{I^lI^j}\mathbf{T}_{IS}p_{S^j}, \tag{11}$$

where t_l is the frame of arrival of the latest scan. The latest arrival time by the latest LiDAR is merged by the following transformation [32]

$$p_{P^iS^j} = \mathbf{T}_{IP}^{-1}{}^{B}\mathbf{T}_{I^iI^l}\mathbf{T}_{IS}p_{S^j}^{u} = \mathbf{T}_{IP}^{-1}{}^{B}\mathbf{T}_{I^iI^j}\mathbf{T}_{IS}p_{S^j}. \tag{12}$$

4.7.4 Uncertainty Propagation

The uncertainty is propagated by a point-by-point basis. The co-variances of the Transformations of the LiDAR IMU frame (T_{IL}) and transformations calculated by B-Spline $^{B}\mathbf{T}$ are obtained according to Iterated Error State Kalman Filter (IESKF). With a fourth order approximation, the transformations are co-variances are combined $\{\mathbf{T}_{P^iS^j}, \Sigma_{P^iS^j}\}$ [32].

4.7.5 Measurement Model with Uncertainty

MA-LIO utilizes a surface measurement model with calculating the features. The LiDAR points are transformed into the global coordinates using the equation:

$$\mathbf{p}_{GS^j} = \mathbf{T}_{GI^i}\mathbf{T}_{IP}\mathbf{p}_{P^iS^j} = \mathbf{T}_{GI^i}{}^{B}\mathbf{T}_{I^iI^l}\mathbf{T}_{IS}\mathbf{p}^{u}_{S^j} \tag{13}$$

To approximate a surface, five nearest points are chosen from the ikd-tree. This introduces uncertainty into the measurement model from the ikd-tree [32]

4.7.6 Iterated Error State Kalman Filter

An error state can be obtained from the state propagation components and a measurement model distribution is obtained. They are utilized to get to obtain a maximum a posteriori (MAP) [32]. An iterated kalman filter is used to solve the a posteriori problem. The iteration is carried out until the error between state is below a specified threshold [32].

4.7.7 Mapping with Uncertainty

The global pose of the LiDAR points can be obtained by utilizing the obtained global state. Points are stored in the ikd-tree on the basis of the uncertainty value. The points are first checked for their uncertainty which has to be below a specified threshold and then the points are downsampled and added to the ikd-tree [32].

The process is repeated to construct a map of the environment using IMU and LiDAR scans.

4.8 Data Acquisition procedure

The data to construct the map of the environment is collected by the robot and is recorded in the form of rosbags. The ground truth data is collected by the tachymeter. The procedure in which the data is collected is as follows:

1. The robot is started and is moved to an initial location in the test environment.

2. The sensors are set to record by running the launch files in the terminal of the robot. The launch files are run for the VLP-16 Velodyne scanner and the IMU

sensor. A shell script file is created to record the rosbags and store them with appropriate names. The recording of the rosbag starts when the script is run.

3. The shell script is initially run, and we wait for about 10 seconds for the tachymeter to get the initial position of the ground truth pose of the robot.

4. The robot is then driven around in the test environment by means of a joystick to collect the data.

5. At the end of the recording session, the robot is stopped at a terminal position, and we wait for about 10 seconds for the tachymeter to read the terminal position. The shell script for recording the rosbags is then stopped and the rosbag is saved.

6. This procedure is repeated for multiple dataset session.

7. Care should be taken that the tachymeter can detect the prism at all times, as failure in detection can cause disruption in ground truth data acquisition and the session must be aborted.

8. The data is then preprocessed to produce a suitable point cloud for evaluation, where all point clouds are transformed to a tachy coordinate frame and all points greater than a distance threshold were filtered out.

4.8.1 Creation of environment map using KISS-ICP

KISS-ICP algorithm accepts LiDAR scans as inputs to create a local map of the environment. It does not use any external sensors such as IMU and GPS for map creation. KISS-ICP can be run on a real time basis through ROS. It can also be run on an offline basis by the use of rosbag files. In our experiments, we use rosbag files recorded by the robot to create a map using KISS-ICP. The steps that we follow are:

1. We use the C++ version of KISS-ICP algorithm to process the data present in the rosbag files.

2. The launch file to start the mapping process using KISS-ICP is executed in a terminal. This has to be done first, as the algorithm would wait for the data from the rosbag files.

3. The rosbag file is executed in another terminal. This starts the process of mapping.

4. We wait for the process of mapping to complete. The process of mapping is complete when the rosbag command is exited.

5. The mapping process can be visualized in RViZ.

6. The trajectories are stored in a text file and is specified with a suitable name for the different experiments conducted.

7. The point clouds are stored before the process of double downsampling as a PLY file in order to obtain a dense map of the environment.

8. The procedure to obtain a dense map is described in Section 4.8.4

The parameters of map processing can be set in a configuration file.

4.8.2 Creation of environment map using SC-LIO

SC-LIO takes in as input 3D LiDAR scan and IMU data. A GPS signal can serve as an optional input. But since in the project we are interested to work with datasets of bridges, we assume that there would not be great reception to operate a GPS. SC-LIO can be run on a real time basis on a computing device. In this case, we use the data from the rosbags. The steps to construct a map using SC-LIO are given as follows:

1. We use a single LiDAR and IMU sensor as input to the SC-LIO algorithm.

2. The transformations between the LiDAR to IMU and baselink must be sent in the configuration file. Wrong transformations matrices lead to erroneous maps.

3. The map parameters are set configured in the configuration file.

4. The SC-LIO algorithm can be launched using a launch file, as it has components from ROS.

5. The rosbag is played and this initiates the mapping process. The mapping process can be visualized in RViZ.

6. Once the mapping process is complete, the point cloud representation of the environment is saved as a .PLY file by a function in the algorithm. An external ROS service can also be called to save the map.

7. The trajectories are also saved in a designated directory. The trajectories which are saved are the pose points obtained after the process of optimization. They are saved in the form a .txt file.

8. The SC-LIO algorithm also saves the corner feature points and the surfaces extracted during the map creation process.

4.8.3 Creation of environment map using MA-LIO

MA-LIO takes as input 3D LiDAR scans and IMU data. MA-LIO can take multiple LiDAR data as input. In our experiment, we use two VLP-16 velodyne LiDAR's as input to MA-LIO. Additionally, we use data of IMU as input to the algorithm. MA-LIO can process data in real time. Here in our experiments we use the data recorded from rosbags to get the map of the environment. The procedure to generate a map using MA-LIO is given as follows:

1. Two LiDAR scanners and IMU are uses as inputs to generate a map using MA-LIO.

2. The MA-LIO algorithm is launched using a launch file, and the rosbag file is played in another terminal.

3. The mapping process can be visualized using RViZ.

4. The transformations between the LiDAR's and the IMU must be set in the configuration file.

5. After the mapping process, the point clouds and the trajectories are saved in a designated directory. The trajectories saved are the optimized trajectories.

4.8.4 Data processing: KISS ICP

KISS-ICP is an odometry pipeline for producing a local map during navigation. It employs the concept of double downsampling to process the incoming point clouds at a faster rate. In order to obtain a dense point cloud, we follow a procedure. The flow diagram for the procedure is as given in Figure 4.8.4. The procedure is given below:

1. In the main KISS-ICP algorithm, we store the processed poses of the robot in a text file.

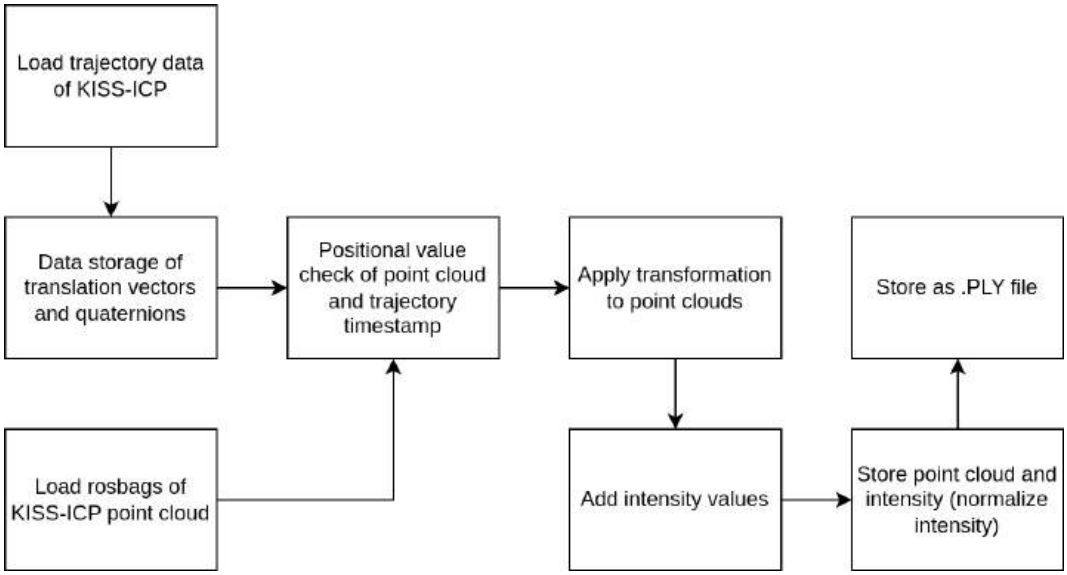


Figure 17: Data processing for KISS-ICP algorithm to obtain a dense scan of the environment

2. After the scan matching procedure in KISS-ICP, the global transformation for the poses are obtained. The poses obtained are the translation of the robot (x, y, z) and the quaternions.

3. We develop a python script to save a dense point cloud for KISS-ICP.

4. The trajectory data is loaded, and the translation vectors are stored. The quaternions are converted into a 3 x 3 rotation matrix and stored.

5. We check the presence of the timestamp of the point cloud in the timestamps of the trajectory points. If the timestamp is present, we obtain the index position of the location of the matching timestamp for the respective LiDAR frame.

6. The rotation matrix and translation vector are obtained accordingly, and the point cloud is rotated and translated according to the values. They are then stored in a point cloud object in Open3D [40].

7. The intensity values are also obtained from the point cloud, normalized and then stored in the Red channel of RGB. The intensity values are also stored in the point cloud object.

8. A .PLY file is stored for every frame obtained, which has its corresponding timestamp in the timestamps of the trajectories.

5 Experiments

A series of experiments are carried out on 3 different mapping algorithms. The algorithms are (1). KISS-ICP (2). LIO-SAM with scan context and (3). MA-LIO. For evaluation, codebases of MA-LIO, SC-LIO-SAM and KISS-ICP have been used. In these experiments, a robot platform acquires environment information using LiDAR sensor and IMU sensors. The environment under consideration is a bridge. A reference point cloud is obtained by using a RTC360 degree scanner. The RTC360 degree scanner produces a colored dense scan of the environment. A tachymeter is used to obtain the ground truth trajectory of the trajectory traversed by the robot.

5.1 Data Acquisition

The data for the experiments is acquired by a robot called clearpath Husky. It contains LiDAR sensors and IMU sensors. The LiDAR sensors acquires 2D and 3D scans of the environment. The LiDAR sensor records the data at 8.8 Hz. The IMU records the pose information (x, y, z) coordinates of the robot and the linear velocity and angular acceleration. The IMU records the data at a frequency of 200 Hz. There are two LiDAR sensors, one having its axis of rotation perpendicular to the direction of the motion of the robot called as the horizontal LiDAR sensor and the one having its axis of rotation parallel to the direction of the motion of the robot called as the vertical LiDAR sensor. The LiDAR sensor and IMU are connected to an Nvidia Jetson Xavier NX device. The data is recorded in the form of rosbags. The

5.2 Location and setup

5.2.1 Location

The dataset for the experiments are collected from a bridge in Lörrach. The location has vegetation surrounding it and it is a partial hilly environment. It also has a cycle pump track under the bridge. The location is as shown in Figure 18

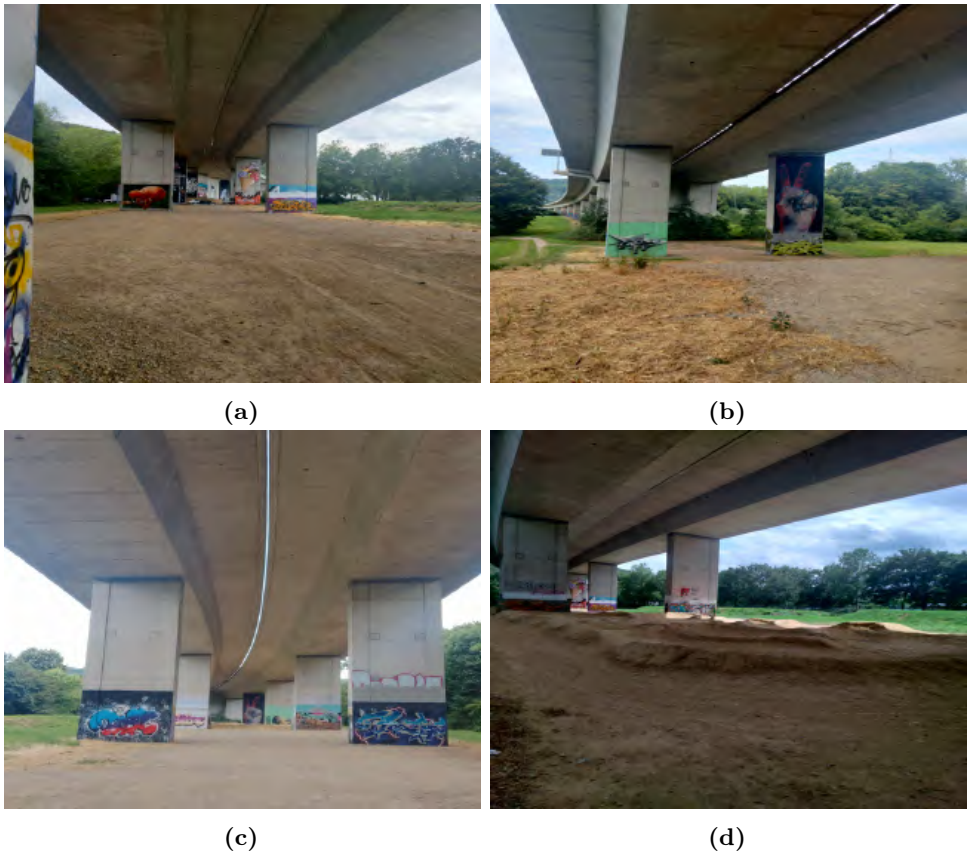
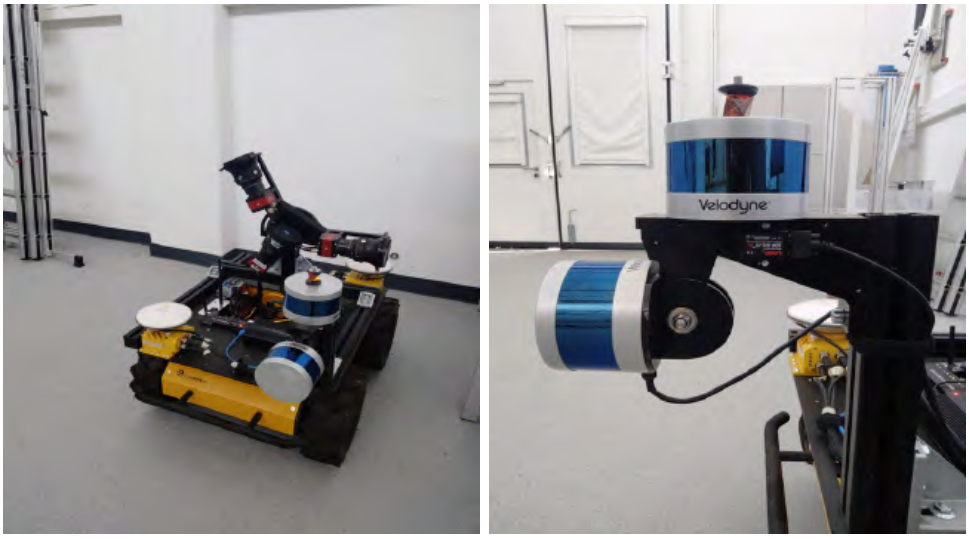


Figure 18: Bridge located at Lörrach (Wisentalbrücke). Here (a), (b), (c) show the images of the bridge, (d) shows the cycle pump track below the bridge

5.2.2 Experimental setup

A prism is mounted on top of the horizontal LiDAR scanner as seen in Figure 19b. The prism enables the acquisition of ground truth data by the tachymeter. The prism must be at all times be visible by the tachymeter, otherwise the process of acquiring the ground truth trajectory by the tachymeter is disrupted. The coordinate frame of the tachymeter is called the tachy coordinate frame. The tachy coordinate frame is obtained by the tachymeter using calibration boards set around the environment. Later in the experiment section, all the point clouds are transformed to the tachy coordinate frame for point cloud comparison analysis. A RTC360 degree scanner is used to record a colored ground truth point cloud of the bridge. The points of the RTC360 degree scanner are also transformed to the tachy coordinate frame. A region of interest in the bridge is a cycle pump track due to its uneven surface. Uneven



(a) **(b)**

Figure 19: Experimental setup of the robot, LiDAR and IMU system. Here (a) describes the overall setup of the clearpath husky robot, (b) describes the vertical LiDAR scanner, horizontal LiDAR scanner and the IMU. The prism used by the Leica scanner to as a frame of reference when generating the ground truth trajectories is mounted on top of the horizontal LiDAR scanner

surfaces help in the comparison study of trajectories of ground truth and the ones acquired by the robot, as there are high chances of miss alignments in such scenarios. The data is acquired by driving the robot under the bridge. Multiple datasets are recorded with different driving trajectories of the robot, as described in Section 4.4.

5.3 RTC 360 Point Cloud

Figure 20: Point Cloud of a bridge taken by a RTC360 degree scanner. The figure show a cropped image of the complete point cloud taken by the RTC360 degree scanner

Figure 20 depicts the point cloud of a bridge section taken by a RTC360 scanner. The point Cloud obtained by RTC360 is a colored dense point cloud. The point cloud is processed by using a software called Cloud Compare. The point cloud is downsampled to ease the process of analysis.

5.4 Dataset Trajectories and Point Clouds using KISS-ICP

The KISS-ICP algorithm is an odometry based pipeline to produce a local map of an area. Due to the process of double downsampling in KISS-ICP, the point clouds obtained for visualization are quite sparse. Hence, the procedure described in Section 4.8.4 is followed to obtain a dense point cloud. The point clouds for different datasets can be seen in the Figure 21. The trajectory points are overlaid on top of the obtained point clouds. As you can see from Figure 21 a substantial amount of deviation is observed from the ground truth trajectory in the case of KISS-ICP algorithm. This can be attributed to the fact that KISS-ICP usually forms local maps of the environment and solely focuses on ICP to build the environment map. This is an advantage when creating local maps for the purpose of robot navigation, but does not

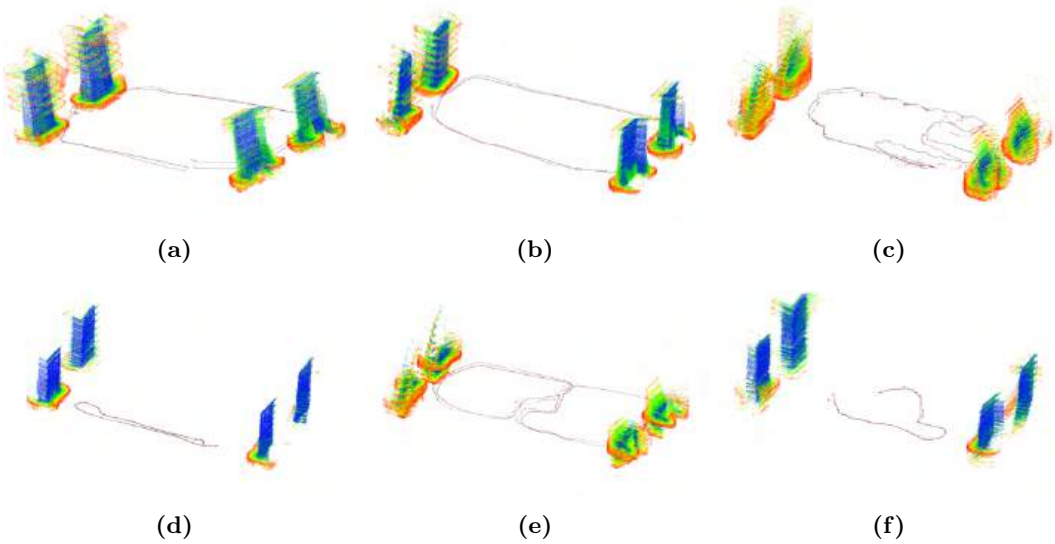


Figure 21: (a). clockwise dataset, (b). counter-clockwise dataset, (c) and (e). through the pump track dataset, (d). linear traversal dataset, (f). different start and end point dataset. The color scale in the figure which spans from blue, green, yellow and red represents the offset of the points from the ground truth in the range of 0.0 m (blue) to 2.0 m (red)

perform quite well in a scenario where a map of an environment is built for traversing purposes at a later time. KISS-ICP does not get input from external sensors such as an IMU or GPS to correct odometry drift, which could attributed to deviations during the mapping process.

5.5 Dataset Trajectories and Point Cloud using MA-LIO

Figure 22 shows the overlay between the point cloud and the trajectories obtained by the MA-LIO algorithm in the tachy coordinate frame.

As seen in the in Figure. 22 an almost accurate map can be obtained of the test environment. This can also be visually observed in the trajectories of the ground truth trajectories and trajectories obtained from the MA-LIO scans. An almost clear overlap can be observed in the two trajectories. This can even be observed in Figure. 22c and Figure. 22f which are datasets where the robot goes through the pump tracks. MA-LIO does not contain a loop closure system. It uses LiDAR scans and IMU measurements and aided with uncertainty propagation and iterated errors state kalman filters for the processing and generation of the environment.

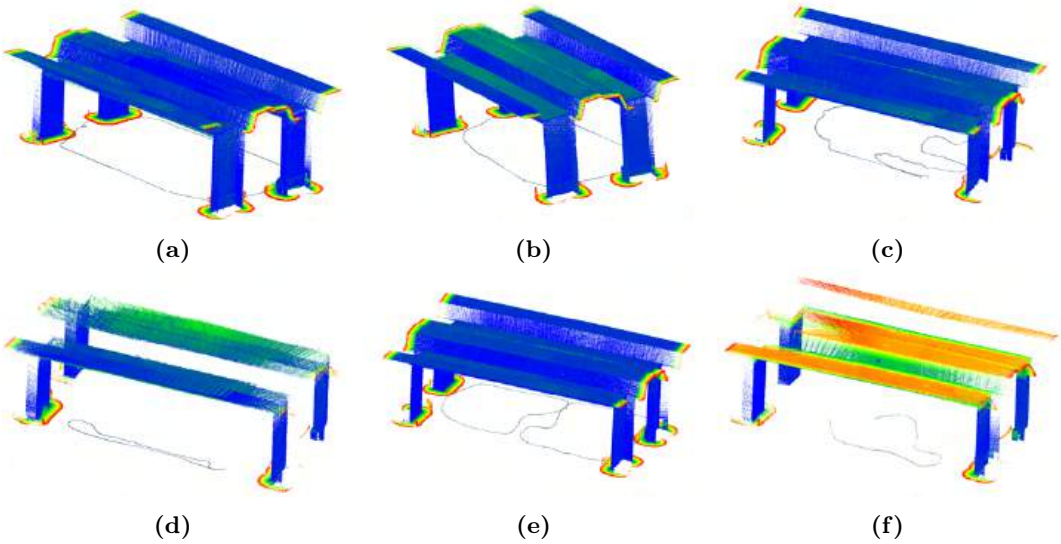


Figure 22: (a). clockwise dataset, (b). counter-clockwise dataset, (c) and (e). through the pump track dataset, (d). linear traversal dataset, (f). different start and end point dataset. The color scale in the figure which spans from blue, green, yellow and red represents the offset of the points from the ground truth in the range of 0.0 m (blue) to 2.0 m (red)

5.6 Dataset Trajectories and Point Cloud using SC-LIO

Figure 22 shows the overlay between the point cloud and the trajectories obtained by the MA-LIO algorithm in the tachy coordinate frame.

As observed in Figure. 23 the ground truth and trajectories obtained from SC-LIO have an almost close match. As we can see from the figure, since SC-LIO uses only a single LiDAR scanner, we can observe only the portion of the bridge support. In SC-LIO one uses an IMU sensor for error correction. It is a SLAM system that depends on graph based optimization loop closure system.

5.7 Dataset trajectory comparison

As observed in Figure 24, the trajectories obtained from MA-LIO, SC-LIO-SAM and KISS-ICP are compared with each other and the ground truth trajectory obtained from the tachy meter. As seen from the figures, the trajectory produced by MA-LIO algorithm closely approximates the ground truth trajectories. This in turn affects the point cloud alignment with the ground truth scan. The correctness of the point

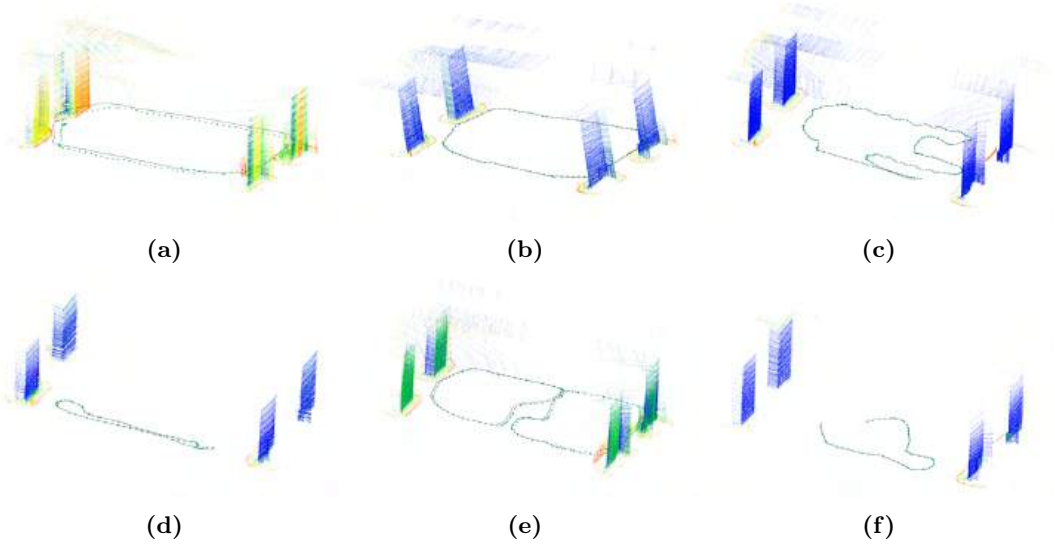
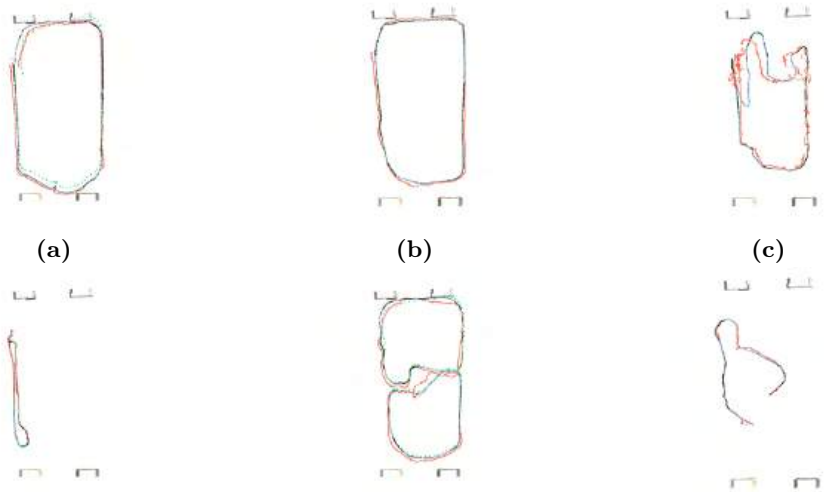


Figure 23: (a). clockwise dataset, (b). counter-clockwise dataset, (c) and (e). through the pump track dataset, (d). linear traversal dataset, (f). different start and end point dataset. The color scale in the figure which spans from blue, green, yellow and red represents the offset of the points from the ground truth in the range of 0.0 m (blue) to 2.0 m (red)

cloud alignments for MA-LIO can be observed in a later section in the histogram plots. Trajectories produced by SC-LIO-SAM show slight deviations in Figure 24a and Figure 24e which correspond to clockwise and through the pump track dataset. This miss-match can also be observed in the distance error histograms. Trajectories produced by KISS-ICP show a great deviation from the ground truth trajectories. The measure of the extent of deviation can be observed in the distance error histogram plot for KISS-ICP. The greatest deviation is show in Figure 24c and Figure 24e which are through the pump track dataset. The deviation can be attributed to erroneous scan-matching in the algorithm. The elevation of the pump track could have also caused the scan matching to be erroneous.

5.8 Cross-section view of long distance scan

A long range dataset was collected spanning a large distance of the bridge. A cross-section view of the dataset can be observed in Figure 25. Figure 25 (a) depicts the cross-section of the bridge obtained by using MA-LIO algorithm. As observed in the Figure 25 the point clouds give a fairly good representation of the structure of the



(a) (b) (c)

(d) (e) (f)

Figure 24: (a). clockwise dataset, (b). counter-clockwise dataset, (c) and (e). through the pump track dataset, (d). linear traversal dataset, (f). different start and end point dataset. Light blue represent MA-LIO trajectory, dotted green represents SC-LIO-SAM trajectory and red represents KISS-ICP trajectory

(a) (b) (c)

Figure 25: Side view of the bridge point cloud by MA-LIO, SC-LIO-SAM and KISS-ICP algorithm (a) point cloud by MA-LIO, (b) point cloud by SC-LIO-SAM, (c) point cloud by KISS-ICP

bridge. Towards the left end of the image, a slope can be observed. This gives a fairly good assessment of the capability of MA-LIO algorithm when mapping elevations. Figure 25 shows the cross-section view of the bridge obtained by using SC-LIO. As observed in the Figure 25b SC-LIO-SAM displays slight difficulty in recognizing elevations. Figure 25 (c) depicts the cross-section of the bridge as obtained by the KISS-ICP algorithm. As seen in the image, does not perform well when there is a presence of elevation in the environment. The map appears to have developed a dip,

indicating wrong scan matching. Although, KISS-ICP does perform well in smaller environments.

5.9 Visual temporal analysis of point cloud formation

In this section, we discuss the results of the formation of the point clouds when the dataset is processed by the algorithm. In cloud compare, a temporal analysis can be performed using scalar point indices to observe the formation of the point clouds. By doing this, one can visualize the deviation of the point clouds when the same region is scanned multiple times during a different instance of time. If the scans taken at different instances of time overlap in the temporal analysis, we can infer that the mapping algorithm performs well in the mapping process. On the contrary, miss-alignment in the scans can be noted by a visual inspection. A gradient in the color can be observed for scans taken at different time intervals.

5.9.1 Temporal point cloud formation analysis of MA-LIO algorithm

According to Figure. 26 we can observe the formation of point cloud data of the bridge under test. In case of MA-LIO, the cross-section of the bridge was taken for temporal analysis. As it can be observed in the sub-figures, blue points represent the initial points during the scan, green points represent the point cloud after a certain duration of time, yellow points represent the subsequent points and the points in red represent the points obtained at the end of the scan process. From Figure. 26 we can observe that the points obtained at the beginning of the scan almost coincide with the points obtained at the end of the scan. A slight deviation can be observed in the points in green to the points in red, indicating a slight scan mismatch.

5.9.2 Temporal point cloud formation analysis of SC-LIO-SAM algorithm

Figure 27 depicts the temporal build up of points to form the bridge using the SC-LIO-SAM algorithm. The blue points indicate the initial set of points and the red points indicate the final set of points. As time progresses, we overlap of points on the same surface. By visual inspection, we can infer that the extent of overlap deviates from the initial set of scans. This indicates a slight miss-alignment of scans by the algorithm. The mismatch of points can be observed in context to Figure 27a

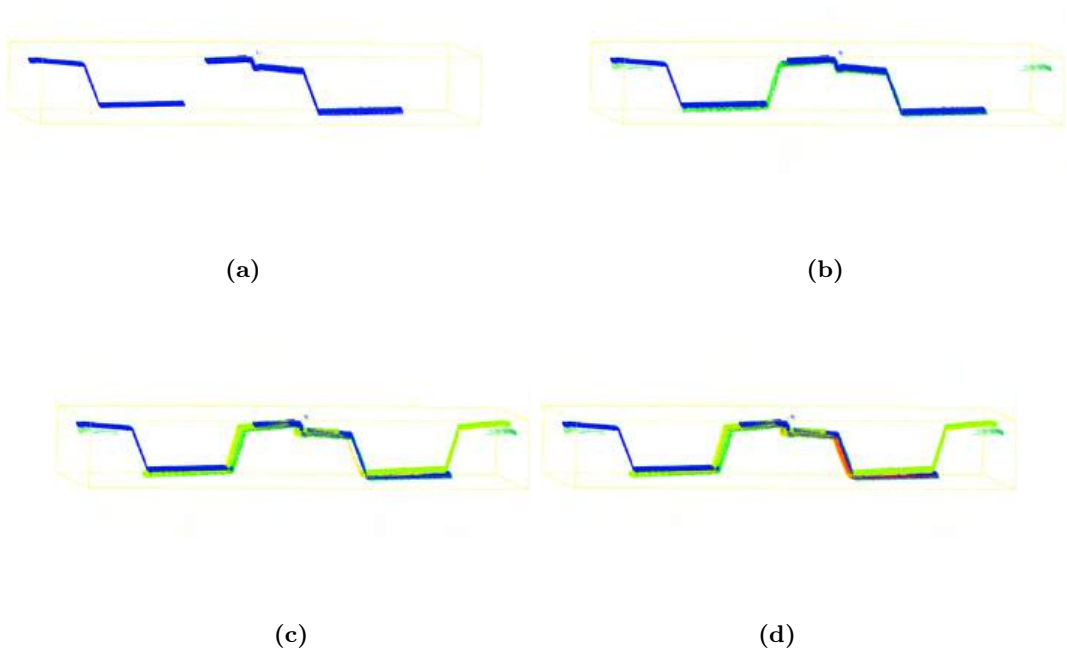


Figure 26: Temporal analysis of point cloud formation for MA-LIO algorithm. (a) describes the initial scan, (d) describes the scan taken at the end of the scanning process, (b) and (c) describe intermediary scan times. Points are colored in blue for initial points. The color then transitions into green, yellow and red points represent the points at the end of the scan.

and Figure 27d.

5.9.3 Temporal point cloud formation analysis of KISS-ICP algorithm

The temporal analysis for KISS-ICP algorithm can be seen in Figure 28. As observed in Figure 28, KISS-ICP does not produce a smooth map of the environment, attributing to the fact that there is no correction during the formation of the map. A large miss-alignment of the scans can be observed for the point cloud results generated by the KISS-ICP algorithm. Although it is found that the point clouds deviate less from previously mapped points in the case of KISS-ICP according to distance error analysis for temporal scans.

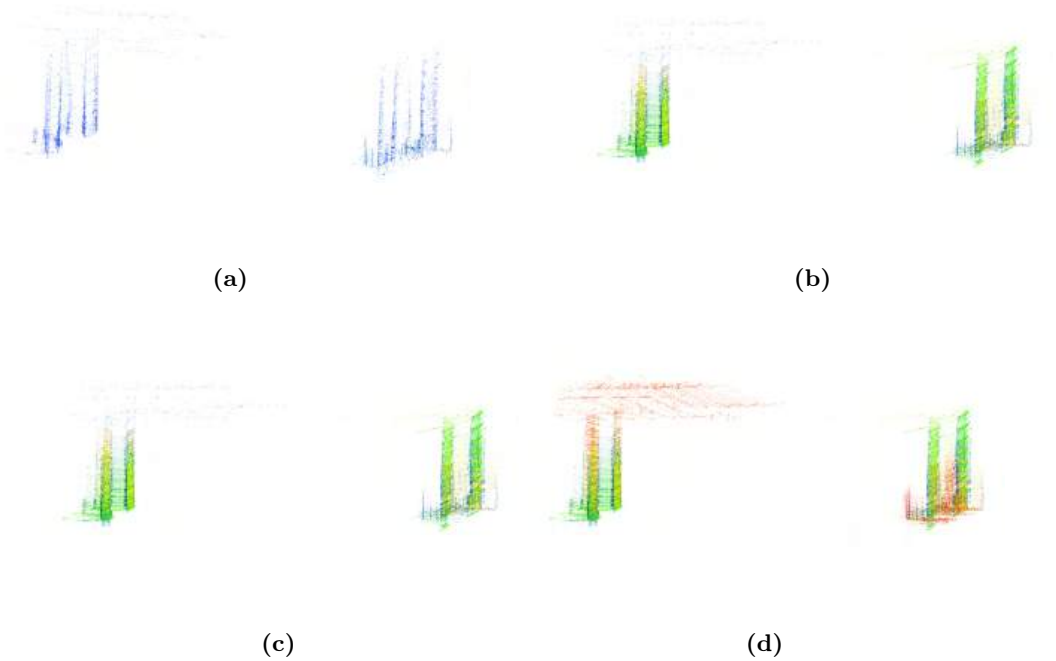


Figure 27: Temporal analysis of point cloud formation for SC-LIO-SAM algorithm. (a) describes the initial scan, (d) describes the scan taken at the end of the scanning process, (b) and (c) describe intermediary scan times. The color then transitions into green, yellow and red points represent the points at the end of the scan. The scan is taken from the side view of the bridge

5.10 Point cloud distance error analysis for temporal scans

As seen in the point cloud temporal analysis section, an offset is observed for the point clouds of the same location taken at different points in time. We perform a distance error analysis of the point clouds to find the offset between the point clouds at different intervals of time. This analysis is performed on the predicted point clouds by MA-LIO, SC-LIO-SAM and KISS-ICP. The ground truth point cloud by Leica RTC360 is not considered for this analysis. The x-axis for the histograms is cloud to cloud distances in meters and the y-axis is the number of points per bin.

For the analysis, the scalar field for the predicted cloud is taken as time indices. Two separate point clouds at different time intervals (one at the beginning which denoted by color blue and one at the end which is denoted by color red) are extracted from

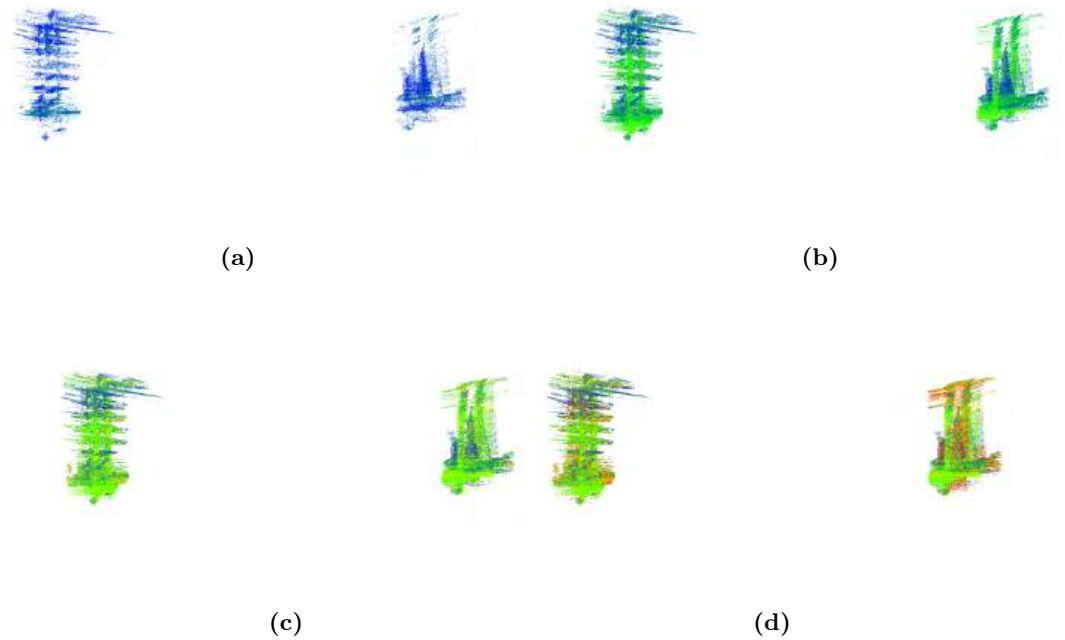


Figure 28: Temporal analysis of point cloud formation for KISS-ICP algorithm. (a) describes the initial scan, (d) describes the scan taken at the end of the scanning process, (b) and (c) describe intermediary scan times. The color then transitions into green, yellow and red points represent the points at the end of the scan. The scan in the images is a side view of the columns of the bridge.

the predicted point clouds of the algorithm. A cloud to cloud distance computation is performed on both the point clouds. A histogram with a Gaussian plot overlaying it is plotted as observed in Figure 29. The analysis is calculated on the clockwise dataset.

For the plot regarding MA-LIO point cloud, most points are within the distance error of 0.2 m, which is reasonable when compared to the struct of the bridge. Although, a large standard deviation is observed in the plot. Nevertheless, MA-LIO was evaluated to produce less distance error and a stable distribution when a distance error was performed between the obtained point cloud and ground truth point cloud.

When compared to SC-LIO-SAM distance error analysis in Figure 29c and 29d, most of the points during the scan process are within 0.6 m. It is observed to have a

comparable mean and standard deviation to that of MA-LIO point cloud. Although a high concentration of points are found to deviate less in the overall scan.

In the analysis of point clouds produced by KISS-ICP as observed in Figure 29e and 29f most of the points are within 0.25 m of distance error. On the basis of this result, we can say that KISS-ICP performed well in the case of point offset in temporal analysis when compared to the other algorithm. Although, it has a very poor performance when distance error analysis is performed on it with respect to the ground truth point cloud.

5.11 Correctness evaluation of the point clouds to ground truth scan

The measure of correctness between the point clouds is obtained by checking their closeness to the ground truth scan in cloud compare. The procedure to do this is as follows:

1. Export the ground truth point cloud of RTC 360 degree scanner to Cloud Compare.

2. Export the point cloud generated by the algorithms to Cloud Compare. The point clouds are transformed to the tachy coordinate frame.

3. Select the two point clouds and use the Cloud-to-Cloud distance option in cloud compare to obtain the error between the two point clouds under observation. Both the point clouds must be in the same coordinate system. Here, both the point clouds are in tachy coordinate system.

4. Choose the RTC360 scanner point cloud to be the reference point cloud and the point cloud generated by the algorithms to be the compared point cloud. Cloud compare will compute the cloud to cloud distance.

5. The color map beside the point clouds as seen in Figure 30 displays the distribution of the points in the point clouds produced by the algorithms as a measure of their distance errors. Blue indicates the least error and red indicates maximum error. The maximum error possible in the color scale is 2 meters.

6. The histogram plot describes the number of points in the point clouds produced by the algorithms present in a discrete set of bins which are in accordance with the distance errors.

5.11.1 Distance error analysis for point clouds by MA-LIO algorithm

In Figure 31, the first column represents the distance error histograms processed by MA-LIO algorithm. The reference point cloud and MA-LIO point cloud are almost correctly aligned. This can be inferred from the distance error histogram for MA-LIO. A large density of point have a distance error ranging from 0.10 m to 0.20 m. Based on the distance error analysis, a quantitative guess can be made that MA-LIO almost correctly fits with the reference point cloud.

5.11.2 Distance error analysis for point clouds by SC-LIO-SAM algorithm

As seen in Figure 31 column 2 represent the distance error histogram for the datasets processed through SC-LIO-SAM algorithm. In Figure 31k, 31h and 31h which are the distance error histograms for counter-clockwise, through the pump track and linear traversal dataset respectively, the majority of the points possess an distance error of less than 0.10 m which indicates SC-LIO-SAM has produced a very good map. For Figure 31n which represents the clockwise dataset, the histogram depicts that a majority of the points present in the point cloud are far from the ground truth scan with an error distance of 1 m to 1.8 m leading to a slightly miss-aligned map. Figure 31a depicts the distance error histogram of through the pump track dataset. The major concentration of points lie in between an distance error of 0.30 m - 0.60 m which would result is an slightly miss-aligned scan. Overall, we can infer that the resultant point clouds produced by SC-LIO-SAM algorithm to a good extent matches the ground truth scans correctly. The causes for miss-alignments can be attributed to vibrations while operating the robot and faulty loop closures resulting in a wrong scan matching.

5.11.3 Distance error analysis for point clouds by KISS-ICP algorithm

In Figure 31, column three represents the distance error histograms. As we can observe in the histograms, KISS-ICP algorithm produces a diverse distribution of point in accordance to the distance error. Majority of the points produce by the KISS-ICP algorithm posses a distance error of greater than 1 m and very few points are present in the distance error range of < 0.5 m. We can infer from this data that KISS-ICP produces resultant point clouds which are not well aligned with the ground truth point cloud.

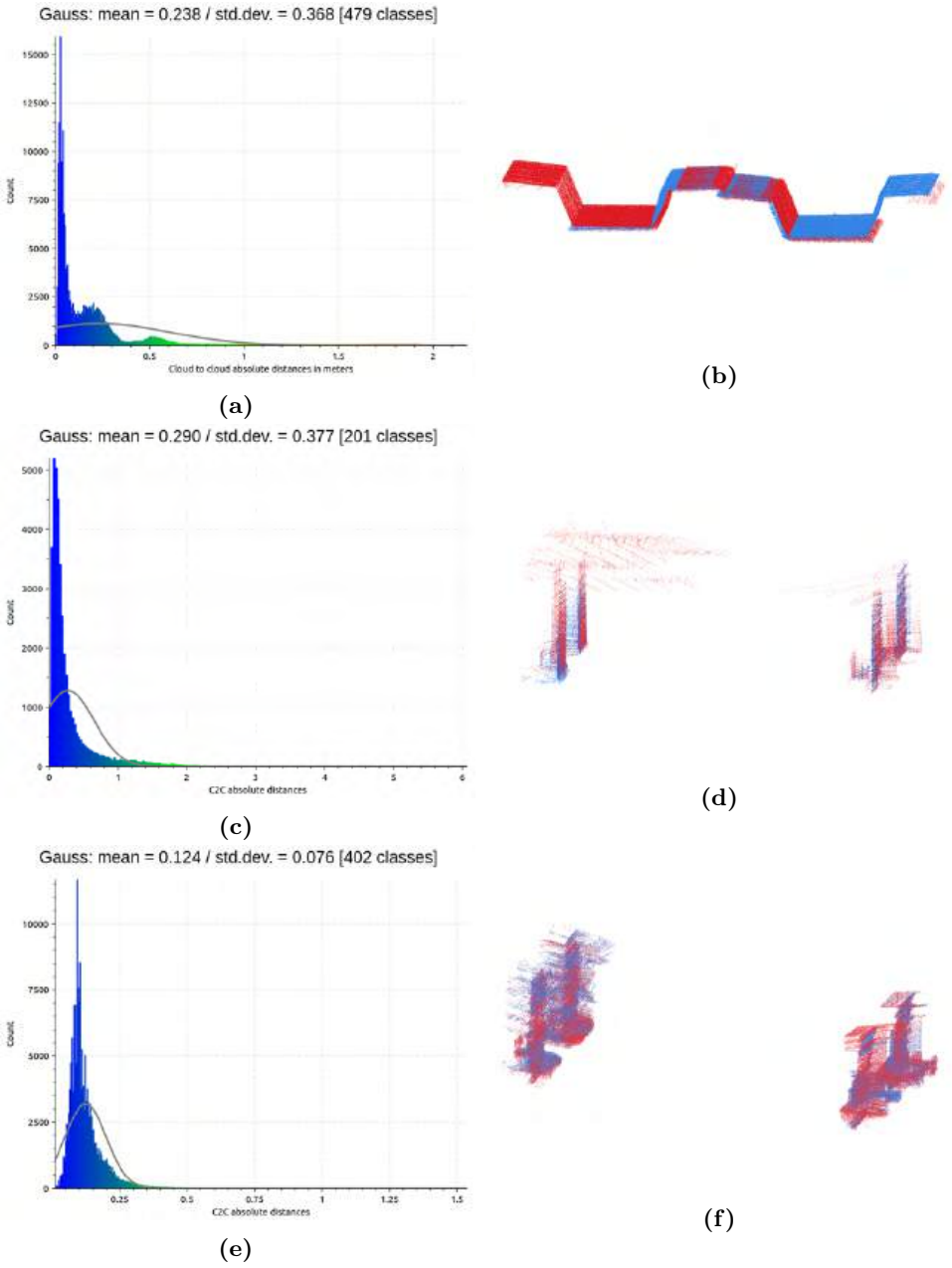


Figure 29: Distance error analysis of point clouds taken at different time intervals. The points in blue represent points taken at an initial time and the points in red represent points taken at the end of the scans (a) and (b) cloud to cloud distance analysis point cloud produced by MA-LIO, (c) and (d) cloud to cloud distance analysis point cloud produced by SC-LIO-SAM, (e) and (f) cloud to cloud distance analysis point cloud produced by KISS-ICP

(a) MA-LIO **(b)** SC-LIO-SAM **(c)** KISS-ICP

Figure 30: Comparisons between point clouds where the reference point cloud is obtained from RTC360 scanner and the compared point clouds are produced from MA-LIO, SC-LIO-SAM and KISS-ICP algorithms in (a), (b) and (c) respectively. The histogram plot describes the number of points present in a discrete set of bins. The color scale represents the distance error of the predicted scan from MA-LIO, SC-LIO-SAM and KISS-ICP from the ground truth point cloud. The error ranges from 0.0 m (blue color) to 2.0 m (red color).

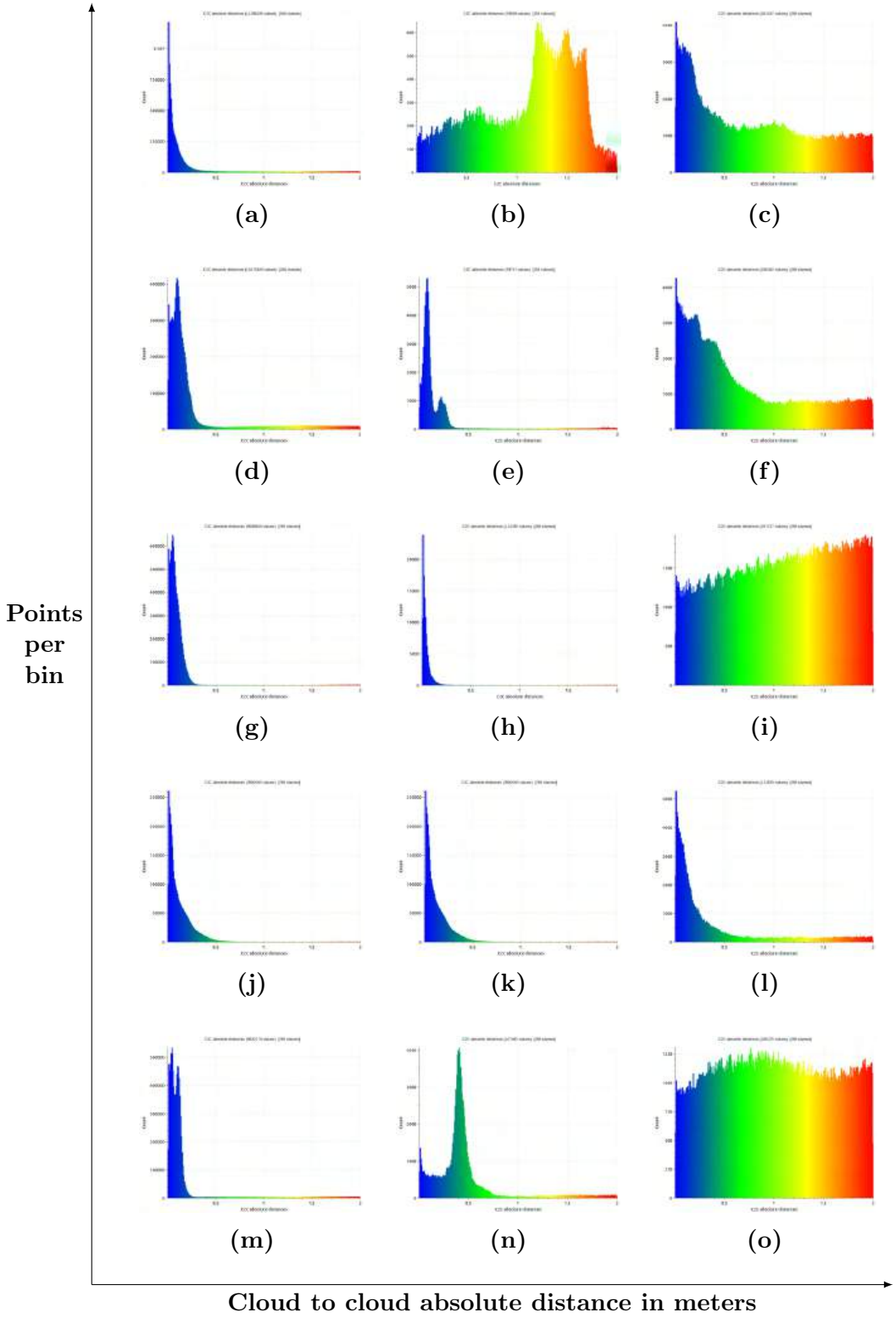


Figure 31: Comparison of cloud to cloud distance error histograms of MA-LIO (first column), SC-LIO-SAM (second. column), KISSICP (third column).

6 Conclusion

As a part of our project, we used a robot platform to acquire the datasets in the form of rosbags. The acquired being the LiDAR scans from a VLP-16 Velodyne and IMU data. The dataset was processed by three different algorithms namely MA-LIO, SC-LIO and KISS-ICP. A comparative study was performed on the performances of the three algorithms in the formation of a global map of the environment where the dataset was collected. Both a qualitative and quantitative analysis was carried out.

The qualitative analysis was performed on the point clouds and the trajectories obtained from different datasets of the test environment. As per a visual inspection, MA-LIO produced a very high density scan. This can be attributed to multiple factors, one of which being the use of dual LiDAR scanner for the purpose of mapping. Another reason is the presence of IMU data to account for the odometry occurring during the motion of the robot. A good quantitative evidence to support the good quality of scans produced by MA-LIO are the distance error histograms where majority of the points are within 0.1 m error. The distance error analysis of temporal point clouds showed that most of the points were present within 0.20 m.

SC-LIO-SAM performed the 2nd best in many scenarios. In the case of visual inspection for the correctness of trajectory as observed in Figure 24a, SC-LIO showed a slight deviation in the clockwise and through the pump track dataset. In the case of temporal analysis, which was performed on clockwise dataset, deviations were observed in accordance to the point cloud alignment to the structures. This can also be supported by the histogram plots from distance errors for SC-LIO-SAM (Column 2)in Figure 31. As far as the long range dataset for SC-LIO-SAM is concerned, SC-LIO-SAM produces a fairly accurate map. It also captures the presence of a slope in the map. It also produced comparable results as MA-LIO in the case of distance error analysis of temporal point clouds, where most of the points were present in the range of 0.60 m

KISS-ICP performed the least in the scenarios. KISS-ICP is primarily an odometry based pipeline meant to produce local maps. In uses ICP to perform scan matching and produce the overall trajectory information. Due to lack of external sensors such as IMU or GPS, drifts accumulate in robot odometry during the motion of the robot. Substantial drift accumulation leads to erroneous scan matching. This in-turn affects the overall trajectory produced by the algorithm. We can observe the result of this in the distance error histogram for KISS-ICP in Figure 31. It shows a large distribution of points present in the error distance of 1.5 - 2.0m. Another evidence to support the fact of erroneous scan matching can be observed in the long range dataset, which creates a map similar to a parabolic curve. It is not able to recognize the hill too. Although, KISS-ICP produce good results in case of error distance analysis of temporal point clouds where most of the points were present within 0.25 m.

In comparison to the three algorithms, MA-LIO outperforms SC-LIO-SAM and KISS-ICP in most of the scenarios. The superiority of MA-LIO in terms of performance can be attributed to its ability to use multiple LiDAR scanners, which increases the chances of extracting richer features. It also improves the field of view of the system [32]. SC-LIO-SAM performed comparatively well. The scans were less dense when compared to MA-LIO, owing to the fact that a single LiDAR was used. KISS-ICP performed the least when compared to the two algorithms. This can be attributed to the presence of external sensors to correct the odometry drift. Use of multiple LiDAR scanners could be employed to improve the accuracy of the generated map. The map construction process could also be positively impacted in terms of accuracy with the addition of cameras to the system.

7 Acknowledgments

First and foremost, I would like to thank Fraunhofer IPM for providing me the opportunity to pursue my master's thesis at the institute.

My sincere thanks to Mr. Dominik Merkle for guiding me through my thesis. I have gained a lot of insight in working on a large scale project under his guidance. His sheer determination and motivation was a great source of motivation to perform well in my master's thesis. Discussion session with him have helped me to understand complicated topics in a simple manner by the use of pictorial graphs.

My sincere thanks to Prof. Dr. Alexander Reiterer for giving me the opportunity to pursue my thesis at Fraunhofer IPM. I would also like to thank Prof. Christoph Müller for his support in the project.

A heartfelt thank you to my family, who have supported me throughout my masters in every way possible. They have been by me in every step of my masters.

A heartfelt thanks to all my friends who have been by my side during my masters studies. Through discussions with professors, tutors and friends, my masters have turned into a wonderful experience. A special thanks to my friends who helped me in my master's thesis in the form of discussions which enabled me to solve few issues in my master's thesis.

8 Future Work

The robot in our project was used to obtain LiDAR scans of an environment containing a bridge, and a comparative analysis on the correctness of the scan was performed. The data was collected in the form of rosbags and then processed offline on a computing machine. A future scope would be on making the robot use the acquired scans and navigate the environment autonomously to assess the safety of the environment, which in our case happens to be a bridge.

To improve the map accuracy, we could also try to include cameras or multiple LiDAR scanners, producing a much more accurate map. Different ways can we investigated to improve the mapping process in environments not having access to GNSS data. LiDARs scanners containing higher number of scan lines such as the VLP-32, VLP-64 can be used of environment mapping. Different types of LiDAR scanners such as Ouster and Livox can also be investigated into while performing environmental scans. One can find a suitable LiDAR scanner to assess different types of environment.

A possibility of multi-robot system including an underwater robot, terrestrial robot and an aerial robot can be investigated to produce a complete map of the bridge.

Bibliography

[1] "My bot shop." https://www.mybotshop.de/Clearpath_EN, 2023. [Online Available; Accessed: October 9, 2023].

[2] "Air supply." https://airsupply.com/product/velodyne-vlp-16-puck-lite-2/, 2023. [Online Available; Accessed: October 9, 2023].

[3] "Stereo labs zed box." https://www.stereolabs.com/zed-box/, 2023. [Online Available; Accessed: October 10, 2023].

[4] "Load indicator system ab." https://lisab.se/product/3dm-gx5-25/, 2023. [Online Available; Accessed: October 9, 2023].

[5] "Leica rtc360 3d laser scanner." https://leica-geosystems.com/de-de/products/laser-scanners/scanners/leica-rtc360, 2023. [Online Available; Accessed: October 9, 2023].

[6] "Global survey." https://globalsurvey.co.nz/shop/solutions/surveying-gis/robotic-total-stations/leica-nova-ms60/, 2023. [Online Available; Accessed: October 9, 2023].

[7] "Point cloud." https://en.wikipedia.org/wiki/Point_cloud, 2023. [Online Available; Accessed: October 5, 2023].

[8] X. Yang, "Slam and navigation of indoor robot based on ros and lidar," *Journal of Physics: Conference Series*, vol. 1748, p. 022038, 01 2021.

[9] Z. Hong, Y. Petillot, and S. Wang, "Radarslam: radar based large-scale slam in all weathers. in 2020 ieee," in *RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5164–5170.

[10] S. Parsons, "Probabilistic robotics by sebastian thrun, wolfram burgard and dieter fox, mit press, 647 pp., $55.00, isbn0 - 262 - 20162 - 3,'' TheKnowledgeEngineeringReview, vol.\ 21, no.\ 3, pp.\ 287 - -289, 2006.$

[11] "Sea bed mapping." https://ausseabed.gov.au/about/mapping, 2023. [Online Available; Accessed: October 8, 2023].

[12] G. A. Acosta-Amaya, J. M. Cadavid-Jimenez, J. A. Jimenez-Builes, *et al.*, "Three-dimensional location and mapping analysis in mobile robotics based on visual slam methods," *Journal of Robotics*, vol. 2023, 2023.

[13] R. Smith, "On the estimation and representation of spatial uncertainty," *Int. J. Robotics Res.*, vol. 5, no. 4, pp. 113–119, 1987.

[14] H. Durrant-Whyte, D. Rye, and E. Nebot, "Localization of autonomous guided vehicles," in *Robotics Research: The Seventh International Symposium*, pp. 613–625, Springer, 1996.

[15] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: part i," *IEEE robotics & automation magazine*, vol. 13, no. 2, pp. 99–110, 2006.

[16] T. Bailey and H. Durrant-Whyte, "Simultaneous localization and mapping (slam): Part ii," *IEEE robotics & automation magazine*, vol. 13, no. 3, pp. 108–117, 2006.

[17] X. Yan, C. Zhao, and J. Xiao, "A novel fastslam algorithm based on iterated unscented kalman filter," in *2011 IEEE International Conference on Robotics and Biomimetics*, pp. 1906–1911, IEEE, 2011.

[18] F. Lu and E. Milios, "Globally consistent range scan alignment for environment mapping," *Autonomous robots*, vol. 4, pp. 333–349, 1997.

[19] F. Lu and E. Milios, "Robot pose estimation in unknown environments by matching 2d range scans," *Journal of Intelligent and Robotic systems*, vol. 18, pp. 249–275, 1997.

[20] "Ma-lio github." https://github.com/minwoo0611/MA-LIO, 2023. [Online Available; Accessed: October 15, 2023].

[21] "Sc-lio-sam github." https://github.com/gisbi-kim/SC-LIO-SAM, 2023. [Online Available; Accessed: October 15, 2023].

[22] "Lio-sam github." https://github.com/TixiaoShan/LIO-SAM, 2023. [Online Available; Accessed: October 15, 2023].

[23] "Scan context github." `https://github.com/irapkaist/scancontext`, 2023. [Online Available; Accessed: October 15, 2023].

[24] "Kiss-icp github." `https://github.com/PRBonn/kiss-icp`, 2023. [Online Available; Accessed: October 15, 2023].

[25] F. Dellaert and G. Contributors, "borglab/gtsam," May 2022.

[26] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, "g 2 o: A general framework for graph optimization," in *2011 IEEE International Conference on Robotics and Automation*, pp. 3607–3613, IEEE, 2011.

[27] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5135–5142, 2020.

[28] G. Kim and A. Kim, "Scan context: Egocentric spatial descriptor for place recognition within 3D point cloud map," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, (Madrid), Oct. 2018.

[29] J. Zhang and S. Singh, "Loam: Lidar odometry and mapping in real-time.," in *Robotics: Science and systems*, vol. 2, pp. 1–9, Berkeley, CA, 2014.

[30] L. Li, X. Kong, X. Zhao, W. Li, F. Wen, H. Zhang, and Y. Liu, "Sa-loam: Semantic-aided lidar slam with loop closure," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 7627–7634, IEEE, 2021.

[31] K. Xiao, W. Yu, W. Liu, F. Qu, and Z. Ma, "High-precision slam based on the tight coupling of dual lidar inertial odometry for multi-scene applications," *Applied Sciences*, vol. 12, no. 3, p. 939, 2022.

[32] M. Jung, S. Jung, and A. Kim, "Asynchronous multiple lidar-inertial odometry using point-wise inter-lidar uncertainty propagation," *IEEE Robotics and Automation Letters*, 2023.

[33] W. Xu and F. Zhang, "Fast-lio: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317–3324, 2021.

[34] I. Vizzo, T. Guadagnino, B. Mersch, L. Wiesmann, J. Behley, and C. Stachniss, "KISS-ICP: In Defense of Point-to-Point ICP – Simple, Accurate, and Robust Registration If Done the Right Way," *IEEE Robotics and Automation Letters (RA-L)*, vol. 8, no. 2, pp. 1029–1036, 2023.

[35] N. Kirchner, A. Alempijevic, S. Caraian, R. Fitch, D. Hordern, G. Hu, G. Paul, D. Richards, S. Singh, and S. Webb, "Robotassist - a platform for human robot interaction research," 01 2010.

[36] N. Ohi, K. Lassak, R. Watson, J. Strader, Y. Du, C. Yang, G. Hedrick, J. Nguyen, S. Harper, D. Reynolds, *et al.*, "Design of an autonomous precision pollination robot," in *2018 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pp. 7711–7718, IEEE, 2018.

[37] M. Labbé and F. Michaud, "Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," *Journal of field robotics*, vol. 36, no. 2, pp. 416–446, 2019.

[38] H. Lim, D. Kim, B. Kim, and H. Myung, "Adalio: Robust adaptive lidar-inertial odometry in degenerate indoor environments," *arXiv preprint arXiv:2304.12577*, 2023.

[39] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for visual (-inertial) odometry," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 7244–7251, IEEE, 2018.

[40] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A modern library for 3D data processing," *arXiv:1801.09847*, 2018.

[41] X. Gao, T. Xiao, C. Bai, D. Zhang, and F. Zhang, "Anderson acceleration for on-manifold iterated error state kalman filters," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 12243–12250, 2022.

[42] A. Chaudhuri, "B-splines," *arXiv preprint arXiv:2108.06617*, 2021.

[43] G. Kim and A. Kim, "Scan context: Egocentric spatial descriptor for place recognition within 3d point cloud map," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4802–4809, IEEE, 2018.

[44] "Husky unmanned ground vehicle." `https://clearpathrobotics.com/husky-unmanned-ground-vehicle-robot/`, 2023. [Online Available; Accessed: October 10, 2023].

[45] "Vlp 16 user manual and programming guide 63 9243 rev a." https://usermanual.wiki/Pdf/VLP1620User20Manual20and20Programming20Guide2063924320Rev20A.1338722902/help, 2023. [Online Available; Accessed: October 10, 2023].

[46] "Leica geosystems." https://leica-geosystems.com/de-de/products/total-stations/multistation/leica-nova-ms60, 2023. [Online Available; Accessed: October 15, 2023].

[47] "Cloud compare: 3d point cloud and mesh processing software open source project." https://www.cloudcompare.org/, 2023. [Online Available; Accessed: October 5, 2023].

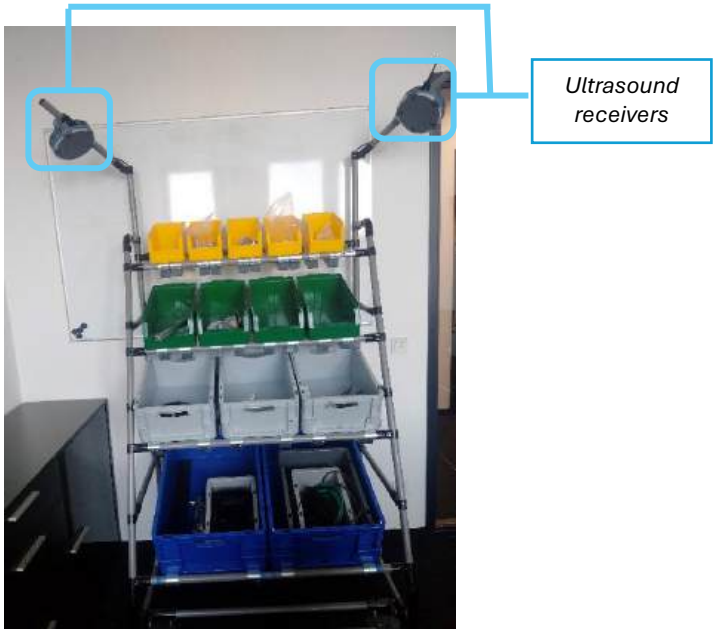
ALISA Project Internal Report

1. Setup

Components:

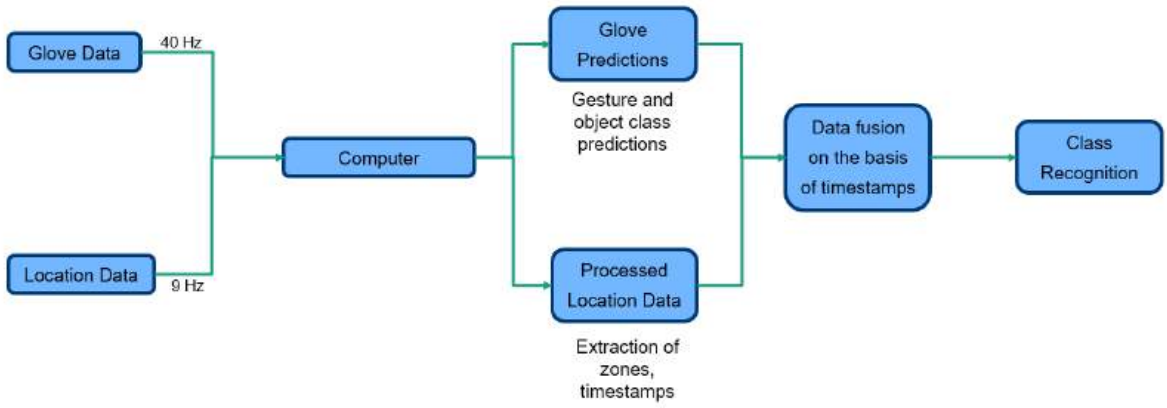
- **Glove:** A smart glove woven with four force resistive sensors.
- **Interface wristband:** An interfacing wristband connected to the gloves to transmit sensor data from the gloves to a central computing system. The interfacing wristband transfers the variations in the sensor data due to change in force applied while an object is picked up and the location data of the glove.
- **Shelf:** The shelf contains different object used for pick and place operations. Each box of the shelf is numbered with a box id for the respective object. The shelf contains ultrasound sensors that transmit location data of the glove to a computing device.
- **Pick and place objects:** Pick and place objects are the objects the deep learning model must identify. There are five objects used for this task which are: Foot stand, cylindrical support for the bike, tyres, mudguard, fixatures of the bike.
- **Computing unit:** The computing unit processes the sensor data and location data information received from the glove. It contains the deep learning model to output the predictions of the gestures performed and the object picked.

Wrist band



Shelf at Telocate

Overall system architecture



Description of the overall system architecture from data acquisition till class recognition

- Glove data is received at the computing unit at the rate of 40 Hz.
- The location data is received in parallel at the rate of 9 Hz.
- Both glove and location data are buffered in a Queue data structure when they are received for future data-processing.
- Data processing is performed on the raw glove data present in the Queue and it is accumulated in an array to form a sequence length of 35 datapoints (the window size used for the data frames during the training process).
- The data is given as an input to the deep learning model and the model provides the output in the form of gesture and object classification.
- In parallel, the location data is processed and the time stamps and zones are extracted.
- The zones and predictions are combined to identify the action of pick-up.
- The zones and class predictions are combined to obtain the result of correct object picked from the respective shelves

2. Items

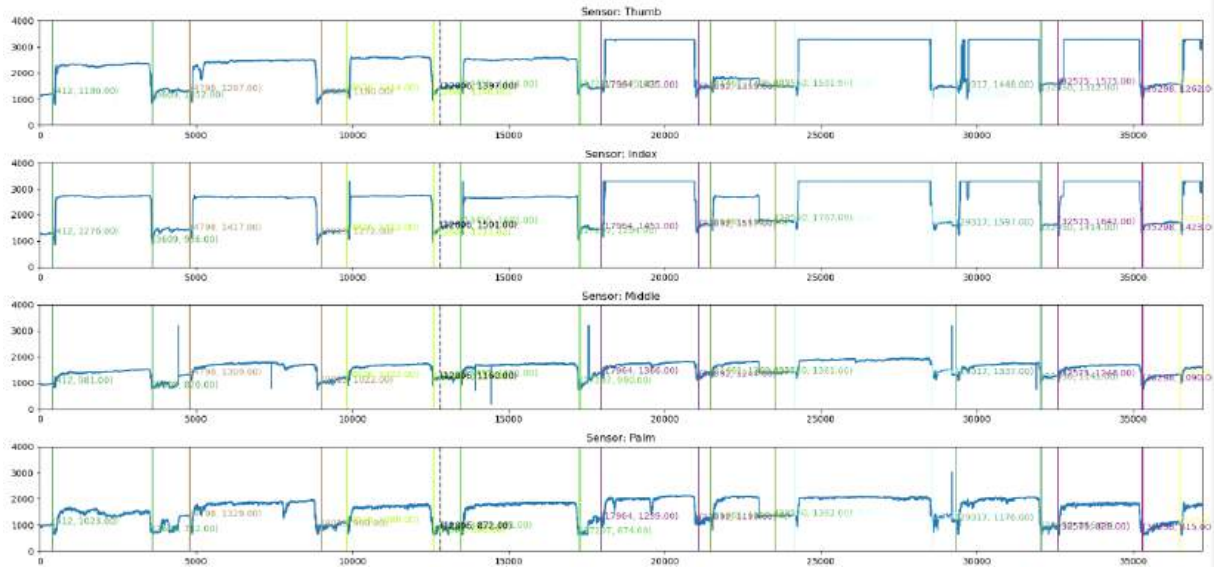
Cylinde

Mud Guard

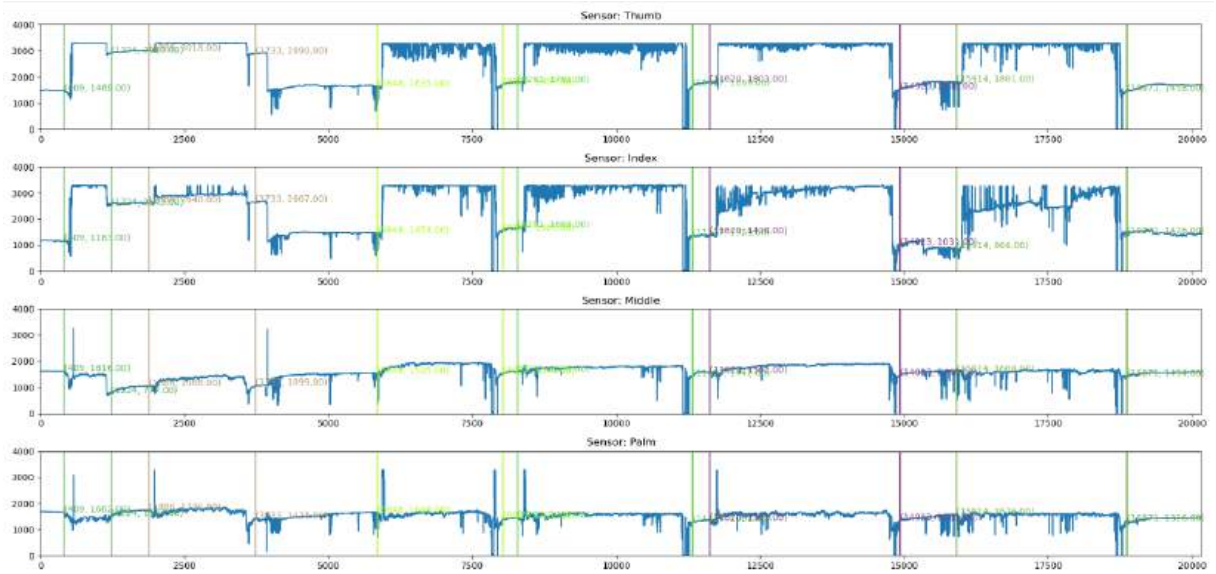
Foot stan

Fixature

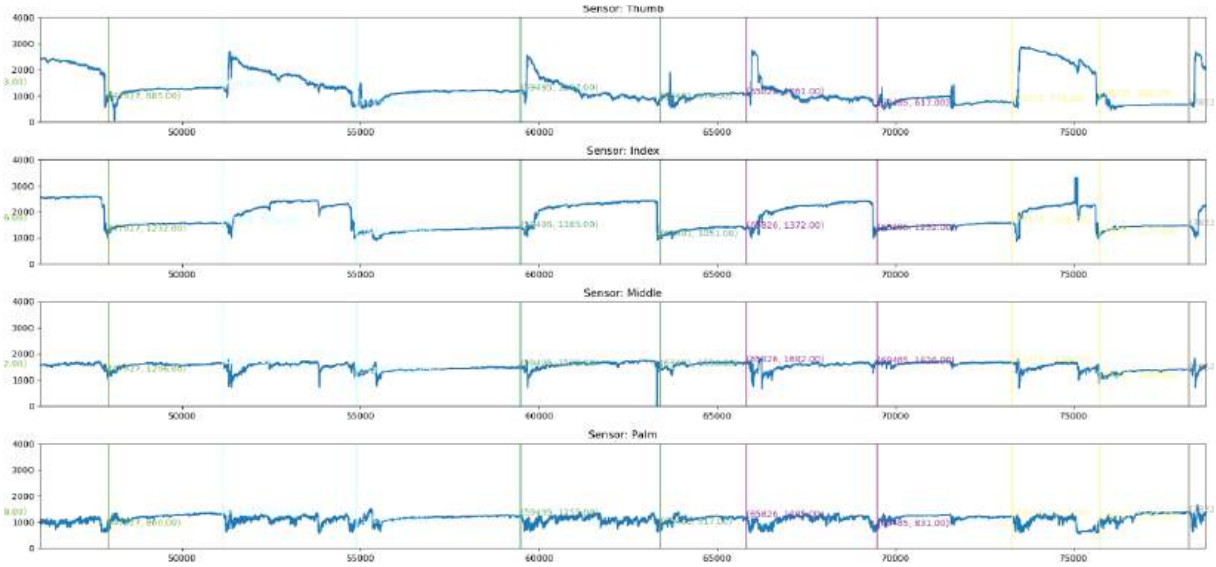
Tyre



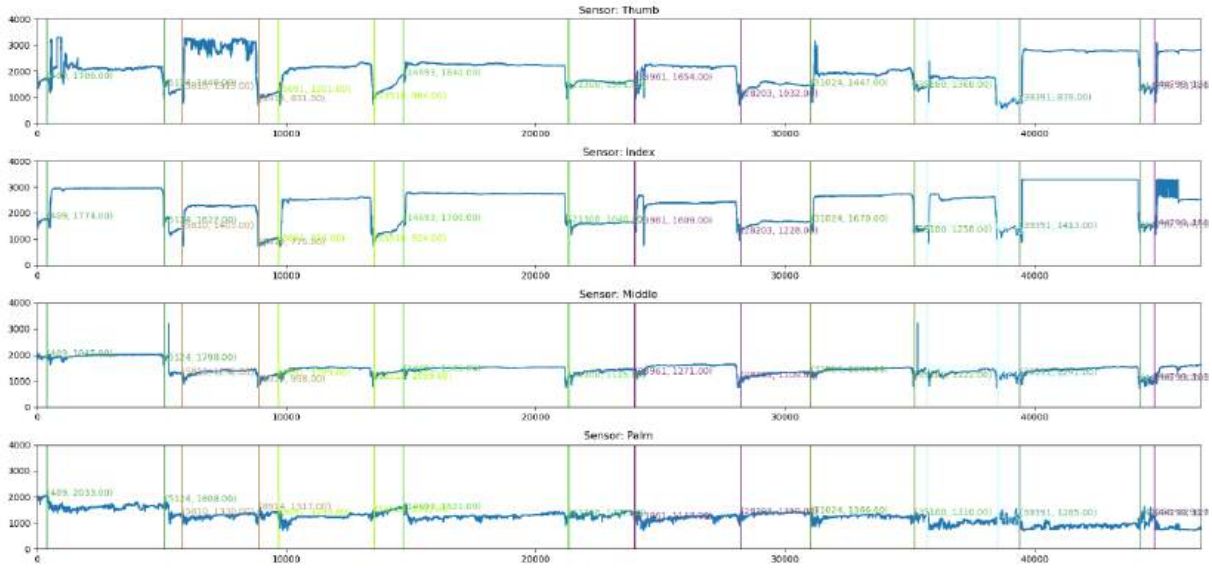
Sample raw data waveform for cylinder data collected using glove



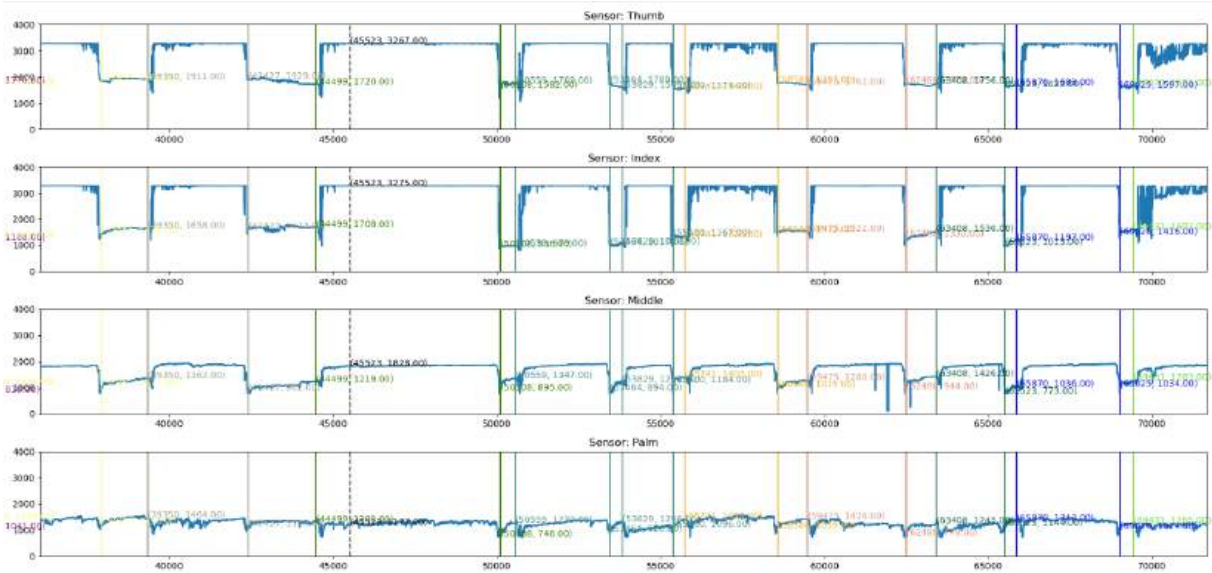
Sample raw data waveform for mud guard data collected using



Sample raw data waveform for Foot Stand data collected using glove 2



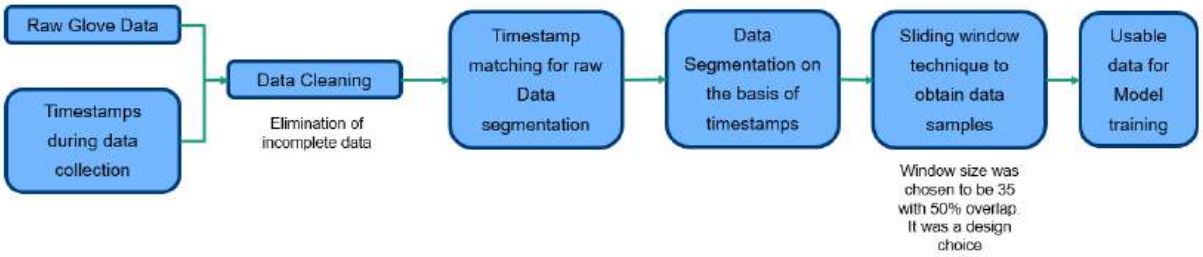
Sample raw data waveform for mechanical fixatures data collected using



Sample raw data waveform for tyres fixatures data collected using

- **Cylindrical support for the bike (Cylinder):** The object forms the cylindrical support of the bike. It is recorded during data collection as a cylindrical shape.
- **Foot Stand:** The object is the foot stand of a mechanical scooter. It resembles a flat piece of tile. The foot-stand is an abrasive surface and can cause rapid deterioration of the glove material if the glove is not coated.
- **Tyres:** They are the tyres of the scooter
- **Fixatures:** Fixatures correspond to the connectors of the scooter. The connectors can be screws, brackets etc.
- **Mud Guard:** The mud guard corresponds to the mudguard of the scooter.

2.1. Data preparation pipeline



Description of a data preparation to process raw data into a suitable format to be used for training the deep learning model

- The raw data collected to train the deep learning model must be pre-processed.
- Incomplete data-points from the raw data are dropped.
- Timestamps while an object is picked-up and placed back are recorded. The duration between the timestamps corresponds to the time the object was grasped. This provides information for object classification and gesture recognition.
- The "Idle" and "picked-up" data are separated on the basis of timestamps.
- A sliding window of size 35 (design choice) used to create data frames having sequence length of 35.
- Each data frame is of the dimension [4 x 35], here 4 is the number of sensors used in the glove and 35 is the number of data points in each frame.
- A set of experiments was performed to select the best window size for the data frame and 25, 35 and 40 turned out to be suitable candidates.
- The window size can be changed to any one of the three values.
- After the data is processed the data is available saved in the following format to be trained, [number of samples, number of sensor channels, sequence length of the data points]

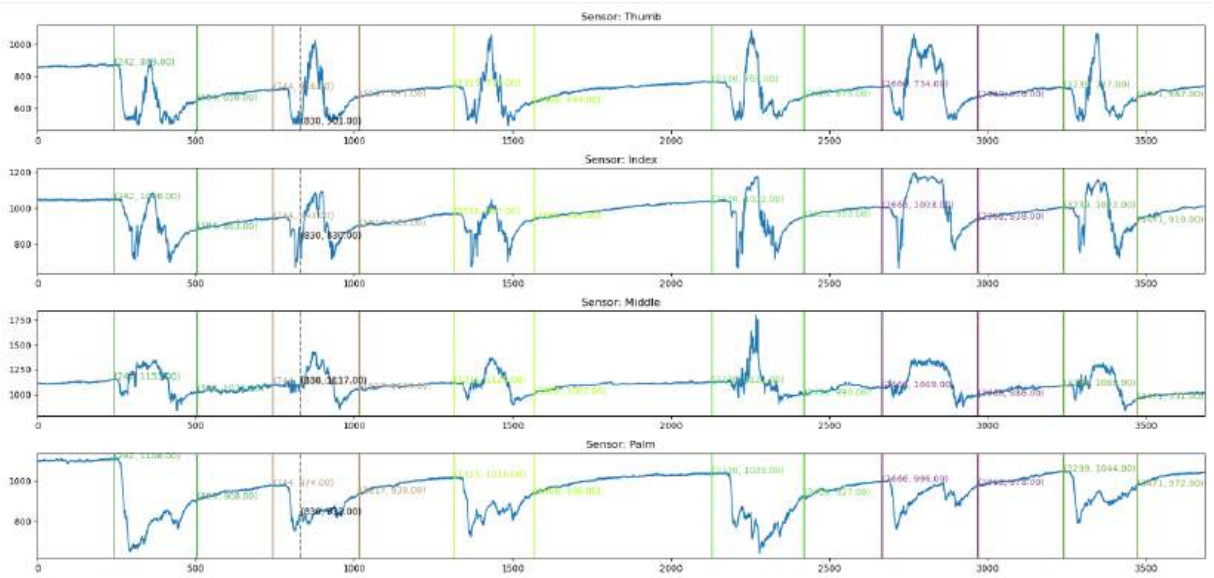
3. Gloves

The gloves contained four sensors. A sensor was present on the tip of the thumb, tip of the index finger, on the middle joint of the middle finger, on the left lower end of the palm. The sensors are force resistive sensors that produce variation in the voltage levels when pressure is applied to them. The glove contains conductive threads which are connected to metallic connectors on the wrist of the hand. The transfer the information from the sensor material to the wristband for information processing.

Unpassivated glove; Glove

3.1 Glove 1

Object Classes	Train Samples	Validation Samples
Idle	17584	4396
Bottle	2431	607
Network switch	1258	315
Pen	3967	992
Small box	1492	373



Sample raw data waveform for glove 1 for pen class

- The first glove used to collect data had passivation. The passivation for the glove was present on the palm.
- The data collected from the glove was used to develop the deep learning models in the initial stage.
- The data from the first glove was not used to prepare the dataset for object classification as the objects that needed to be classified were different.

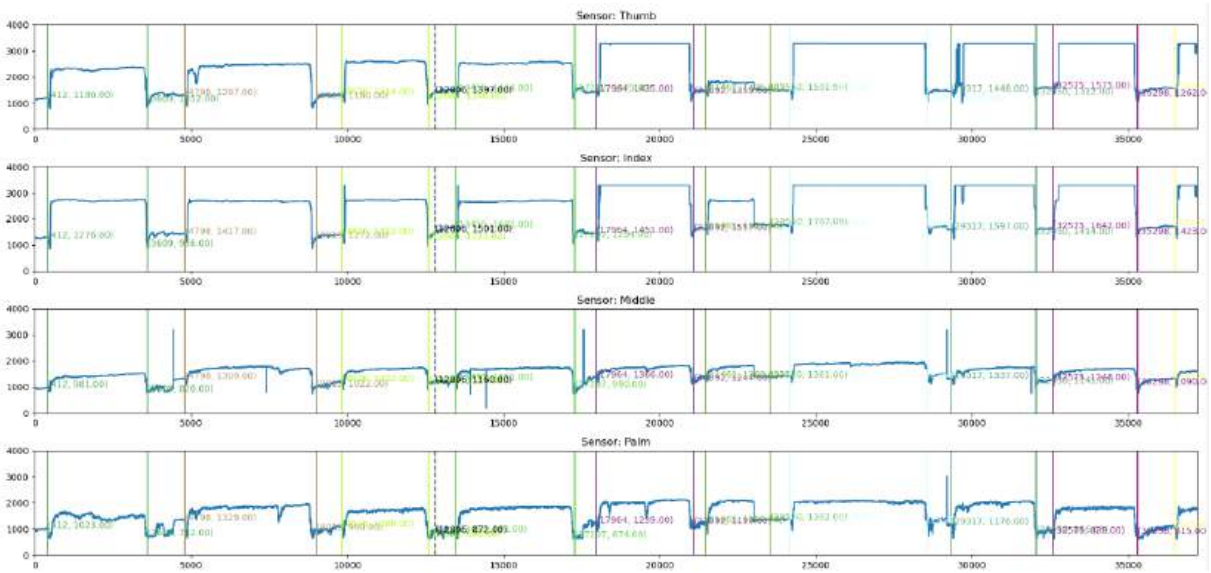
Description of the glove (number and type of sensors, i.e. strain gauges, location sensor, etc., …, insulation (yes or no), other relevant information)

Description of the data collected with the glove, e.g. how many samples of which kind, and the sensors involved. Give the samples a name, e.g. glove1.idle or glove1.tyre, for later use, e.g. in the 'models' section

Provide exemplary plots of the some of the collected data and clearly indicate to which respective dataset above the plotted data pertains

3.2 Glove 2

Object Classes	Train Samples	Validation Samples
glove2.idle	61742	15436
glove2.cylinder	39587	9897
glove2.mud_guard	27806	6952
glove2.foot_stand	35738	8934
glove2.fixatures	34080	8520
glove2.tyres	38523	9631

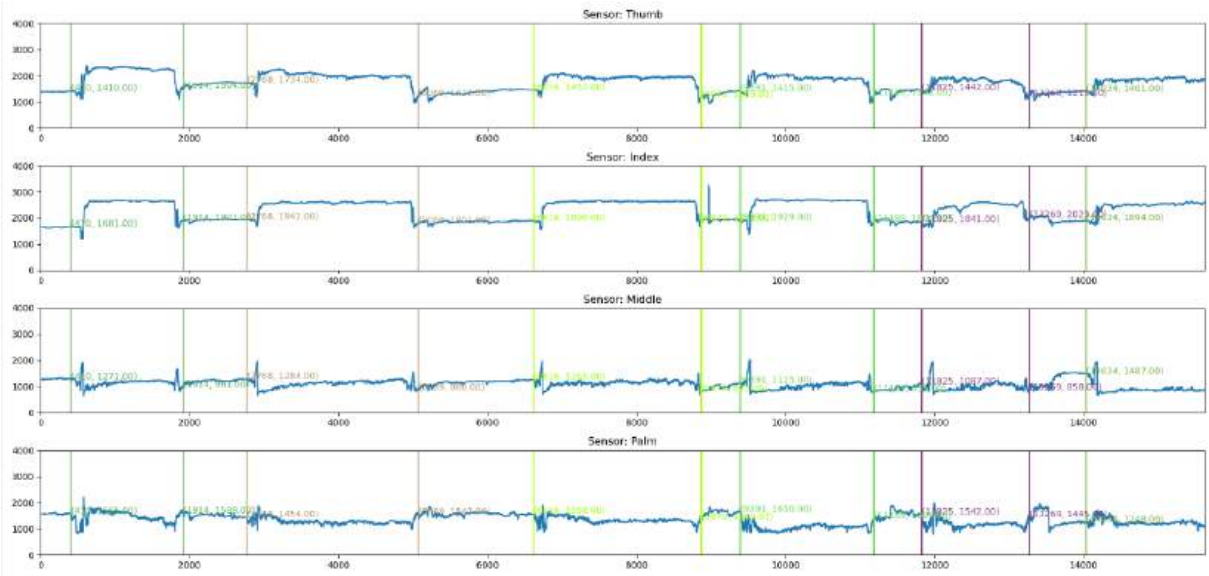


Sample raw data waveform for glove 2 for cylinder

- The second glove had no passivation present.
- It had the same number of sensors as the first glove and they were present in the same position.
- The second glove was worn-out after use, which deteriorated the usable sensor information from the glove.
- Hence a third glove of similar material was used to further collect data.

3.3 Glove 3

Object Classes	Train Samples	Validation Samples
glove3.idle	13138	3285
glove3.cylinder	13522	3381
glove3.mud_guard	24312	6080
glove3.foot_stand	6758	1689
glove3.fixatures	6082	1520
glove3.tyres	11048	2762

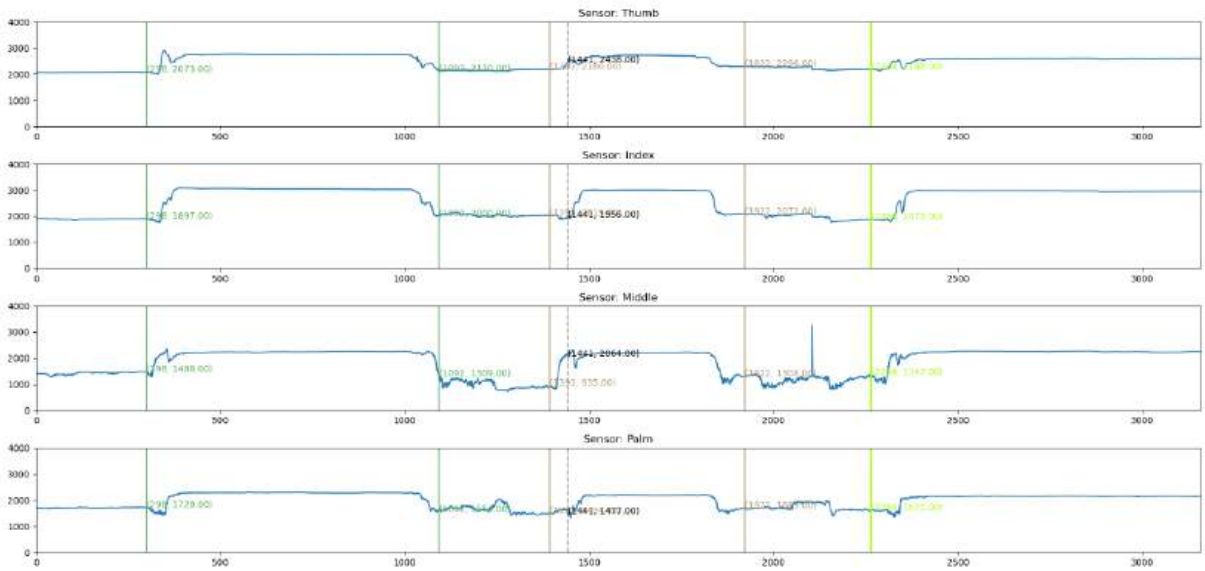


Sample raw data waveform for glove 3 for Foot Stand

- The third glove had no insulation.
- It had the same number of sensors as that of the first glove and they were present in the same position.
- The third glove is currently used to perform picking operations.

3.4 Glove 4

Object Classes	Train Samples
glove1.idle	3063
glove1.cylinder	1460
glove1.mud_guard	1360
glove1.foot_stand	1399
glove1.fixatures	1129
glove1.tyres	1180



Sample raw data waveform for glove 4 for Foot Stand

- Glove 4 was passivated on the underside of the palm.
- The passivation reduced the short circuit caused between the sensors when they are in contact with each other or when the gloves are used to grasp a metal object.
- The issue regarding short circuit occurred with un-passivated gloves.
- The gloves had the four sensors present in the same position as that of the Glove 1.

4. Models

Several deep learning models were experimented on to produce the best results for gesture recognition and object classification. The models used for the experiments are:

- Transformer architecture
- Transformer with GeLU activation
- Spatio-Temporal Transformer architecture
- CNN-LSTM Architecture
- Inception Architecture
- Inception with Residual Architecture

Among the deep learning models some of the displayed similar results, like Inception architecture and Inception with residual network architecture.

4.1 CNN-LSTM Model

Hyperparameters	Value
Learning Rate	0.0001
Batch Size	32
Optimizer	AdamW
Loss Function	Cross Entropy Loss
Balancing	True
Epochs	30
Number of hidden layers (LSTM)	6
Number of hidden layers (CNN)	2

- CNN-LSTM model was one of the best performing models in terms of gesture classification and object classification.
- The training curves were stable and did not have presence of spikes in the training curves.
- The first block of the CNN_LSTM model was a CNN block. The output of the CNN block was fed into the LSTM block from which the final output was taken.
- The CNN-LSTM model was used with data from glove 2, glove 3, glove 4.
- A CNN-BiLSTM, a variant of LSTM architecture was trained using the dataset. The model offered similar results as that of CNN-LSTM model but with increased inference time.
- The CNN-BiLSTM model was used with glove 2 and glove 3.

Class	Accuracy (in %)	F1-Score (in %)
Idle	84.50	80.74
Cylinder	67.86	42.96
Guard	71.17	31.55
Slab	88.82	35.14
Small	75.98	20.57
Tyre	93.12	65.36

Performance of CNN-LSTM model for multiclass object detection evaluated on data from glove 2 and glove 3

Class	Accuracy (in %)	F1-Score (in %)
Idle	75.64	84.19
Picked-up	97.44	91.57

Performance of CNN-LSTM model for gesture detection evaluated on data from glove 2 and glove 3

4.2 *Inception Model*

Hyperparameters	Value
Learning Rate	0.0001
Batch Size	32
Optimizer	AdamW
Loss Function	Cross Entropy Loss
Balancing	True
Epochs	30
Number of inception modules	2
Inception kernel size used	(1 x 1), (3 x 3), (5 x 5)

- The base inception model performed marginally better when compared to the CNN-LSTM model
- Inception network with residual connections was explored to counteract vanishing gradient problems in the deep learning network. It had a similar performance that of the base inception model with a minute decrease in the performance.
- The Inception model was used with glove 3.
- Inception with residual network architecture was used with glove 3.

Class	Accuracy (in %)	F1-Score (in %)
Idle	82.42	76.18
Cylinder	27.20	34.44
Guard	38.29	52.08
Slab	62.74	58.70
Small	86.09	47.56
Tyre	54.59	56.83

Performance of Base Inception model for multiclass object detection evaluated on data from glove 2 and glove 3

Class	Accuracy (in %)	F1-Score (in %)
Idle	94.54	84.78
Picked-Up	87.30	92.02

Performance of CNN-LSTM model for gesture detection evaluated on data from glove 2 and glove 3

4.3 Transformer Base Model

Hyperparameters	Value
Learning Rate	0.0001
Batch Size	32
Optimizer	AdamW
Loss Function	Cross Entropy Loss
Balancing	True
Epochs	30
Sinusoidal Scale Factor	1000
Number of Encode Blocks	6
Activation function	ReLU

- The base transformer architecture performs the second best among the models.
- During the training process the waveforms were observed to contain peaks.
- The evaluation of the model on the test data did not provide comparably good results as CNN-LSTM and Inception model.
- The transformer base model was tested on data from glove 1, 2 and 3.
- A transformer model with GeLU activation was used which provided results not as good as the base transformer model. The transformer model with GeLU was tested on data from glove 2 and 3.

Class	Accuracy (in %)	F1-Score (in %)
Idle	94.11	80.74
Cylinder	42.33	42.96
Guard	20.82	31.55
Slab	24.18	35.14
Small	47.86	20.57
Tyre	69.43	65.36

Performance of Transformer model for multiclass object detection evaluated on data from glove 2 and glove 3

Class	Accuracy (in %)	F1-Score (in %)
Idle	94.92	84.19
Picked-up	86.37	91.57

Performance of Transformer model for gesture detection evaluated on data from glove 2 and glove 3

4.4. Spatio-Temporal Transformer

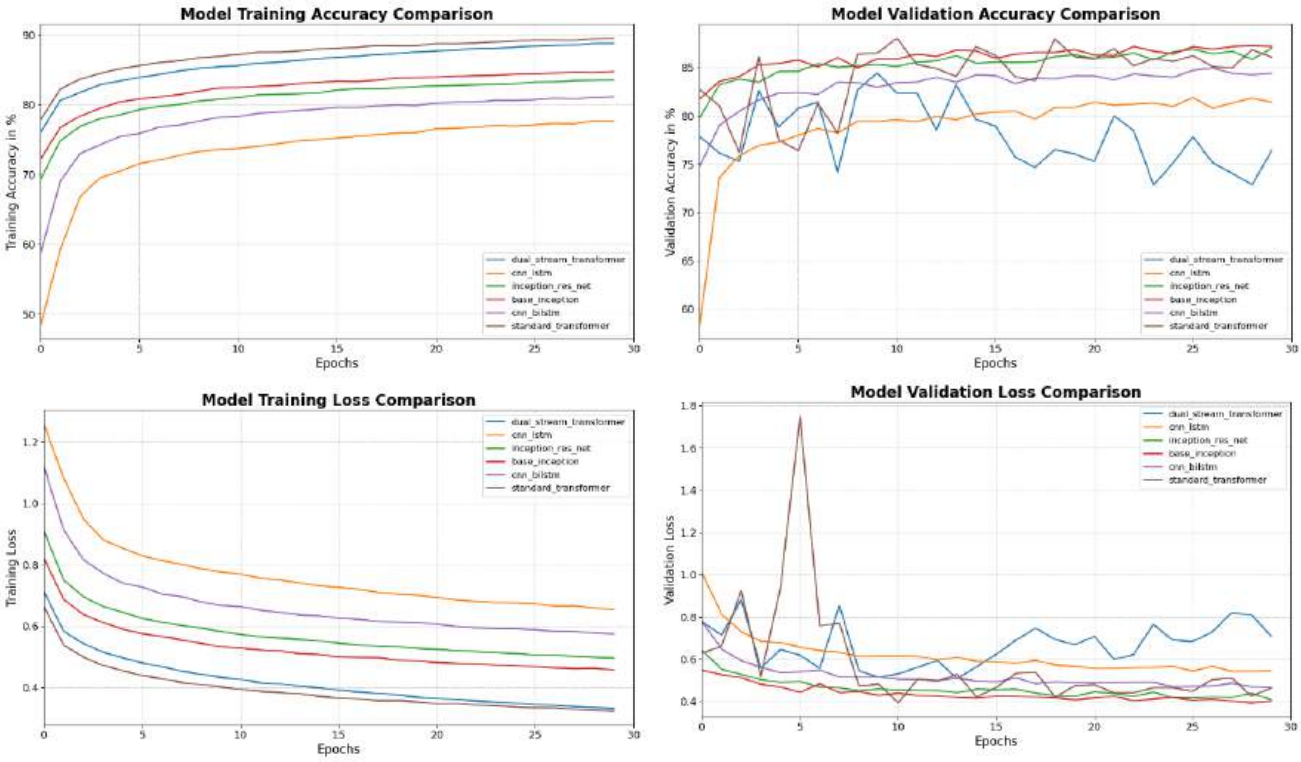
- The spatio-temporal transformer uses two implementations of the transformer architecture.
- The spatial-block obtains features across channels.
- The temporal block extracts feature across the sequence.
- Spatio-temporal transformer was used to for classification in multi-object classification. It was not used in gesture classification.
- During training, the spatio-temporal transformer did not have as high accuracy as CNN-LSTM models or Inception models.

- A reason for low accuracy and F1- score can be attributed to the fact the architecture is complex for handling information from four channels or has a smaller dataset as transformer require a large dataset.
- As the performance of Spatio-Temporal transformer was low performance metrics were not evaluated.
- The Spatio - Temporal model was used with glove 3

Performance Metrics:

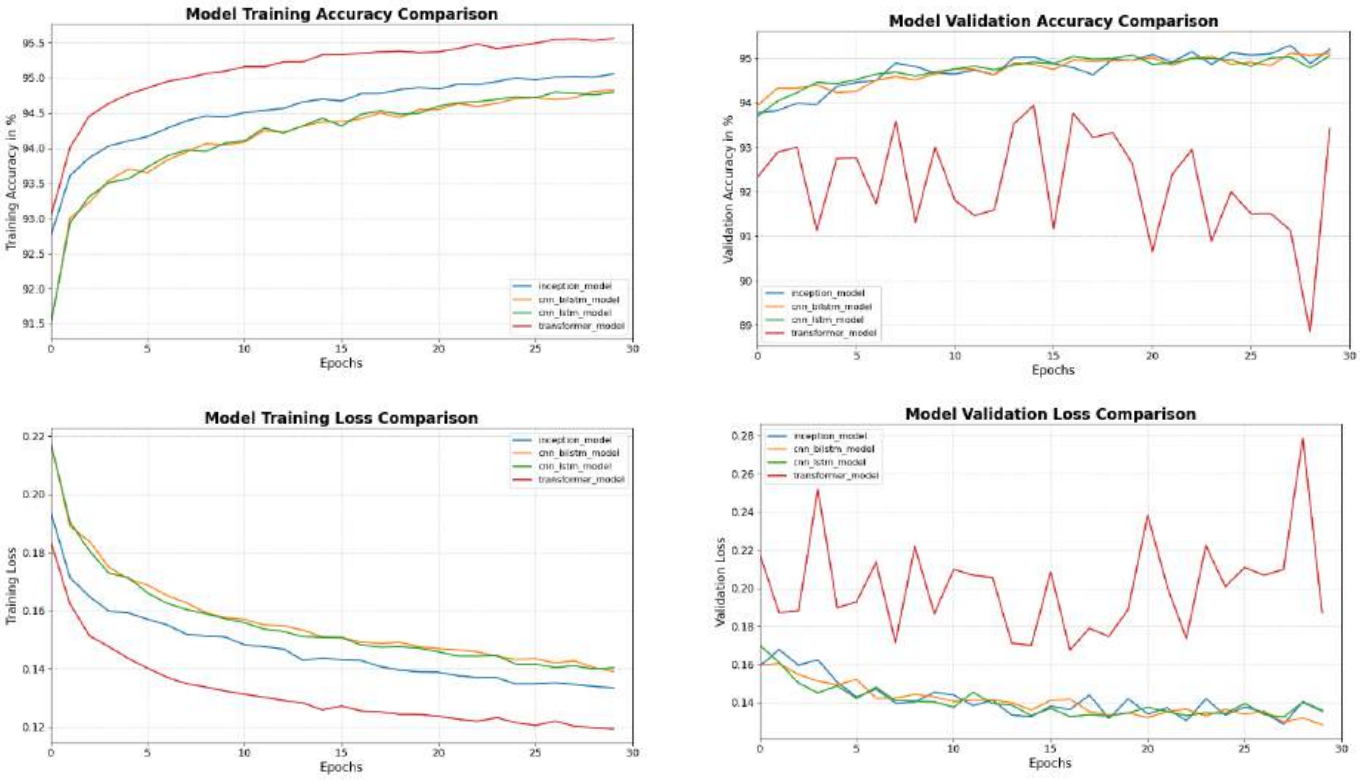
- The performance of the deep learning model is evaluated by using performance metrics such as accuracy, precision, recall, and the F1 score.
- Accuracy provides information about the percentage of correctly identified classes. Recall represents the proportion of samples that were correctly classified to the total correctly identifiable classes. Precision corresponds to the proportion of the model's classifications that are positive. The F1-score is the harmonic mean of precision and recall.
- Accuracy is a good metric when used in scenarios where the datasets are balanced. In the case of unbalanced data sets, accuracy does not provide appropriate results. Here, the F1-score is a better measure of performance for unbalanced data sets.
- In our experiments, importance was focused towards per class accuracy (Recall) and F1- Score as they provided an overall picture of the model performance.

Waveform comparison of Object Classification



Learning curves of different deep learning models for object classification evaluated on data from glove 2 and glove 3

Waveforms of Gesture Classification

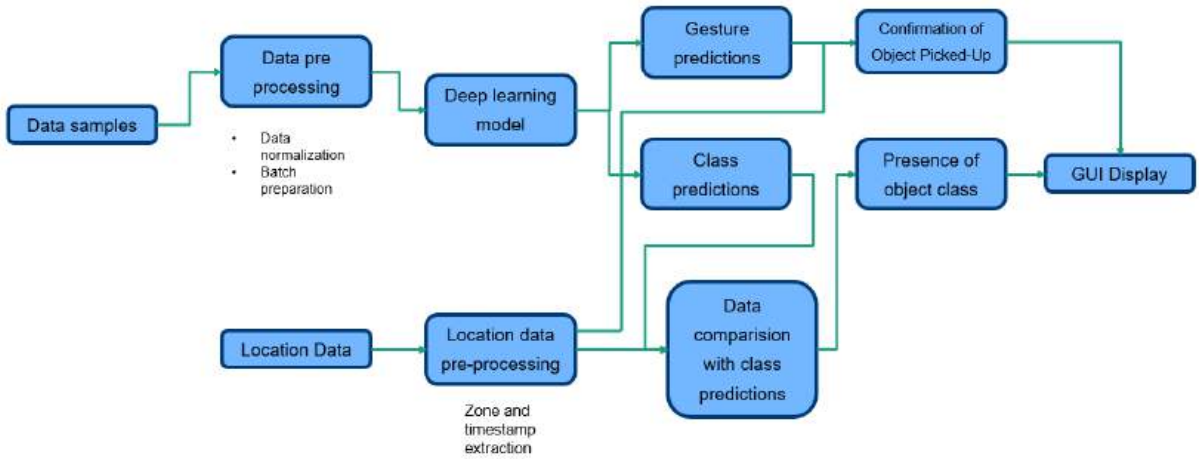


Learning curves of different deep learning models for gesture detection evaluated on data from glove 2 and glove 3

Comparative test for multiple active sensors

- Experiments were performed to investigate the performance of the glove by activating different combinations of sensors in the glove.
- In these experiments, single, double, three and four active sensors were used.
- The experiments were performed using the transformer architecture.
- As per the results we observed that the per class accuracy and F1-score increased as the number of active sensors increased.
- The experiments were performed for gesture classification. The experiments were not performed for object classification.
- For one active sensor the best results were given by “Index” active sensor and the worst result was given by “Palm” active sensor.
- For two active sensors the best results were given by “Index and Middle” active sensors and the worst result was given by “Thumb and Palm” active sensors
- For three active sensors the best results were given by “Index, Middle and Palm” active sensors and the worst result was given by “Index, Middle and Thumb” active sensors

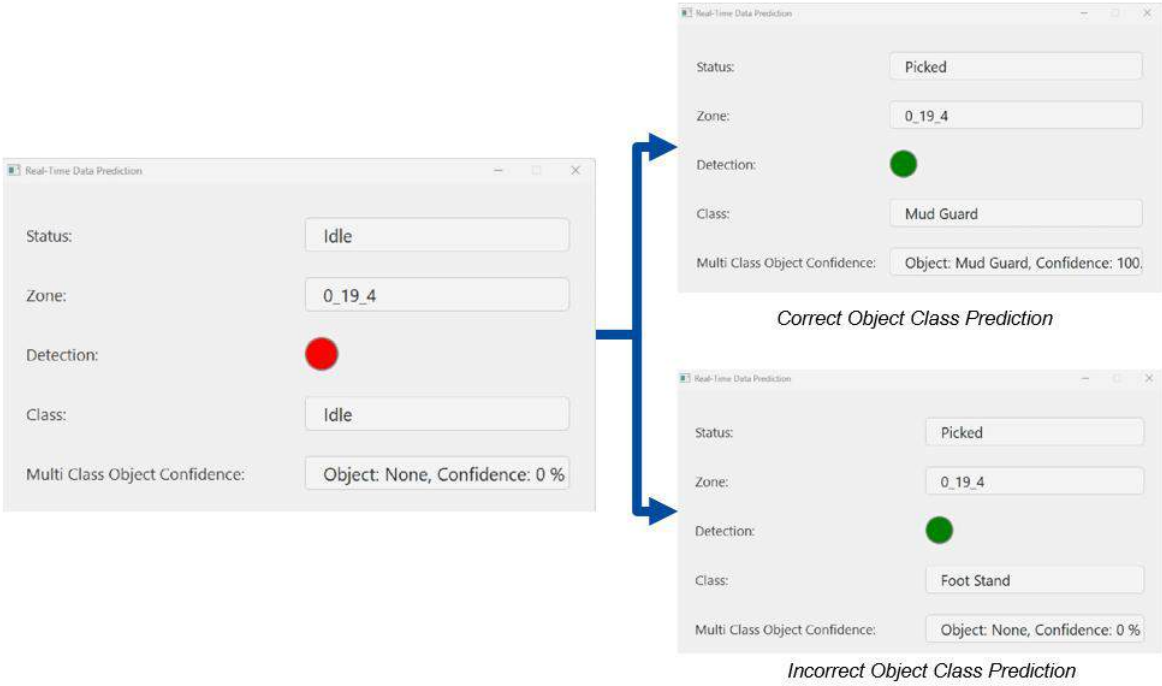
Overall Software architecture



Overall software flow of the prediction and sensor fusion algorithm which are run in parallel during the execution of the code

- Data samples of the format [number of samples, number of sensor channels, sequence length of the data points] are processed before being provided as input to the deep learning model.
- Data is normalized to a range of [0, 1]. Data normalization reduces the effect of gradient bias caused during model training.
- Data are prepared in batches of 32 samples each to be fed to the deep learning model.
- The deep learning model provides gesture classification and object class predictions.
- In parallel, location data is received and stored in a Queue and during real time predictions.
- The location data is processed, where the timestamps, zones and [x,y,z] position co-ordinates of the glove are extracted.
- The deep learning algorithm provides gesture and object classification.
- Gesture classification corresponds to the status of the glove during pick and idle process. If an object is picked by the glove the gesture classification reads “picked-up” else it reads “idle”.
- The gesture classification is combined with zones extracted from location data to identify the task of “pick-up” from the respective shelves.
- The object classification results are compared with the box ID, to identify if the correct object classification is performed. If the object is present in the box with the corresponding zone ID and matches with the object class predicted the prediction of the object class is displayed with a confidence score of 100%. If the prediction does not match the box ID. The class of the object is not given as an output and the confidence score is 0.

- A functionality to check the loss of location data signal was incorporated. When the zones are buffered if the time duration between the last buffered zone and the current real time is more than 100 ms the location data signal is assumed to be lost and a note of the last zone value before the connection is disrupted is made in a text file. The zone data at this time retains the last visited zone value. This allows us to view cases when the connection was disrupted.
- The result of the pick-up is displayed on a GUI.



- In the GUI above three scenarios are depicted. Firstly, we have the case where the glove has not picked up an object. Hence the status is displayed as “Idle”

- In the top-right corner, the object “Mud Guard” was detected by the object classifier and the same object was present in the box attributed to zone 0_19_4. Hence, the confidence score was 100 %

- In the bottom-right corner, the object is classified as “Foot Stand”, but as foot stand is not present in zone 0_19_4, instead it was “Mud Guard” which is misclassified as “Foot Stand”.

5. Sensor data fusion

- The data received from the glove is processed and given as an input to a deep learning model. The model predicts the class of the object and the gesture (idle or picked-up).
- A time stamp is recorded during the prediction. Let it be called “prediction_timestamp”. The location data of the glove is buffered in a Queue. A search is performed while comparing the “prediction_timestamp” with the timestamps of the

location data. The location data timestamp closest to the prediction timestamp is taken with the correspond location information.

- Gesture classification utilizes the zone value to identify the pick-up event which is recorded in a CSV file. The event "pick-up" is triggered when the gesture recognition result is "picked_up" and the glove is present in the zone. This provides us conformation regarding pick detection when an object is picked from the shelf. The gesture classification is obtained by taking the Boolean of the object class prediction. If any class other than "Idle" is reported as the output of the model the gesture recognition status corresponds to "picked_up" else the status corresponds to "Idle"
- During object classification, the deep learning model outputs the detections of each object at the rate of 0.01 seconds. Ten object classifications are buffered and the most commonly occurring object prediction is taken as the result for the object classification.
- During object classification in parallel, information is extracted from the location data. The current zone in which the glove is present is saved and extrapolated to ten values. Each zone contains an object with its ID. The zone ID and its corresponding object are recorded in a look-up table. The ID of the object present in the zone is compared with the object class predictions, If the object class prediction matches with the object zone ID a high confidence score is given on the basis of number of correct matches among the 10 predictions of the object classification. If the object classification does not match with the zone ID, a confidence score of 0 is given.
- The "Async", library is used to process the functions of the sensor fusion algorithm in parallel rather than a sequential manner.
- The Async library handled location data retrieval, sensor data retrieval and model predictions in a parallel manner with handoff among the different functions.

Running the code:

- Go to Executable code folder
- If you do not have the "alisa conda environment", install the *"alisa_environment.yaml"* by running the command: *conda env create -f alisa_environment.yaml.*
- Run **python location_client_async_mod.py**
- This should display a GUI with the output of the actions performed and class detections.
- The config file consists of the model weights used is "6_classes_val_accuracy_increment_9_84_cnn_lstm.pt"
- Here you can change the model weights file for the model
- Only inference model file such as "config_lstm_cnn_multi_class_inference.yaml" should be used while running the code.
- In the main code, self.model_architecture can be changed to "cnn_lstm" or "inception". They the current best performing model architectures. The current one is set as "cnn_lstm". Be sure to change the model weights file accordingly and use an inception model weight file: "6_classes_val_accuracy_inception_multi_class_11_87.pt"